GUIA PRÁTICO EM PYTHON

Estruturas de Dados e Algoritmos

Da complexidade aos algoritmos de grafos e programação dinâmica: como pensar, implementar e explicar soluções para problemas difíceis.

Edição de novembro de 2026 · todo código testado

Sumário

Como usar este guia 7
Convenções 7
Como estudar 8
1 **Python para algoritmos** 9
Variáveis e tipos básicos 9
Operadores 9
Strings 11
Controle de fluxo 12
Listas em profundidade 13
Tuplas e desempacotamento 16
Dicionários e conjuntos (resumo) 17
Comprehensions 17
Funções 18
Classes 19
Funções embutidas essenciais 20
Simplificações: o jeito "pythônico" 20
Erros comuns e o que significam 22
2 **Fundamentos: complexidade, memória e recursão** 23
Notação Big-O 23
Custo das operações em Python 25
Memória: variáveis são referências 26
Recursão 27
Prática 29
3 **Arrays e strings** 30
O array por dentro 30
Técnica 1: Two Pointers (dois ponteiros) 31
Técnica 2: Sliding Window (janela deslizante) 34
Técnica 3: Prefix Sum (soma de prefixos) 36
Técnica 4: Kadane (subarray de soma máxima) 38
Técnica 5: Matrizes 38
Prática 39
4 **Tabelas hash: dict e set** 41
Como funciona 41
Ferramentas do Python que você deve dominar 41
Padrão 1: "já vi o complemento?" (Two Sum) 42
Padrão 2: agrupar por assinatura (Group Anagrams) 42
Padrão 3: set para busca O(1) (Longest Consecutive Sequence) 42

Padrão 4: contagem de frequência . . . 43
Padrão 5: hash de estado para detectar repetição . . . 43
Armadilhas . . . 44
Prática . . . 44

5 Listas ligadas . . . 45

Estrutura . . . 45
Operações básicas . . . 45
Reverter uma lista (problema 206) . . . 46
Ponteiros rápido e lento . . . 47
Remover o N-ésimo nó a partir do fim (problema 19) . . . 48
Mesclar duas listas ordenadas (problema 21) . . . 48
Somar dois números (problema 2) . . . 48
Combinando técnicas: Reorder List (problema 143) . . . 49
Estrutura avançada: LRU Cache (problema 146) . . . 50
Armadilhas . . . 52
Prática . . . 52

6 Pilhas, filas e estruturas monotônicas . . . 53

Pilha (stack): LIFO . . . 53
Fila (queue): FIFO . . . 55
Pilha monotônica . . . 55
Deque monotônico: Sliding Window Maximum (problema 239) . . . 57
Implementar fila com duas pilhas (problema 232) . . . 58
Prática . . . 59

7 Busca binária . . . 60

Ideia . . . 60
O template único . . . 60
Busca binária na resposta . . . 61
Arrays rotacionados . . . 62
Busca binária em matriz . . . 63
Difícil: mediana de dois arrays ordenados (problema 4) . . . 64
Armadilhas . . . 65
Prática . . . 65

8 Backtracking . . . 66

Ideia . . . 66
O template . . . 66
Subconjuntos (problema 78) . . . 66
Permutações (problema 46) . . . 67
Combination Sum (problema 39): reuso e poda . . . 68
Busca em grade: Word Search (problema 79) . . . 69
N-Rainhas (problema 51) . . . 69

Partição em palíndromos (problema 131) . . . 70
Sudoku Solver (problema 37): backtracking com retorno antecipado . . . 71
Armadilhas . . . 72
Prática . . . 72

9 Árvores binárias e BST . . . 73

Vocabulário . . . 73
O modelo mental de recursão em árvores . . . 73
Percursos (traversals) . . . 73
Padrão: retornar um valor e atualizar uma resposta global . . . 75
Informação de cima para baixo . . . 76
Árvore Binária de Busca (BST) . . . 77
Ancestral Comum Mais Baixo (LCA) . . . 78
Construir árvore a partir de percursos (problema 105) . . . 79
Serializar e desserializar (problema 297), difícil . . . 79
Complexidade em árvores . . . 80
Prática . . . 80

10 Heaps e filas de prioridade . . . 82

Ideia . . . 82
heapq em Python . . . 82
Implementação manual (entenda o sobe e desce) . . . 83
Padrão 1: Top K com heap de tamanho K . . . 84
Padrão 2: mesclar K sequências ordenadas (problema 23) . . . 85
Padrão 3: dois heaps (mediana em fluxo, problema 295) . . . 85
Padrão 4: simulação com heap (Task Scheduler, problema 621) . . . 86
Prática . . . 86

11 Grafos . . . 88

O que é um grafo . . . 88
Representações . . . 88
DFS: busca em profundidade . . . 89
BFS: busca em largura . . . 89
Grades como grafos . . . 90
Ciclos e ordenação topológica . . . 92
Union-Find (Disjoint Set Union) . . . 93
Caminho mínimo com pesos . . . 94
Árvore Geradora Mínima (MST) . . . 96
Grafo bipartido (problema 785) . . . 97
Qual algoritmo usar? . . . 98
Prática . . . 98

12 Programação dinâmica . . . 100

Ideia . . . 100

O roteiro em 5 passos . . . 100
Top-down vs bottom-up . . . 101
DP 1D . . . 102
DP 2D . . . 105
DP em intervalos . . . 108
DP com máquina de estados . . . 109
DP com bitmask (introdução) . . . 109
Como identificar o tipo de DP . . . 110
Armadilhas . . . 111
Prática (em ordem de dificuldade) . . . 111

13 Algoritmos gulosos e intervalos . . . 112

Ideia do guloso . . . 112
Jump Game (problemas 55 e 45) . . . 112
Gas Station (problema 134) . . . 113
Problemas de intervalos . . . 113
Outros gulosos clássicos . . . 116
Prática . . . 117

14 Estruturas avançadas: Trie, Fenwick e Segment Tree . . . 118

Trie (árvore de prefixos) . . . 118
Consultas de intervalo com atualizações . . . 120
Prática . . . 123

15 Strings, bits e matemática . . . 124

Busca de padrão em string . . . 124
Manipulação de bits . . . 125
Matemática que aparece . . . 127
Prática . . . 128

16 Ordenação e dividir para conquistar . . . 130

Ordenação em Python . . . 130
Ordenações simples: $O(n^2)$ . . . 130
Merge Sort . . . 132
Quick Sort e Quickselect . . . 133
Partição em 3 vias: Sort Colors (problema 75) . . . 135
Ordenações lineares (sem comparação) . . . 135
Resumo . . . 135
Prática . . . 136

17 Avançado II: grafos e árvores além do básico . . . 137

Prim: árvore geradora mínima a partir de um vértice . . . 137
BFS 0-1 . . . 138
DP em árvores . . . 139
Segment Tree com propagação preguiçosa (lazy) . . . 142

Componentes fortemente conexos (SCC) . . . 144
Pontes: Critical Connections (problema 1192), difícil . . . 145
Estruturas ordenadas (o substituto das árvores balanceadas) . . . 146
Prática . . . 147

18 Método para resolver e explicar problemas . . . 148

O roteiro em 6 passos . . . 148
Tabela: sinal no enunciado → técnica . . . 149
Como explicar uma solução . . . 150
Exemplo completo: Longest Increasing Path in a Matrix (problema 329) . . . 150
Checklist de bugs antes de enviar . . . 153
Plano de estudo sugerido (8 semanas) . . . 153
Referência rápida de Python para algoritmos . . . 154

Como usar este guia

Este material tem um objetivo: fazer você **resolver e explicar** problemas de algoritmos difíceis, no estilo LeetCode e entrevistas técnicas. Ele não é uma enciclopédia. Cada tópico existe porque aparece com frequência em problemas reais, e cada um é apresentado sempre na mesma ordem:

1. **Ideia**: o que a estrutura ou técnica é, em uma ou duas frases.
2. **Como funciona**: o mecanismo, com desenho da memória ou da execução.
3. **Complexidade**: custo de tempo e de espaço de cada operação, com o porquê.
4. **Quando usar**: os *sinais* no enunciado que indicam essa técnica.
5. **Código**: implementação em Python, comentada linha a linha quando necessário.
6. **Rastreio**: execução passo a passo em um exemplo pequeno.
7. **Armadilhas**: os erros mais comuns.
8. **Prática**: problemas do LeetCode para fixar.

Convenções

- Todo código é **Python 3.10+**. Os blocos de código completos terminam com `assert`s: eles foram **executados** e passam. Um bloco pode usar funções definidas em blocos anteriores **do mesmo capítulo** (por exemplo, o template `first_true` da busca binária). Copiando os blocos de um capítulo em ordem para um arquivo, tudo roda.
- Nos exemplos estilo LeetCode, `ListNode` e `TreeNode` têm as mesmas definições do site:

```python
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

- Nos testes também aparecem três funções auxiliares (não fazem parte das soluções):
  - `build_list([1, 2, 3])` monta a lista ligada `1 -> 2 -> 3` e devolve a cabeça;
  - `to_list(head)` faz o caminho inverso e devolve `[1, 2, 3]`;
  - `build_tree([3, 9, 20, None, None, 15, 7])` monta uma árvore a partir da notação em nível do LeetCode.
- `n` é o tamanho da entrada. Em grafos, `V` é o número de vértices e `E` o número de arestas.

Como estudar

O método que funciona

1. Leia a seção do tópico e **digite** o código, sem copiar e colar. Digitar força você a entender cada linha.
2. Faça o rastreio à mão, com papel, num exemplo pequeno.
3. Resolva os problemas de prática **sem olhar** a solução por pelo menos 25 minutos.
4. Depois de resolver, **explique em voz alta**: ideia, complexidade e por que funciona. Se você não consegue explicar, ainda não entendeu.
5. Revise o problema após 3 dias e após 1 semana (repetição espaçada).

Se você está começando em Python, leia primeiro o capítulo **"Python para algoritmos"**: ele cobre a sintaxe, listas, slicing (como `[::-1]`) e as simplificações usadas no resto do guia. A ordem dos capítulos é a ordem recomendada de estudo, porque cada capítulo usa ideias dos anteriores. O último capítulo, **"Método para resolver e explicar problemas"**, reúne tudo num roteiro e traz uma tabela de "sinal no enunciado → técnica". Vale lê-lo cedo e voltar a ele com frequência.

CAPÍTULO 1

Python para algoritmos

Este capítulo ensina **o Python que você precisa para resolver problemas**: sintaxe, os tipos principais, listas em profundidade, slicing, comprehensions, funções, classes e as **simplificações** que deixam o código curto e legível. Se você já programa em Python, use-o como referência; as tabelas de idiomas e de armadilhas no fim valem para todos.

Variáveis e tipos básicos

Python é de **tipagem dinâmica**: a variável não tem tipo; o **valor** tem. Não é preciso declarar nada.

```
n = 10              # int: inteiro de tamanho ilimitado (nunca estoura)
x = 3.5             # float: ponto flutuante (tem imprecisão!)
nome = "ana"        # str: texto (imutável)
ok = True           # bool: True ou False (com maiúscula)
nada = None         # None: "ausência de valor" (como null)

assert type(n) is int and type(nome) is str
assert 2 ** 100 == 1267650600228229401496703205376   # sem overflow
assert 0.1 + 0.2 != 0.3                               # imprecisão de float!
assert abs((0.1 + 0.2) - 0.3) < 1e-9                  # compare floats com tolerância
```

Type hints (anotações de tipo)

O LeetCode escreve as assinaturas com **anotações**. Elas são só documentação: o Python **não as verifica** ao executar.

```
from typing import Optional

def soma(a: int, b: int) -> int:          # recebe dois int, devolve int
    return a + b

def primeiro(nums: list[int]) -> Optional[int]:   # Optional[int] = int ou None
    return nums[0] if nums else None

assert soma(2, 3) == 5
assert primeiro([]) is None
```

`Optional[ListNode]` e `ListNode | None` significam a mesma coisa: "um `ListNode` ou `None`".

Operadores

Operador	Significado	Exemplo	Resultado
`+ - *`	soma, subtração, multiplicação	`7 * 2`	`14`

Operador	Significado	Exemplo	Resultado
`/`	divisão **real** (sempre float)	`7 / 2`	`3.5`
`//`	divisão **inteira** (arredonda para baixo)	`7 // 2`	`3`
`%`	resto	`7 % 2`	`1`
`**`	potência	`2 ** 10`	`1024`
`== !=`	igual, diferente (compara **valor**)	`[1] == [1]`	`True`
`is`	é o **mesmo objeto**?	`[1] is [1]`	`False`
`< <= > >=`	comparação (pode encadear)	`0 <= i < n`	—
`and or not`	lógicos	`not (a and b)`	—
`in`	pertence?	`3 in [1, 2, 3]`	`True`

```python
assert -7 // 2 == -4          # para BAIXO, não em direção a zero!
assert int(-7 / 2) == -3      # truncar em direção a zero
assert -7 % 3 == 2            # resto em Python é sempre >= 0 (com divisor positivo)
assert divmod(17, 5) == (3, 2)

i, n = 3, 5
assert 0 <= i < n             # comparação encadeada: 0 <= i and i < n

x = 5
x += 1; x *= 2; x //= 3       # operadores compostos
assert x == 4
```

Use `is` só com `None`

`x is None` é o correto para testar `None`. Para comparar números ou strings, use `==`. O `is` compara identidade, e pode "funcionar por acaso" com números pequenos e falhar com grandes.

Verdade e falsidade (truthiness)

Em um `if`, qualquer valor vira booleano. São **falsos**: `False`, `None`, `0`, `0.0`, `""`, `[]`, `{}`, `set()`, `()`. **Todo o resto é verdadeiro.**

```python
nums = []
if not nums:                  # forma idiomática de "lista vazia"
    vazio = True
assert vazio

node = None
assert (node and node.val) is None    # curto-circuito: não acessa node.val se node é None

nome = "" or "padrão"         # 'or' devolve o primeiro valor verdadeiro
assert nome == "padrão"
```

O **curto-circuito** é muito usado: em `a and b`, se `a` é falso, `b` nem é avaliado. Por isso `while fast and fast.next:` nunca acessa `.next` de `None`.

Strings

Strings são sequências **imutáveis** de caracteres: indexação e slicing funcionam como em listas, mas não dá para alterar uma posição.

```
s = "Hello, World"
assert s[0] == "H" and s[-1] == "d"     # índices negativos contam do fim
assert s[0:5] == "Hello"
assert len(s) == 12

# métodos mais usados
assert s.lower() == "hello, world" and s.upper() == "HELLO, WORLD"
assert "  oi  ".strip() == "oi"                       # remove espaços das pontas
assert "a,b,c".split(",") == ["a", "b", "c"]          # quebra em lista
assert "uma frase  qualquer".split() == ["uma", "frase", "qualquer"]  # sem argumento:
qualquer espaço
assert "-".join(["a", "b", "c"]) == "a-b-c"           # junta lista em string
assert s.replace("World", "Py") == "Hello, Py"
assert s.find("o") == 4 and s.find("z") == -1         # índice ou -1
assert s.startswith("Hell") and s.endswith("ld")
assert s.count("l") == 3
assert "abc123".isalnum() and "abc".isalpha() and "123".isdigit()

# caractere <-> código numérico
assert ord("a") == 97 and chr(97) == "a"
assert ord("c") - ord("a") == 2         # posição da letra no alfabeto (0 = 'a')

# f-strings: formatação
nome, idade = "Ana", 30
assert f"{nome} tem {idade} anos" == "Ana tem 30 anos"
assert f"{3.14159:.2f}" == "3.14"
assert "ab" * 3 == "ababab"
```

Strings são imutáveis

`s[0] = "h"` dá `TypeError`. Para "modificar", crie outra: `s = "h" + s[1:]`. Para muitas alterações, converta para lista (`chars = list(s)`), altere e volte com `"".join(chars)`.

Controle de fluxo

```python
def classifica(n: int) -> str:
    if n < 0:
        return "negativo"
    elif n == 0:
        return "zero"
    else:
        return "positivo"

assert classifica(-3) == "negativo" and classifica(0) == "zero"

# operador ternário (if em uma linha)
x = 7
paridade = "par" if x % 2 == 0 else "ímpar"
assert paridade == "ímpar"
```

`for` e `range`

`range(start, stop, step)` gera inteiros de `start` até `stop` (**excluído**), de `step` em `step`.

Chamada	Gera
`range(5)`	0, 1, 2, 3, 4
`range(2, 5)`	2, 3, 4
`range(0, 10, 3)`	0, 3, 6, 9
`range(4, -1, -1)`	4, 3, 2, 1, 0 (de trás para frente)
`range(n - 1, -1, -1)`	índices de uma lista de tamanho n, do último ao primeiro

```python
total = 0
for i in range(1, 4):          # 1, 2, 3
    total += i
assert total == 6

# percorrer elementos diretamente (preferível a range(len(...)))
for letra in "abc":
    pass

# while, break e continue
i = 0
while True:
    i += 1
    if i % 2:                  # ímpar: pula para a próxima volta
        continue
    if i > 6:
        break                  # sai do laço
assert i == 8

# for-else: o else roda se o laço terminou SEM break
def tem_negativo(nums):
    for x in nums:
        if x < 0:
```

```
            break
    else:
        return False          # não houve break
    return True

assert tem_negativo([1, -2]) and not tem_negativo([1, 2])
```

Listas em profundidade

A `list` é a estrutura mais usada em algoritmos: uma sequência **mutável** e ordenada, que aceita qualquer tipo de elemento.

Criar

```
a = [1, 2, 3]
vazia = []
zeros = [0] * 5                       # [0, 0, 0, 0, 0]
de_range = list(range(5))             # [0, 1, 2, 3, 4]
de_string = list("abc")               # ['a', 'b', 'c']
matriz = [[0] * 3 for _ in range(2)]  # 2 linhas x 3 colunas (forma CORRETA)

assert zeros == [0, 0, 0, 0, 0] and de_range == [0, 1, 2, 3, 4]
assert de_string == ["a", "b", "c"]
assert matriz == [[0, 0, 0], [0, 0, 0]]
```

Acessar e alterar

```
a = [10, 20, 30, 40, 50]
assert a[0] == 10 and a[-1] == 50 and a[-2] == 40
a[1] = 99                             # listas são mutáveis
assert a == [10, 99, 30, 40, 50]

m = [[1, 2, 3], [4, 5, 6]]
assert m[1][2] == 6                   # linha 1, coluna 2
assert len(m) == 2 and len(m[0]) == 3 # linhas e colunas
```

Acessar um índice que não existe (`a[10]`) levanta `IndexError: list index out of range`. É o erro mais comum em problemas de array.

Métodos

Método	Faz	Custo	Retorna
`a.append(x)`	adiciona `x` no fim	O(1)	`None`
`a.extend(b)`	adiciona todos de `b` no fim	O(len b)	`None`
`a.insert(i, x)`	insere `x` na posição `i`	O(n)	`None`
`a.pop()`	remove e devolve o último	O(1)	o elemento

Método	Faz	Custo	Retorna
`a.pop(i)`	remove e devolve o da posição `i`	O(n)	o elemento
`a.remove(x)`	remove a **primeira ocorrência** de `x`	O(n)	`None`
`a.index(x)`	posição da primeira ocorrência	O(n)	int
`a.count(x)`	quantas vezes `x` aparece	O(n)	int
`a.sort()`	ordena **a própria lista**	O(n log n)	`None`
`a.reverse()`	inverte **a própria lista**	O(n)	`None`
`a.copy()`	cópia rasa	O(n)	lista nova
`a.clear()`	esvazia	O(n)	`None`

```
a = [3, 1, 2]
a.append(4)
a.extend([5, 6])
a.insert(0, 0)
assert a == [0, 3, 1, 2, 4, 5, 6]
assert a.pop() == 6 and a.pop(0) == 0
a.remove(2)
assert a == [3, 1, 4, 5]
assert a.index(4) == 2
a.sort()
assert a == [1, 3, 4, 5]
a.sort(reverse=True)
assert a == [5, 4, 3, 1]
```

`sort()` e `reverse()` devolvem `None`

`a = a.sort()` apaga sua lista, porque `a` passa a ser `None`. Use `a.sort()` sozinho (altera a lista) ou `b = sorted(a)` (cria uma nova). O mesmo vale para `reversed(a)` (um iterador) e `a[::-1]` (uma lista nova) versus `a.reverse()` (altera e devolve `None`).

```
a = [3, 1, 2]
b = sorted(a)                # lista nova; a não muda
assert a == [3, 1, 2] and b == [1, 2, 3]
assert a.sort() is None      # altera a e devolve None
assert a == [1, 2, 3]
```

Slicing: `seq[start:stop:step]`

O slicing (fatiamento) extrai partes de listas, strings e tuplas. As três partes são opcionais:

- `start`: onde começa (incluso). Padrão: o início.
- `stop`: onde para (**excluído**). Padrão: o fim.

- `step`: o passo. Padrão: 1. **Se negativo, anda de trás para frente**, e os padrões de `start` e `stop` se invertem (começa no fim e vai até antes do início).

```
          0    1    2    3    4    5       ← índices positivos
a  =   [ 'p', 'y', 't', 'h', 'o', 'n' ]
         -6   -5   -4   -3   -2   -1       ← índices negativos
```

Slice	Resultado	Leitura
`a[2:5]`	`t h o`	do 2 até o 5 (excluído)
`a[:3]`	`p y t`	os 3 primeiros
`a[3:]`	`h o n`	do 3 até o fim
`a[-2:]`	`o n`	os 2 últimos
`a[:-2]`	`p y t h`	tudo menos os 2 últimos
`a[::2]`	`p t o`	de 2 em 2
`a[1::2]`	`y h n`	de 2 em 2, começando do 1
`a[::-1]`	`n o h t y p`	**invertido**
`a[4:1:-1]`	`o h t`	do 4 até o 1 (excluído), para trás
`a[:]`	tudo	**cópia rasa**

```python
a = list("python")
assert a[2:5] == ["t", "h", "o"]
assert a[::-1] == list("nohtyp")
assert a[4:1:-1] == ["o", "h", "t"]
assert a[10:] == []                    # slice fora do limite NÃO dá erro: devolve vazio

s = "arara"
assert s == s[::-1]                    # teste de palíndromo em uma linha

# atribuir a um slice (só em listas): substitui um trecho
b = [0, 1, 2, 3, 4]
b[1:3] = ["x", "y", "z"]
assert b == [0, "x", "y", "z", 3, 4]
del b[0:2]                             # remove um trecho
assert b == ["y", "z", 3, 4]
```

Lembre: **todo slice cria um objeto novo** e custa O(k), onde k é o tamanho do pedaço. `a[::-1]` cria uma lista invertida nova; `a.reverse()` inverte a própria lista sem criar outra.

Percorrer listas: os jeitos idiomáticos

```python
nomes = ["ana", "bia", "caio"]
idades = [30, 25, 41]

# só os valores
```

```python
for nome in nomes:
    pass

# índice + valor: enumerate (em vez de range(len(...)))
pares = [(i, nome) for i, nome in enumerate(nomes)]
assert pares == [(0, "ana"), (1, "bia"), (2, "caio")]
assert list(enumerate("ab", start=1)) == [(1, "a"), (2, "b")]

# duas listas em paralelo: zip (para no fim da menor)
assert list(zip(nomes, idades)) == [("ana", 30), ("bia", 25), ("caio", 41)]

# de trás para frente
assert list(reversed(nomes)) == ["caio", "bia", "ana"]

# pares de vizinhos: zip com a própria lista deslocada
a = [1, 3, 6, 10]
assert [y - x for x, y in zip(a, a[1:])] == [2, 3, 4]
```

Não altere uma lista enquanto a percorre

Remover elementos de `a` dentro de `for x in a` pula elementos, porque os índices se deslocam. Crie uma lista nova (`a = [x for x in a if cond(x)]`) ou percorra uma cópia (`for x in a[:]`).

Tuplas e desempacotamento

Uma **tupla** é como uma lista **imutável**: `(1, 2)`. Por ser imutável, pode ser chave de dicionário e elemento de `set`. Funções devolvem vários valores como tupla.

```python
p = (3, 4)
x, y = p                        # desempacotamento
assert x == 3 and y == 4

a, b = 1, 2
a, b = b, a                     # troca sem variável auxiliar
assert (a, b) == (2, 1)

primeiro, *resto = [1, 2, 3, 4]  # * captura "o resto" numa lista
assert primeiro == 1 and resto == [2, 3, 4]
*inicio, ultimo = [1, 2, 3]
assert ultimo == 3

def min_max(nums):
    return min(nums), max(nums)  # devolve uma tupla
lo, hi = min_max([4, 1, 9])
assert (lo, hi) == (1, 9)

for i, (nome, idade) in enumerate([("ana", 30), ("bia", 25)]):
    pass                        # desempacotamento aninhado

uma = (5,)                      # tupla de 1 elemento PRECISA da vírgula
assert type(uma) is tuple and type((5)) is int
```

Dicionários e conjuntos (resumo)

Detalhes no capítulo de tabelas hash. A sintaxe básica:

```
d = {"ana": 30, "bia": 25}
d["caio"] = 41                      # insere ou atualiza
assert d["ana"] == 30
assert d.get("zé") is None          # get não dá KeyError
assert d.get("zé", 0) == 0          # valor padrão
assert "bia" in d                   # testa CHAVE, em O(1)
del d["bia"]
assert list(d.keys()) == ["ana", "caio"]
assert list(d.values()) == [30, 41]
for chave, valor in d.items():      # percorre pares
    pass

s = {1, 2, 3}
s.add(4); s.discard(10)             # discard não dá erro se não existir
assert 2 in s and len(s) == 4
vazio = set()                       # {} é dict vazio, NÃO set!
assert type({}) is dict
```

Comprehensions

Uma forma compacta de **construir** listas, dicionários e conjuntos a partir de outra sequência.

```
[ expressão   for item in sequência   if condição ]
   o que vai     de onde vem            filtro (opcional)
   na lista
```

```
quadrados = [x * x for x in range(5)]
assert quadrados == [0, 1, 4, 9, 16]

pares = [x for x in range(10) if x % 2 == 0]
assert pares == [0, 2, 4, 6, 8]

# if-else NA EXPRESSÃO (transforma), diferente do if no fim (filtra)
rotulos = ["par" if x % 2 == 0 else "ímpar" for x in range(3)]
assert rotulos == ["par", "ímpar", "par"]

# dois for: a ordem é a mesma de laços aninhados
pares_ij = [(i, j) for i in range(2) for j in range(2)]
assert pares_ij == [(0, 0), (0, 1), (1, 0), (1, 1)]

# achatar matriz
m = [[1, 2], [3, 4]]
assert [x for linha in m for x in linha] == [1, 2, 3, 4]

# dict e set comprehension
pos = {v: i for i, v in enumerate("abc")}
assert pos == {"a": 0, "b": 1, "c": 2}
assert {x % 3 for x in range(10)} == {0, 1, 2}

# generator expression: sem colchetes, não cria lista (economiza memória)
```

```python
assert sum(x * x for x in range(4)) == 14
assert any(x > 2 for x in [1, 3]) and all(x > 0 for x in [1, 3])
```

O laço equivalente da primeira linha seria:

```python
quadrados = []
for x in range(5):
    quadrados.append(x * x)
```

Funções

```python
def saudacao(nome, cumprimento="Olá"):      # parâmetro com valor padrão
    return f"{cumprimento}, {nome}!"

assert saudacao("Ana") == "Olá, Ana!"
assert saudacao("Ana", cumprimento="Oi") == "Oi, Ana!"   # argumento nomeado

def soma_tudo(*nums):                        # *args: qualquer número de argumentos
    return sum(nums)
assert soma_tudo(1, 2, 3) == 6

# lambda: função anônima de uma expressão (muito usada em sort e key=)
dobro = lambda x: x * 2
assert dobro(4) == 8
assert sorted(["bb", "a", "ccc"], key=lambda s: len(s)) == ["a", "bb", "ccc"]

# função dentro de função: enxerga as variáveis de fora (closure)
def contador_de_chamadas():
    chamadas = 0
    def chama():
        nonlocal chamadas        # necessário para REATRIBUIR variável de fora
        chamadas += 1
        return chamadas
    return chama

c = contador_de_chamadas()
c(); c()
assert c() == 3
```

`nonlocal` em soluções do LeetCode

O padrão de função auxiliar interna (`def dfs(...)` dentro do método) aparece o tempo todo. A regra:

- **Ler** ou **modificar** um objeto de fora (`res.append(x)`, `visited.add(u)`): funciona direto.
- **Reatribuir** uma variável de fora (`best = max(best, x)`, `count += 1`): precisa de `nonlocal best`. Sem isso, dá `UnboundLocalError`.

```python
def maior_valor(nums):
    best = float("-inf")
    vistos = []
    def visita(x):
        nonlocal best           # reatribuição: precisa
        vistos.append(x)        # modificação de objeto: não precisa
        best = max(best, x)
    for x in nums:
        visita(x)
    return best, vistos

assert maior_valor([3, 9, 2]) == (9, [3, 9, 2])
```

Valor padrão mutável

`def f(x, acc=[])` cria **uma única lista** compartilhada entre todas as chamadas, e os valores "vazam" de uma chamada para a outra. Use `def f(x, acc=None)` e, dentro, `if acc is None: acc = []`.

Classes

O LeetCode usa classes em dois lugares: a casca `class Solution` e as estruturas (`ListNode`, `TreeNode`, problemas de *design* como LRU Cache).

```python
class Ponto:
    def __init__(self, x, y):        # construtor: roda ao criar Ponto(...)
        self.x = x                   # self = o próprio objeto
        self.y = y

    def dist2(self, outro):          # método: primeiro parâmetro é sempre self
        return (self.x - outro.x) ** 2 + (self.y - outro.y) ** 2

    def __repr__(self):              # como o objeto aparece no print
        return f"Ponto({self.x}, {self.y})"

    def __lt__(self, outro):         # define '<': permite sorted() e heapq
        return (self.x, self.y) < (outro.x, outro.y)

p, q = Ponto(0, 0), Ponto(3, 4)
assert p.dist2(q) == 25
assert repr(q) == "Ponto(3, 4)"
assert sorted([q, p])[0] is p
```

A estrutura de uma solução no LeetCode

```python
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        visto = {}
        for i, x in enumerate(nums):
            if target - x in visto:
                return [visto[target - x], i]
            visto[x] = i
```

O site cria `Solution()` e chama o método. O `self` é só a instância e você raramente o usa. Para testar no seu computador, chame `Solution().twoSum([2, 7, 11, 15], 9)`. Para chamar **outro método** da mesma classe, use `self.outro_metodo(...)`.

Funções embutidas essenciais

```
nums = [4, -1, 7, 3]
assert len(nums) == 4 and sum(nums) == 13
assert min(nums) == -1 and max(nums) == 7
assert max(nums, key=abs) == 7                    # máximo por um critério
assert min(["ana", "bo"], key=len) == "bo"
assert max([], default=0) == 0                    # evita erro em lista vazia
assert abs(-5) == 5 and round(2.567, 2) == 2.57
assert sorted(nums, reverse=True) == [7, 4, 3, -1]
assert sorted(["b", "A", "c"], key=str.lower) == ["A", "b", "c"]
assert list(map(int, ["1", "2"])) == [1, 2]       # aplica função a cada um
assert list(filter(lambda x: x > 0, nums)) == [4, 7, 3]
assert any(x < 0 for x in nums) and not all(x < 0 for x in nums)
assert int("42") == 42 and str(42) == "42" and float("1.5") == 1.5
assert int("101", 2) == 5 and bin(5) == "0b101"   # bases numéricas
assert isinstance(3, int)
assert float("inf") > 10**18 and float("-inf") < -10**18   # infinito
```

Simplificações: o jeito "pythônico"

Esta tabela mostra o código que iniciantes costumam escrever e a forma idiomática equivalente. As duas colunas funcionam; a da direita é mais curta, mais legível e, muitas vezes, mais rápida.

Objetivo	Jeito longo	Jeito idiomático
inverter	laço de trás para frente	`a[::-1]` ou `reversed(a)`
palíndromo	dois índices manuais	`s == s[::-1]`
último elemento	`a[len(a) - 1]`	`a[-1]`
lista vazia?	`if len(a) == 0:`	`if not a:`
índice + valor	`for i in range(len(a)): x = a[i]`	`for i, x in enumerate(a):`
duas listas juntas	`for i in range(len(a)): a[i], b[i]`	`for x, y in zip(a, b):`
trocar valores	`tmp = a; a = b; b = tmp`	`a, b = b, a`
contar frequência	dict + `if x in d`	`Counter(a)`
agrupar	`if k not in d: d[k] = []`	`defaultdict(list)`
valor padrão no dict	`d[k] if k in d else 0`	`d.get(k, 0)`

Objetivo	Jeito longo	Jeito idiomático
construir lista com laço	`res = []` + `append`	list comprehension
juntar strings	`s += c` em laço	`"".join(partes)`
intervalo	`x >= 0 and x < n`	`0 <= x < n`
máximo por critério	laço guardando o melhor	`max(a, key=...)`
alguém satisfaz?	laço com flag	`any(cond(x) for x in a)`
todos satisfazem?	laço com flag	`all(cond(x) for x in a)`
soma condicional	laço com `if`	`sum(1 for x in a if cond(x))`
transpor matriz	dois laços	`[list(l) for l in zip(*m)]`
anagramas	contar letra por letra	`sorted(s) == sorted(t)`
achatar matriz	dois laços	`[x for l in m for x in l]`
cópia de lista	laço copiando	`a[:]` ou `a.copy()`
remover duplicatas (mantendo ordem)	laço com lista auxiliar	`list(dict.fromkeys(a))`
prefix sum	laço acumulando	`list(accumulate(a))`
quociente e resto	`a // b` e `a % b`	`q, r = divmod(a, b)`
infinito	`999999999`	`float("inf")`

```python
from collections import Counter, defaultdict
from itertools import accumulate

a = [3, 1, 3, 2]
assert a[::-1] == [2, 3, 1, 3]
assert Counter(a)[3] == 2
assert list(dict.fromkeys(a)) == [3, 1, 2]
assert list(accumulate(a)) == [3, 4, 7, 9]
assert sum(1 for x in a if x > 1) == 3
assert [list(l) for l in zip(*[[1, 2], [3, 4]])] == [[1, 3], [2, 4]]
assert sorted("listen") == sorted("silent")
assert max(["ana", "beatriz", "bo"], key=len) == "beatriz"
g = defaultdict(list)
g["x"].append(1)
assert g == {"x": [1]}
```

Legível vence curto

Use os idiomas quando deixam o código **mais claro**. Uma list comprehension com três `for` e dois `if` é pior que um laço comum. Numa entrevista, o entrevistador precisa entender seu código.

Erros comuns e o que significam

Erro	Causa típica	Correção
`IndexError: list index out of range`	índice ≥ `len(a)`, ou lista vazia	confira os limites do laço; teste `if a:`
`KeyError: 'x'`	chave inexistente no dict	`d.get(k)`, `defaultdict`, ou `if k in d`
`TypeError: unhashable type: 'list'`	lista como chave de dict ou elemento de set	use `tuple(lista)`
`AttributeError: 'NoneType' object has no attribute 'next'`	acessou `.next` ou `.val` de `None`	verifique `if node:` antes
`UnboundLocalError`	reatribuiu variável de fora numa função interna	`nonlocal nome`
`RecursionError: maximum recursion depth`	recursão muito profunda (> ~1000)	versão iterativa ou `sys.setrecursionlimit`
`TypeError: '<' not supported`	heap ou sort com objetos não comparáveis	inclua um desempate na tupla, ou defina `__lt__`
`RuntimeError: dictionary changed size during iteration`	alterou o dict enquanto percorria	percorra `list(d)`
resultado `None` inesperado	`a = a.sort()`, ou função sem `return`	`a.sort()` sozinho; confira os `return`
todas as linhas da matriz mudam juntas	`[[0] * m] * n`	`[[0] * m for _ in range(n)]`

CAPÍTULO 2

Fundamentos: complexidade, memória e recursão

Antes de qualquer estrutura de dados, você precisa de três ferramentas mentais: medir o custo de um algoritmo (**complexidade**), entender o que uma variável realmente guarda (**referências**) e entender como uma função chama a si mesma (**recursão**).

Notação Big-O

Ideia

Big-O descreve **como o custo cresce** quando a entrada cresce. Não mede segundos, mede a *forma* do crescimento. Dizer que um algoritmo é O(n) significa: se a entrada dobra, o trabalho dobra (aproximadamente). Se é $O(n^2)$, dobrar a entrada **quadruplica** o trabalho.

Regras práticas para calcular

1. **Ignore constantes**: O(2n) = O(n). O(n/2) = O(n).
2. **Fique com o termo dominante**: $O(n^2 + n) = O(n^2)$. O(n + log n) = O(n).
3. **Laços em sequência somam**: um laço de `n` seguido de outro de `m` dá O(n + m).
4. **Laços aninhados multiplicam**: um laço de `n` dentro de um de `m` dá O(n · m).
5. **Entradas diferentes, variáveis diferentes**: se há duas listas, use `n` e `m`, não "n^2".
6. **Dividir pela metade a cada passo** custa O(log n). Quantas vezes dá para dividir 1.000.000 por 2 até chegar em 1? Cerca de 20 vezes, porque $\log_2(10^6) \approx 20$.

```python
def exemplo_1(a):          # O(n): um laço sobre n elementos
    total = 0
    for x in a:
        total += x
    return total

def exemplo_2(a):          # O(n²): para cada i, percorre todos os j
    pares = 0
    for i in range(len(a)):
        for j in range(len(a)):
            if a[i] + a[j] == 0:
                pares += 1
    return pares

def exemplo_3(n):          # O(log n): n cai pela metade a cada volta
    passos = 0
    while n > 1:
        n //= 2
        passos += 1
```

```
    return passos

assert exemplo_1([1, 2, 3]) == 6
assert exemplo_2([1, -1]) == 2          # (0,1) e (1,0)
assert exemplo_3(1_000_000) == 19
```

Escala de complexidades

Complexidade	Nome	Exemplo típico	n = 10^6 (operações aprox.)
O(1)	constante	acessar `a[i]`, `x in set`	1
O(log n)	logarítmica	busca binária	20
O(n)	linear	percorrer uma lista	10^6
O(n log n)	linearítmica	ordenar (`sorted`)	$2 \cdot 10^7$
O(n^2)	quadrática	dois laços aninhados	10^{12} (inviável)
O(2^n)	exponencial	todos os subconjuntos	inviável para n > 30
O(n!)	fatorial	todas as permutações	inviável para n > 11

Usando as restrições do enunciado para descobrir a complexidade esperada

Este é um dos truques mais úteis do LeetCode. Python faz cerca de **10^7 operações simples por segundo**, e o limite de tempo costuma ser de 1 a 2 segundos. Então o tamanho máximo de `n` no enunciado *diz* qual complexidade é aceitável:

Restrição	Complexidade que cabe	Técnicas prováveis
n ≤ 10	O(n!) ou O($2^n \cdot n$)	backtracking, permutações
n ≤ 20	O(2^n)	subconjuntos, bitmask
n ≤ 500	O(n^3)	DP em intervalos, Floyd-Warshall
n ≤ 5.000	O(n^2)	DP 2D, dois laços
n ≤ 10^5 a 10^6	O(n log n) ou O(n)	ordenação, heap, busca binária, two pointers, hash
n ≤ 10^9 ou mais	O(log n) ou O(1)	busca binária, matemática

Leia as restrições primeiro

Se o enunciado diz `n ≤ 10^5`, uma solução O(n^2) (10^{10} operações) **vai estourar o tempo**. Você precisa de O(n log n) ou melhor, e isso já elimina metade das ideias antes de você começar.

Complexidade de espaço

Mede a memória **extra** que o algoritmo usa, além da entrada. Uma lista auxiliar de tamanho `n` custa O(n). Um dicionário com até `n` chaves custa O(n). A **pilha de recursão** também conta: uma recursão com profundidade `n` usa O(n) de espaço, mesmo sem criar nenhuma lista.

Análise amortizada

Algumas operações são *quase sempre* baratas e *raramente* caras. O exemplo clássico é `list.append` do Python. A lista reserva espaço extra; quando enche, aloca um bloco maior (cerca de 1,125×, mais uma folga) e copia tudo, o que custa O(n). Isso acontece tão raramente que, **na média ao longo de muitas operações**, cada `append` custa O(1). Dizemos que `append` é **O(1) amortizado**.

Custo das operações em Python

Saber isso evita muitos estouros de tempo "misteriosos":

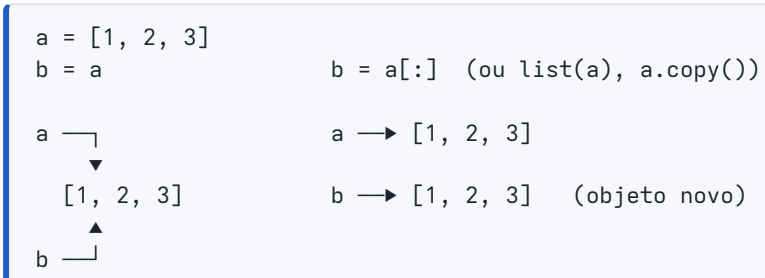
Operação	`list`	`dict` / `set`	`collections.deque`
acessar por índice `a[i]`	O(1)	—	O(n) no meio
`append` / `pop()` no fim	O(1) amort.	—	O(1)
`insert(0, x)` / `pop(0)` no início	**O(n)**	—	O(1) (`appendleft` / `popleft`)
`x in estrutura`	**O(n)**	**O(1)** médio	O(n)
inserir / remover chave	—	O(1) médio	—
fatiar `a[i:j]`	O(j − i) (cria cópia)	—	—
`sorted(a)` / `a.sort()`	O(n log n)	—	—
`len(a)`	O(1)	O(1)	O(1)
`min(a)` / `max(a)` / `sum(a)`	O(n)	O(n)	O(n)

Os três vilões de desempenho

- `lista.pop(0)` dentro de um laço transforma um BFS O(n) em O(n²). Use `deque.popleft()`.
- `if x in lista` dentro de um laço gera O(n²). Converta a lista em `set` antes.
- Concatenar strings com `s = s + c` em laço pode virar O(n²). Junte partes numa lista e use `"".join(partes)` no fim.

Memória: variáveis são referências

Em Python, **toda variável é uma etiqueta que aponta para um objeto**. Atribuir (`b = a`) não copia nada: cria uma segunda etiqueta para o *mesmo* objeto.




```python
a = [1, 2, 3]
b = a            # mesma lista
b.append(4)
assert a == [1, 2, 3, 4]      # a "mudou" porque a e b são o mesmo objeto
assert a is b

c = a[:]         # cópia rasa: lista nova
c.append(5)
assert a == [1, 2, 3, 4]      # a não mudou
assert a is not c
```

Existem duas operações diferentes que parecem iguais:

- **Modificar o objeto** (`b.append(4)`, `node.val = 9`, `node.next = outro`): todas as etiquetas que apontam para ele "veem" a mudança.
- **Reatribuir a etiqueta** (`b = outra_coisa`, `curr = curr.next`): só aquela etiqueta passa a apontar para outro lugar. O objeto antigo e as outras etiquetas não mudam.

Essa distinção é **a** chave para entender listas ligadas e árvores. Quando você faz `curr = head` e depois `curr = curr.next`, o `head` continua no começo. Quando você faz `curr.next = None`, a lista de fato é cortada.

Armadilha clássica: matriz com multiplicação de lista

`grid = [[0] * 3] * 3` cria **uma** linha e três referências para ela. Alterar `grid[0][0]` altera as três linhas. O correto é `grid = [[0] * 3 for _ in range(3)]`.

```python
errado = [[0] * 3] * 3
errado[0][0] = 1
assert errado == [[1, 0, 0], [1, 0, 0], [1, 0, 0]]   # as 3 linhas são a mesma!

certo = [[0] * 3 for _ in range(3)]
certo[0][0] = 1
assert certo == [[1, 0, 0], [0, 0, 0], [0, 0, 0]]
```

Cópia rasa e cópia profunda

`a[:]`, `list(a)` e `a.copy()` criam uma lista nova, mas os **elementos** continuam sendo os mesmos objetos. Se os elementos forem listas, as listas internas são compartilhadas. Para copiar tudo recursivamente, use `copy.deepcopy`. Em algoritmos, o caso mais comum é o backtracking: ao salvar o caminho atual na resposta, faça `res.append(path[:])`, e não `res.append(path)`. Senão você salva uma referência que continuará sendo modificada (ver o capítulo de backtracking).

Argumentos de função

Python passa **a referência** para a função. Se a função *modifica* o objeto recebido, quem chamou vê a mudança. Se a função *reatribui* o parâmetro, quem chamou não vê.

```
def modifica(lst):
    lst.append(99)        # altera o objeto compartilhado

def reatribui(lst):
    lst = [0]             # só muda a etiqueta local

x = [1]
modifica(x)
assert x == [1, 99]
reatribui(x)
assert x == [1, 99]       # não mudou
```

Recursão

Ideia

Uma função recursiva resolve um problema **usando a resposta de versões menores do mesmo problema**. Toda recursão correta tem duas partes:

1. **Caso base**: a entrada é tão pequena que a resposta é direta (lista vazia, `n == 0`, nó `None`).
2. **Passo recursivo**: reduz o problema e *confia* que a chamada recursiva devolve a resposta certa para a parte menor.

> **O salto de fé**
>
> Não tente simular todas as chamadas na cabeça. Pergunte apenas: *"se a chamada recursiva devolver a resposta certa para o problema menor, consigo montar a resposta do problema atual?"* Se sim e o caso base está certo, a função está correta. Isso é indução matemática.

```python
def soma(a, i=0):
    if i == len(a):          # caso base: nada a somar
        return 0
    return a[i] + soma(a, i + 1)   # elemento atual + soma do resto

def fatorial(n):
    if n <= 1:
        return 1
    return n * fatorial(n - 1)

assert soma([1, 2, 3, 4]) == 10
assert fatorial(5) == 120
```

A pilha de chamadas (call stack)

Cada chamada cria um **quadro** (frame) na pilha, com suas próprias variáveis locais. O quadro só sai da pilha quando a função retorna:

```
fatorial(4)
| espera 4 * fatorial(3)
|    | espera 3 * fatorial(2)
|    |    | espera 2 * fatorial(1)
|    |    |    └ fatorial(1) = 1      ← caso base, começa a "voltar"
|    |    └ 2 * 1 = 2
|    └ 3 * 2 = 6
└ 4 * 6 = 24
```

A profundidade máxima da pilha é o **espaço** da recursão: O(n) para `fatorial(n)`.

> **Limite de recursão do Python**
>
> Por padrão, o Python permite cerca de 1000 chamadas aninhadas e depois levanta `RecursionError`. Em listas ligadas ou árvores degeneradas com 10^5 nós, uma recursão ingênua quebra. Soluções: (1) escrever a versão iterativa com pilha explícita; (2) `sys.setrecursionlimit(10**6)`, o que funciona no LeetCode na maioria dos casos, mas pode estourar a pilha do sistema.

Como analisar a complexidade de uma recursão

Use a **árvore de recursão**: custo total = (número de chamadas) × (trabalho por chamada).

- `soma(a)`: n chamadas, O(1) cada, total **O(n)**.
- `fib(n) = fib(n-1) + fib(n-2)` ingênuo: cada chamada gera duas, a árvore tem cerca de 2^n nós, total **$O(2^n)$**. Com memoização cai para O(n) (capítulo de programação dinâmica).
- Merge sort: log n níveis, cada nível faz O(n) de trabalho, total **O(n log n)**.

Teorema Mestre (versão prática)

Para recorrências do tipo $T(n) = a \cdot T(n / b) + O(n^d)$, onde `a` é o número de subchamadas, `n/b` o tamanho de cada uma e n^d o trabalho fora delas:

Caso	Condição	Resultado	Exemplo
1	$a < b^d$	$O(n^d)$	—
2	$a = b^d$	$O(n^d \log n)$	merge sort: a=2, b=2, d=1, dá O(n log n)
3	$a > b^d$	O(n^(log_b a))	—

Busca binária: a=1, b=2, d=0, então $1 = 2^0$ (caso 2) e o resultado é O(log n).

Prática

Problema	Ideia
509. Fibonacci Number	recursão e depois memoização
344. Reverse String	recursão com dois índices
50. Pow(x, n)	exponenciação rápida: x^n = (x^(n/2))², O(log n)

CAPÍTULO 3

Arrays e strings

O array por dentro

Um array é um bloco **contíguo** de memória: os elementos ficam lado a lado. Por isso acessar `a[i]` é O(1). O endereço é calculado diretamente: `início + i × tamanho_do_elemento`. A `list` do Python é um array dinâmico de *referências* para objetos.



Consequências:

- **Ler ou escrever** em `a[i]`: O(1).
- **Inserir ou remover no meio ou no início**: O(n), porque todos os elementos seguintes precisam ser deslocados.
- **Inserir ou remover no fim**: O(1) amortizado.

Strings em Python são **imutáveis**: `s[0] = "x"` dá erro, e toda "modificação" cria uma string nova. Para construir strings em laço, use uma lista e `"".join(...)`.

Slicing: `seq[start:stop:step]`

Expressão	Resultado com `a = [0, 1, 2, 3, 4]`	Leitura
`a[1:4]`	`[1, 2, 3]`	do índice 1 até o 4 (excluído)
`a[:2]`	`[0, 1]`	os 2 primeiros
`a[-2:]`	`[3, 4]`	os 2 últimos
`a[::2]`	`[0, 2, 4]`	de 2 em 2
`a[::-1]`	`[4, 3, 2, 1, 0]`	invertido
`a[:]`	`[0, 1, 2, 3, 4]`	cópia rasa

Todo slice **cria uma lista nova**, o que custa O(k) para k elementos copiados. Fatiar dentro de uma recursão (`f(a[1:])`) adiciona O(n) por chamada; prefira passar índices.

Técnica 1: Two Pointers (dois ponteiros)

Ideia

Usar dois índices que andam pela sequência de forma coordenada, eliminando a necessidade de um laço aninhado. Transforma muitas soluções $O(n^2)$ em $O(n)$.

Há dois formatos principais:

```
Formato A, em direções opostas (array ordenado, palíndromos):
  L →                          ← R
  [1,  2,  4,  6,  8,  11,  15]

Formato B, mesma direção, rápido e lento (compactar, remover):
  slow
  fast →
  [0,  0,  1,  1,  1,  2,  3]
```

Sinais no enunciado

- O array está **ordenado**, ou pode ser ordenado sem perder a resposta.
- Procura **par** ou **trinca** com soma alvo.
- Pede para modificar o array **in-place** (remover duplicatas, mover zeros).
- Palíndromo e comparação das pontas.

Exemplo A: Two Sum II (array ordenado)

Problema (167): dado um array ordenado e um `target`, retorne os índices (base 1) de dois números que somam `target`.

Por que funciona: com `L` no início e `R` no fim, olhe a soma `a[L] + a[R]`:

- Se a soma é **pequena demais**, o único jeito de aumentá-la é mover `L` para a direita. Mover `R` só diminuiria.
- Se é **grande demais**, mova `R` para a esquerda.
- Cada passo **descarta um elemento com certeza**. Se a soma é pequena, `a[L]` com *qualquer* outro elemento restante (todos ≤ `a[R]`) também é pequeno, então `a[L]` nunca faz parte da resposta.

```python
def two_sum_sorted(nums: list[int], target: int) -> list[int]:
    L, R = 0, len(nums) - 1
    while L < R:
        s = nums[L] + nums[R]
        if s == target:
            return [L + 1, R + 1]
        if s < target:
            L += 1          # precisa de soma maior
        else:
            R -= 1          # precisa de soma menor
    return []

assert two_sum_sorted([2, 7, 11, 15], 9) == [1, 2]
assert two_sum_sorted([1, 3, 4, 6, 8, 11], 10) == [3, 4]
```

Rastreio com `[1, 3, 4, 6, 8, 11]` e target = 10:

L	R	a[L] + a[R]	Ação
0	5	1 + 11 = 12	grande, R−−
0	4	1 + 8 = 9	pequena, L++
1	4	3 + 8 = 11	grande, R−−
1	3	3 + 6 = 9	pequena, L++
2	3	4 + 6 = 10	achou: [3, 4]

Complexidade: O(n) de tempo e O(1) de espaço.

Exemplo B: 3Sum (problema 15)

Encontre todas as trincas **únicas** com soma zero. Estratégia: **ordene**, fixe o primeiro elemento `i` e rode Two Sum com dois ponteiros no resto. O ponto difícil são as **duplicatas**: pule valores iguais ao anterior, tanto para `i` quanto para `L` depois de achar uma trinca.

```python
def three_sum(nums: list[int]) -> list[list[int]]:
    nums.sort()
    res = []
    n = len(nums)
    for i in range(n - 2):
        if nums[i] > 0:                      # tudo à direita é ≥ nums[i] > 0
            break
        if i > 0 and nums[i] == nums[i - 1]:  # mesmo 1º elemento já foi testado
            continue
        L, R = i + 1, n - 1
        while L < R:
            s = nums[i] + nums[L] + nums[R]
            if s < 0:
                L += 1
            elif s > 0:
                R -= 1
            else:
                res.append([nums[i], nums[L], nums[R]])
                L += 1
```

```
                R -= 1
                while L < R and nums[L] == nums[L - 1]:   # pula duplicatas
                    L += 1
    return res

assert three_sum([-1, 0, 1, 2, -1, -4]) == [[-1, -1, 2], [-1, 0, 1]]
assert three_sum([0, 0, 0, 0]) == [[0, 0, 0]]
```

Complexidade: O(n log n) da ordenação mais O(n²) dos laços, total **O(n²)**. É o melhor possível na prática para 3Sum.

Exemplo C: Container With Most Water (problema 11)

Dadas alturas `h`, escolha duas linhas que formem o recipiente com mais água: `área = min(h[L], h[R]) × (R - L)`.

O argumento de corretude (é isso que se explica numa entrevista): comece com a largura máxima. A área é limitada pela linha **mais baixa**. Se você mover a linha mais alta, a largura diminui e a altura mínima não pode aumentar, então a área **só pode piorar**. Logo, a única esperança é mover a linha mais baixa.

```
def max_area(h: list[int]) -> int:
    L, R = 0, len(h) - 1
    best = 0
    while L < R:
        best = max(best, min(h[L], h[R]) * (R - L))
        if h[L] < h[R]:
            L += 1
        else:
            R -= 1
    return best

assert max_area([1, 8, 6, 2, 5, 4, 8, 3, 7]) == 49
```

Exemplo D: ponteiro lento e rápido (remover duplicatas in-place)

`slow` marca a próxima posição de escrita, e `fast` lê todos os elementos.

```
def remove_duplicates(nums: list[int]) -> int:
    if not nums:
        return 0
    slow = 1                          # nums[0] sempre fica
    for fast in range(1, len(nums)):
        if nums[fast] != nums[fast - 1]:
            nums[slow] = nums[fast]
            slow += 1
    return slow                       # tamanho da parte válida

a = [0, 0, 1, 1, 1, 2, 2, 3, 3, 4]
k = remove_duplicates(a)
assert k == 5 and a[:k] == [0, 1, 2, 3, 4]
```

Técnica 2: Sliding Window (janela deslizante)

Ideia

Manter uma **janela** `[L, R]` contígua sobre o array ou a string. `R` expande a janela e `L` encolhe. Em vez de recalcular tudo para cada subarray (O(n²) ou pior), você **atualiza incrementalmente**: adiciona o elemento que entra e remove o que sai.

Sinais no enunciado

- "**subarray** / **substring** contígua" combinado com "maior", "menor", "mais longa" ou "contém".
- Restrição do tipo "no máximo K distintos", "sem repetir", "soma ≥ alvo".

Os dois tipos de janela

Janela fixa (tamanho k): a cada passo, entra `a[R]` e sai `a[R - k]`.

```python
def max_sum_k(a: list[int], k: int) -> int:
    window = sum(a[:k])
    best = window
    for R in range(k, len(a)):
        window += a[R] - a[R - k]     # entra um, sai outro
        best = max(best, window)
    return best

assert max_sum_k([2, 1, 5, 1, 3, 2], 3) == 9   # [5, 1, 3]
```

Janela variável: o template universal. Quase todo problema de janela variável segue este molde:

```python
L = 0
for R in range(len(s)):
    # 1. adiciona s[R] ao estado da janela
    while janela_invalida():
        # 2. remove s[L] do estado
        L += 1
    # 3. a janela [L, R] é válida: atualiza a resposta
    resposta = max(resposta, R - L + 1)
```

Por que é O(n) mesmo com um `while` dentro do `for`: cada índice entra na janela uma vez (via `R`) e sai no máximo uma vez (via `L`). `L` nunca volta. O total de movimentos é ≤ 2n.

Exemplo: Longest Substring Without Repeating Characters (problema 3)

```python
def length_of_longest_substring(s: str) -> int:
    seen = set()                # caracteres dentro da janela atual
    L = best = 0
    for R, c in enumerate(s):
        while c in seen:        # janela inválida: c repetiria
            seen.remove(s[L])
            L += 1
        seen.add(c)
        best = max(best, R - L + 1)
    return best

assert length_of_longest_substring("abcabcbb") == 3   # "abc"
assert length_of_longest_substring("bbbbb") == 1
assert length_of_longest_substring("pwwkew") == 3     # "wke"
assert length_of_longest_substring("") == 0
```

Rastreio de `"abcabcbb"` (primeiros passos):

R	c	Ação	Janela	best
0	a	adiciona	`a`	1
1	b	adiciona	`ab`	2
2	c	adiciona	`abc`	3
3	a	`a` repetido: remove `a` (L=1), adiciona	`bca`	3
4	b	`b` repetido: remove `b` (L=2), adiciona	`cab`	3

Otimização: guarde o último índice de cada caractere num dict e pule `L` direto para `último[c] + 1`:

```python
def length_of_longest_substring_v2(s: str) -> int:
    last = {}
    L = best = 0
    for R, c in enumerate(s):
        if c in last and last[c] >= L:   # c está dentro da janela
            L = last[c] + 1
        last[c] = R
        best = max(best, R - L + 1)
    return best

assert length_of_longest_substring_v2("abba") == 2
assert length_of_longest_substring_v2("abcabcbb") == 3
```

> **A condição `last[c] >= L`**
>
> Sem ela, `"abba"` falha: no último `a`, `last['a'] = 0` está **fora** da janela (L=2), e `L` voltaria para 1. Lembre: `L` nunca pode andar para trás.

Exemplo difícil: Minimum Window Substring (problema 76)

Menor substring de `s` que contém todos os caracteres de `t`, com repetições. Aqui a janela é "válida" quando cobre `t`, e queremos **encolher** enquanto ela continua válida.

Truque: em vez de comparar dois contadores inteiros a cada passo (custo de 26 a 52 por comparação), mantenha `faltam` = quantos caracteres de `t` ainda não estão cobertos.

```
from collections import Counter

def min_window(s: str, t: str) -> str:
    need = Counter(t)          # quanto ainda preciso de cada caractere
    faltam = len(t)
    L = 0
    best = (float("inf"), 0, 0)
    for R, c in enumerate(s):
        if need[c] > 0:        # c era necessário
            faltam -= 1
        need[c] -= 1           # pode ficar negativo = excesso na janela
        while faltam == 0:     # janela cobre t: tenta encolher
            if R - L + 1 < best[0]:
                best = (R - L + 1, L, R)
            need[s[L]] += 1
            if need[s[L]] > 0: # removi algo necessário
                faltam += 1
            L += 1
    size, i, j = best
    return "" if size == float("inf") else s[i : j + 1]

assert min_window("ADOBECODEBANC", "ABC") == "BANC"
assert min_window("a", "aa") == ""
```

Complexidade: O(|s| + |t|) de tempo e O(alfabeto) de espaço.

Técnica 3: Prefix Sum (soma de prefixos)

Ideia

Pré-calcular `P[i] = a[0] + a[1] + ... + a[i-1]`. Então a soma de qualquer intervalo `a[i..j]` vira uma subtração O(1):

```
a = [3,  1,  4,  1,  5]
P = [0,  3,  4,  8,  9, 14]       (P tem n+1 posições; P[0] = 0)

soma(a[1..3]) = P[4] - P[1] = 9 - 3 = 6     (1 + 4 + 1)
```

```python
from itertools import accumulate

a = [3, 1, 4, 1, 5]
P = [0] + list(accumulate(a))
def range_sum(i, j):           # soma de a[i..j], inclusive
    return P[j + 1] - P[i]

assert P == [0, 3, 4, 8, 9, 14]
assert range_sum(1, 3) == 6
assert range_sum(0, 4) == 14
```

Prefix sum + hash: Subarray Sum Equals K (problema 560)

Conte quantos subarrays somam `k`. Os números podem ser negativos, então janela deslizante **não funciona** (a soma não é monótona).

Raciocínio: a soma de `a[i..j]` é `P[j+1] - P[i]`. Queremos `P[j+1] - P[i] = k`, ou seja, `P[i] = P[j+1] - k`. Ao percorrer o array, para cada prefixo atual basta perguntar: *"quantos prefixos anteriores valiam `atual - k`?"*. Um dict de contagens responde isso em O(1).

```python
from collections import defaultdict

def subarray_sum(nums: list[int], k: int) -> int:
    count = defaultdict(int)
    count[0] = 1               # prefixo vazio: permite subarrays que começam no índice 0
    prefix = res = 0
    for x in nums:
        prefix += x
        res += count[prefix - k]
        count[prefix] += 1
    return res

assert subarray_sum([1, 1, 1], 2) == 2
assert subarray_sum([1, 2, 3], 3) == 2          # [1,2] e [3]
assert subarray_sum([1, -1, 0], 0) == 3
```

> **Padrão importante**
>
> "Contar subarrays com propriedade X" com números negativos quase sempre significa **prefix sum + dicionário**. Variações: soma divisível por k (guarde `prefix % k`), número igual de 0s e 1s (troque 0 por −1 e procure soma 0).

Produto exceto a si mesmo (problema 238)

Prefixos não servem só para somas. `res[i]` = (produto de tudo à esquerda de i) × (produto de tudo à direita de i), sem usar divisão:

```python
def product_except_self(nums: list[int]) -> list[int]:
    n = len(nums)
    res = [1] * n
    left = 1
    for i in range(n):            # res[i] = produto à esquerda
        res[i] = left
        left *= nums[i]
    right = 1
    for i in range(n - 1, -1, -1):  # multiplica pelo produto à direita
        res[i] *= right
        right *= nums[i]
    return res

assert product_except_self([1, 2, 3, 4]) == [24, 12, 8, 6]
```

Técnica 4: Kadane (subarray de soma máxima)

Ideia

Percorra o array mantendo `cur` = maior soma de subarray que **termina na posição atual**. Em cada posição só há duas opções: estender o subarray anterior (`cur + x`) ou começar um novo em `x`. Se `cur` ficou negativo, ele só atrapalha, então recomece.

```python
def max_subarray(nums: list[int]) -> int:
    cur = best = nums[0]
    for x in nums[1:]:
        cur = max(x, cur + x)     # estender ou recomeçar
        best = max(best, cur)
    return best

assert max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4]) == 6   # [4, -1, 2, 1]
assert max_subarray([-3, -1, -2]) == -1
```

Isso já é **programação dinâmica**: `cur` é o estado "melhor terminando em i", e `best` é o máximo sobre todos os estados. Variação: no produto máximo (problema 152), guarde também o **mínimo**, porque negativo × negativo vira o maior.

Técnica 5: Matrizes

Percorrer vizinhos em grade aparece o tempo todo (ilhas, labirintos, BFS). Use um vetor de direções e cheque os limites:

```python
DIRS = [(1, 0), (-1, 0), (0, 1), (0, -1)]

def vizinhos(grid, r, c):
    R, C = len(grid), len(grid[0])
    for dr, dc in DIRS:
        nr, nc = r + dr, c + dc
        if 0 <= nr < R and 0 <= nc < C:
            yield nr, nc

g = [[1, 2], [3, 4]]
assert sorted(vizinhos(g, 0, 0)) == [(0, 1), (1, 0)]
```

Rotacionar uma matriz 90° no sentido horário, in-place (problema 48): transponha (troque `m[i][j]` com `m[j][i]`) e depois inverta cada linha.

```python
def rotate(m: list[list[int]]) -> None:
    n = len(m)
    for i in range(n):
        for j in range(i + 1, n):
            m[i][j], m[j][i] = m[j][i], m[i][j]
    for row in m:
        row.reverse()

m = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
rotate(m)
assert m == [[7, 4, 1], [8, 5, 2], [9, 6, 3]]
```

Prática

Problema	Técnica
125. Valid Palindrome	two pointers opostos
167. Two Sum II	two pointers opostos
15. 3Sum	ordenar + two pointers
11. Container With Most Water	two pointers + argumento guloso
42. Trapping Rain Water	two pointers com máximos (difícil)
283. Move Zeroes	rápido e lento
3. Longest Substring Without Repeating	janela variável
424. Longest Repeating Character Replacement	janela: `tamanho - maior_freq ≤ k`
76. Minimum Window Substring	janela + contador (difícil)
560. Subarray Sum Equals K	prefix sum + hash
238. Product of Array Except Self	prefixo e sufixo
53. Maximum Subarray	Kadane

Problema	Técnica
54. Spiral Matrix	simulação com limites

CAPÍTULO 4

Tabelas hash: `dict` e `set`

Como funciona

Uma tabela hash guarda pares chave → valor num array. Para descobrir **em qual posição** guardar uma chave, ela aplica uma **função hash** (`hash(chave)`), que transforma a chave num inteiro, e usa esse inteiro módulo o tamanho do array:

```
hash("ana")   % 8 = 3        posição:  0    1    2    3       4    5       6    7
hash("bia")   % 8 = 6                [ ]  [ ]  [ ]  [ana:7] [ ]  [ ]     [bia:2] [ ]
hash("caio")  % 8 = 3  ← colisão!     resolve sondando outra posição livre
```

- **Busca, inserção e remoção** custam O(1) em média: calcula o hash e vai direto na posição.
- **Colisões** (duas chaves no mesmo lugar) são resolvidas sondando outras posições. Com uma boa função hash e redimensionamento automático (quando a tabela fica cerca de 2/3 cheia), as colisões são raras. O **pior caso** é O(n), mas na prática você trata como O(1).
- As chaves precisam ser **hasheáveis** (imutáveis): `int`, `str` e `tuple` de imutáveis servem. `list`, `dict` e `set` **não**. Para usar uma lista como chave, converta: `tuple(lista)`.
- `set` é uma tabela hash só de chaves, sem valores.

Ferramentas do Python que você deve dominar

```python
from collections import Counter, defaultdict

# Counter: conta frequências
c = Counter("banana")
assert c["a"] == 3 and c["z"] == 0          # chave ausente vale 0, sem KeyError
assert c.most_common(1) == [("a", 3)]

# defaultdict: cria valor padrão automaticamente
grupos = defaultdict(list)
for palavra in ["ana", "bia", "alan"]:
    grupos[palavra[0]].append(palavra)      # sem precisar testar "if chave in"
assert grupos["a"] == ["ana", "alan"]

# dict.get com padrão
d = {}
d["x"] = d.get("x", 0) + 1
assert d == {"x": 1}

# operações de conjunto
a, b = {1, 2, 3}, {2, 3, 4}
assert a & b == {2, 3}       # interseção
assert a | b == {1, 2, 3, 4} # união
assert a - b == {1}          # diferença
```

Padrão 1: "já vi o complemento?" (Two Sum)

Problema 1: retorne os índices de dois números que somam `target`. O array **não** está ordenado.

Força bruta: testar todos os pares, O(n²). Com hash: para cada `x`, o par que procuro é `target - x`. Pergunto ao dict se ele **já apareceu**, em O(1).

```python
def two_sum(nums: list[int], target: int) -> list[int]:
    visto = {}                      # valor -> índice
    for i, x in enumerate(nums):
        comp = target - x
        if comp in visto:
            return [visto[comp], i]
        visto[x] = i                # registra DEPOIS de checar (não usar x com ele mesmo)
    return []

assert two_sum([2, 7, 11, 15], 9) == [0, 1]
assert two_sum([3, 3], 6) == [0, 1]
assert two_sum([3, 2, 4], 6) == [1, 2]
```

Complexidade: O(n) de tempo e O(n) de espaço. É a troca clássica: **gastar memória para ganhar tempo**.

Padrão 2: agrupar por assinatura (Group Anagrams)

Problema 49: agrupe palavras que são anagramas entre si. A ideia é encontrar uma **chave canônica** igual para todos os anagramas:

- palavra ordenada: `"eat"`, `"tea"` e `"ate"` viram `"aet"` (O(k log k) por palavra);
- ou a tupla de 26 contagens (O(k) por palavra).

```python
from collections import defaultdict

def group_anagrams(strs: list[str]) -> list[list[str]]:
    grupos = defaultdict(list)
    for s in strs:
        cont = [0] * 26
        for ch in s:
            cont[ord(ch) - ord("a")] += 1
        grupos[tuple(cont)].append(s)      # lista não é hasheável: vira tupla
    return list(grupos.values())

res = group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"])
assert sorted(map(sorted, res)) == [["ate", "eat", "tea"], ["bat"], ["nat", "tan"]]
```

Padrão 3: set para busca O(1) (Longest Consecutive Sequence)

Problema 128: maior sequência de inteiros consecutivos, em **O(n)** (então não pode ordenar).

Ideia-chave: só comece a contar uma sequência a partir do seu **início**, ou seja, de um número `x` tal que `x - 1` **não** está no conjunto. Assim cada número é visitado no máximo duas vezes: uma no

laço externo e uma dentro de uma única sequência.

```python
def longest_consecutive(nums: list[int]) -> int:
    s = set(nums)
    best = 0
    for x in s:
        if x - 1 not in s:          # x é início de sequência
            y = x
            while y + 1 in s:
                y += 1
            best = max(best, y - x + 1)
    return best

assert longest_consecutive([100, 4, 200, 1, 3, 2]) == 4     # 1,2,3,4
assert longest_consecutive([0, 3, 7, 2, 5, 8, 4, 6, 0, 1]) == 9
```

Por que é O(n) se há um `while` dentro do `for`?

O `while` só roda para inícios de sequência, e anda exatamente pelo tamanho daquela sequência. Somando todas as sequências, o `while` anda no total ≤ n passos. Esse tipo de argumento ("cada elemento é processado um número constante de vezes no total") é **análise agregada**, e você vai usá-lo muito.

Padrão 4: contagem de frequência

- **Anagrama válido (242):** `Counter(s) == Counter(t)`.
- **Top K mais frequentes (347):** `Counter` + heap, ou *bucket sort* por frequência em O(n):

```python
from collections import Counter

def top_k_frequent(nums: list[int], k: int) -> list[int]:
    freq = Counter(nums)
    buckets = [[] for _ in range(len(nums) + 1)]   # índice = frequência
    for x, f in freq.items():
        buckets[f].append(x)
    res = []
    for f in range(len(buckets) - 1, 0, -1):       # das maiores frequências para as menores
        for x in buckets[f]:
            res.append(x)
            if len(res) == k:
                return res
    return res

assert sorted(top_k_frequent([1, 1, 1, 2, 2, 3], 2)) == [1, 2]
```

Padrão 5: hash de estado para detectar repetição

Para detectar se algo "já aconteceu" (ciclo numa sequência, configuração repetida), guarde os estados num `set`. Exemplo, **Happy Number (202)**: repita "soma dos quadrados dos dígitos"; se chegar em 1 é feliz, se repetir um valor entrou em ciclo.

```python
def is_happy(n: int) -> bool:
    visto = set()
    while n != 1 and n not in visto:
        visto.add(n)
        n = sum(int(d) ** 2 for d in str(n))
    return n == 1

assert is_happy(19) is True
assert is_happy(2) is False
```

(No capítulo de listas ligadas veremos como detectar ciclos com O(1) de espaço, usando o algoritmo de Floyd.)

Armadilhas

Cuidados com hash

- Usar `list` como chave dá `TypeError: unhashable type: 'list'`. Use `tuple`.
- Modificar um dict **enquanto itera** sobre ele levanta `RuntimeError`. Itere sobre `list(d.keys())`.
- `Counter` permite contagens negativas e zero: `c[x] -= 1` não remove a chave. Se a lógica depende de `len(c)`, apague com `del c[x]` quando chegar a 0.
- Ordem: `dict` preserva **ordem de inserção** (Python 3.7+); `set` **não** tem ordem garantida.

Prática

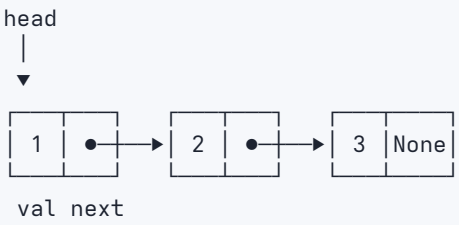
Problema	Padrão
1. Two Sum	complemento
217. Contains Duplicate	set
242. Valid Anagram	contagem
49. Group Anagrams	chave canônica
347. Top K Frequent Elements	contagem + bucket
128. Longest Consecutive Sequence	set + início de sequência
36. Valid Sudoku	sets por linha, coluna e caixa `(r//3, c//3)`
560. Subarray Sum Equals K	prefix sum + hash

CAPÍTULO 5

Listas ligadas

Estrutura

Uma lista ligada é uma sequência de **nós**. Cada nó guarda um valor e uma referência para o próximo. Ao contrário do array, os nós ficam **espalhados** na memória; só os ponteiros `next` os conectam.



Operação	Lista ligada	Array
acessar o i-ésimo	O(n), precisa andar do início	O(1)
inserir ou remover no início	**O(1)**	O(n)
inserir ou remover depois de um nó que você já tem	**O(1)**	O(n)
buscar um valor	O(n)	O(n)

Na prática, listas ligadas aparecem em entrevistas porque **testam manipulação de ponteiros**. O segredo é desenhar e lembrar a regra do capítulo de fundamentos: `x = y` move uma etiqueta; `x.next = y` **altera a estrutura**.

Operações básicas

Percorrer

```
curr = head            # etiqueta auxiliar; head continua no começo
while curr:            # equivale a: while curr is not None
    print(curr.val)
    curr = curr.next   # avança a etiqueta (não muda a lista)
```

O padrão dummy + tail (construir uma lista)

Um nó falso (`dummy`) antes da cabeça elimina o caso especial "lista ainda vazia": todo nó novo é pendurado da mesma forma, em `tail.next`. No fim, a resposta é `dummy.next`.

```python
def from_digits(digits):
    dummy = ListNode()
    tail = dummy
    for d in digits:
        tail.next = ListNode(d)   # pendura no fim (altera a estrutura)
        tail = tail.next          # anda a etiqueta para o novo fim
    return dummy.next

assert to_list(from_digits([7, 0, 8])) == [7, 0, 8]
```

O dummy também simplifica **remoções**: se a cabeça pode ser removida, comece com `dummy.next = head`, e o nó anterior à cabeça passa a existir.

Reverter uma lista (problema 206)

É **o** exercício fundamental. Precisamos virar cada seta `a → b` para `a ← b`. Três etiquetas: `prev` (já revertido), `curr` (sendo processado) e `nxt` (para não perder o resto).

```python
def reverse_list(head):
    prev = None
    curr = head
    while curr:
        nxt = curr.next     # 1. guarda o resto
        curr.next = prev    # 2. vira a seta
        prev = curr         # 3. avança prev
        curr = nxt          # 4. avança curr
    return prev             # prev é a nova cabeça

assert to_list(reverse_list(build_list([1, 2, 3, 4]))) == [4, 3, 2, 1]
assert reverse_list(None) is None
```

Rastreio com `1 → 2 → 3`:

```
início:   prev=None  curr=1     None   1 → 2 → 3
passo 1:  1.next = None         None ← 1   2 → 3      prev=1 curr=2
passo 2:  2.next = 1            None ← 1 ← 2   3      prev=2 curr=3
passo 3:  3.next = 2            None ← 1 ← 2 ← 3      prev=3 curr=None
fim: retorna prev = 3           3 → 2 → 1
```

Versão recursiva: "reverta o resto e depois pendure a cabeça no fim do resto revertido".

```python
def reverse_rec(head):
    if head is None or head.next is None:
        return head                 # lista de 0 ou 1 nó já está revertida
    new_head = reverse_rec(head.next)
    head.next.next = head           # o nó seguinte (agora último) aponta para head
    head.next = None                # head vira o novo último
    return new_head

assert to_list(reverse_rec(build_list([1, 2, 3, 4]))) == [4, 3, 2, 1]
```

Tempo O(n) nas duas versões. Espaço O(1) na iterativa e O(n) na recursiva (pilha).

Ponteiros rápido e lento

`slow` anda 1 passo e `fast` anda 2. Quando `fast` chega ao fim, `slow` está no **meio**.

```
def middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow

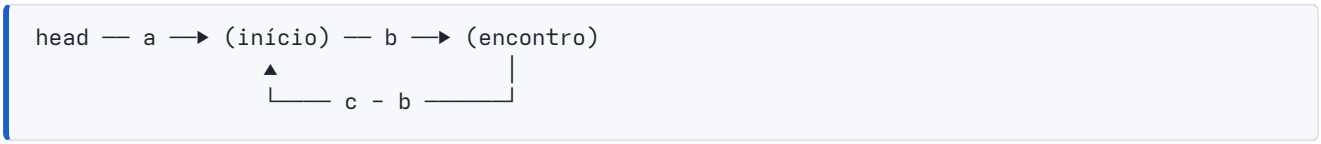
assert middle(build_list([1, 2, 3, 4, 5])).val == 3
assert middle(build_list([1, 2, 3, 4, 5, 6])).val == 4   # em par, o segundo do meio
```

Detectar ciclo: algoritmo de Floyd (problemas 141 e 142)

Se a lista tem ciclo, `fast` nunca chega a `None`: ele entra no ciclo e, a cada passo, **ganha uma posição** sobre `slow`. A distância entre os dois diminui de 1 em 1 até zerar, então eles se encontram. Se não há ciclo, `fast` chega ao fim.

Achar o início do ciclo (142). Seja:

- `a` = distância da cabeça até o início do ciclo;
- `b` = distância do início do ciclo até o ponto de encontro;
- `c` = tamanho do ciclo.



No encontro, `slow` andou `a + b` e `fast` andou `2(a + b)`. A diferença `a + b` é um número inteiro de voltas: `a + b = k·c`. Logo `a = k·c - b`. Isso quer dizer que andar `a` passos a partir do encontro é o mesmo que andar `c - b` (mais voltas completas), e isso leva **exatamente ao início do ciclo**. Então: recoloque um ponteiro na cabeça e ande os dois de 1 em 1; eles se encontram no início do ciclo.

```
def detect_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:                # há ciclo
            p = head
            while p is not slow:
                p = p.next
                slow = slow.next
            return p                    # início do ciclo
    return None

# monta 3 → 2 → 0 → -4 → (volta para o 2)
nodes = [ListNode(v) for v in [3, 2, 0, -4]]
for x, y in zip(nodes, nodes[1:]):
```

```
    x.next = y
nodes[-1].next = nodes[1]
assert detect_cycle(nodes[0]) is nodes[1]
assert detect_cycle(build_list([1, 2, 3])) is None
```

Use `is`, e não `==`, para comparar **nós**: queremos saber se são o mesmo objeto, não se têm o mesmo valor.

Remover o N-ésimo nó a partir do fim (problema 19)

Abra uma **distância de n nós** entre dois ponteiros. Quando o da frente chegar ao fim, o de trás está logo antes do nó a remover. O dummy cobre o caso em que a cabeça é removida.

```
def remove_nth_from_end(head, n):
    dummy = ListNode(0, head)
    ahead = behind = dummy
    for _ in range(n + 1):        # n+1 para "behind" parar ANTES do alvo
        ahead = ahead.next
    while ahead:
        ahead = ahead.next
        behind = behind.next
    behind.next = behind.next.next   # pula o alvo
    return dummy.next

assert to_list(remove_nth_from_end(build_list([1, 2, 3, 4, 5]), 2)) == [1, 2, 3, 5]
assert to_list(remove_nth_from_end(build_list([1]), 1)) == []
```

Mesclar duas listas ordenadas (problema 21)

```
def merge_two(l1, l2):
    dummy = tail = ListNode()
    while l1 and l2:
        if l1.val <= l2.val:
            tail.next, l1 = l1, l1.next
        else:
            tail.next, l2 = l2, l2.next
        tail = tail.next
    tail.next = l1 or l2           # pendura o que sobrou (já está ordenado)
    return dummy.next

assert to_list(merge_two(build_list([1, 2, 4]), build_list([1, 3, 4]))) == [1, 1, 2, 3, 4, 4]
```

Esse é o passo "merge" do merge sort. Para **K** listas (problema 23), use um heap; veja o capítulo de heaps.

Somar dois números (problema 2)

Os dígitos vêm em **ordem reversa**: o primeiro nó é a unidade. Isso é ótimo, porque é a mesma ordem em que somamos no papel, da direita para a esquerda, com "vai um" (*carry*).

```
  2 → 4 → 3      (342)
+ 5 → 6 → 4      (465)
  ─────────
  7 → 0 → 8      (807)

coluna 1: 2 + 5 + 0 = 7    → dígito 7, carry 0
coluna 2: 4 + 6 + 0 = 10   → dígito 0, carry 1
coluna 3: 3 + 4 + 1 = 8    → dígito 8, carry 0
```

```python
def add_two_numbers(l1, l2):
    dummy = tail = ListNode()
    carry = 0
    while l1 or l2 or carry:              # "or carry": 5 + 5 = 10 precisa de um nó extra
        v1 = l1.val if l1 else 0
        v2 = l2.val if l2 else 0
        carry, digit = divmod(v1 + v2 + carry, 10)
        tail.next = ListNode(digit)
        tail = tail.next
        l1 = l1.next if l1 else None
        l2 = l2.next if l2 else None
    return dummy.next

assert to_list(add_two_numbers(build_list([2, 4, 3]), build_list([5, 6, 4]))) == [7, 0, 8]
assert to_list(add_two_numbers(build_list([9, 9, 9]), build_list([1]))) == [0, 0, 0, 1]
assert to_list(add_two_numbers(build_list([5]), build_list([5]))) == [0, 1]
```

O(max(n, m)) de tempo, sem converter para inteiro ou string. Funciona em qualquer linguagem, mesmo com números de 1000 dígitos.

Combinando técnicas: Reorder List (problema 143)

Transforme `L0 → L1 → … → Ln` em `L0 → Ln → L1 → Ln-1 → …`. É um ótimo exemplo de problema "difícil" que é só a **composição de três técnicas simples**:

1. achar o meio (rápido e lento);
2. reverter a segunda metade;
3. intercalar as duas metades.

```python
def reorder_list(head) -> None:
    # 1. meio
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    # 2. reverte a segunda metade e corta a lista em duas
    prev, curr = None, slow.next
    slow.next = None
    while curr:
        curr.next, prev, curr = prev, curr, curr.next
    # 3. intercala
    first, second = head, prev
    while second:
        n1, n2 = first.next, second.next
        first.next = second
```

```
        second.next = n1
        first, second = n1, n2

h = build_list([1, 2, 3, 4, 5])
reorder_list(h)
assert to_list(h) == [1, 5, 2, 4, 3]
```

Atribuição múltipla com ponteiros

`curr.next, prev, curr = prev, curr, curr.next` funciona porque o lado direito é avaliado **inteiro** antes de qualquer atribuição. Mas a ordem das atribuições do lado esquerdo importa: são feitas da esquerda para a direita. Se `curr` fosse reatribuído antes de `curr.next`, você alteraria o nó errado. Na dúvida, use a versão com variável `nxt`.

Estrutura avançada: LRU Cache (problema 146)

Problema: um cache de capacidade fixa com `get(key)` e `put(key, value)` em **O(1)**. Quando enche, remove o item **menos recentemente usado**.

Por que duas estruturas?

- Um `dict` dá acesso O(1) pela chave, mas não sabe a ordem de uso.
- Uma **lista duplamente ligada** mantém a ordem de uso e permite mover ou remover um nó em O(1), *se você já tem o nó*.
- Juntando as duas: o dict mapeia `chave → nó`, e a lista mantém a ordem (mais recente perto da cauda).

```
dict: {1: ●, 3: ●, 2: ●}
          |     |     |
          ▼     ▼     ▼
 [head] ⇄ [1] ⇄ [3] ⇄ [2] ⇄ [tail]
 sentinela  LRU         MRU  sentinela
```

As sentinelas `head` e `tail` (como o dummy) eliminam os casos especiais de lista vazia.

```
class _Node:
    __slots__ = ("key", "val", "prev", "next")
    def __init__(self, key=0, val=0):
        self.key, self.val = key, val
        self.prev = self.next = None

class LRUCache:
    def __init__(self, capacity: int):
        self.cap = capacity
        self.map = {}
        self.head, self.tail = _Node(), _Node()   # sentinelas
        self.head.next, self.tail.prev = self.tail, self.head

    def _remove(self, node):            # desliga o nó de onde ele está
        node.prev.next = node.next
        node.next.prev = node.prev
```

```
    def _append(self, node):            # insere antes da cauda (posição MRU)
        node.prev, node.next = self.tail.prev, self.tail
        self.tail.prev.next = node
        self.tail.prev = node

    def get(self, key: int) -> int:
        if key not in self.map:
            return -1
        node = self.map[key]
        self._remove(node)              # foi usado: move para MRU
        self._append(node)
        return node.val

    def put(self, key: int, value: int) -> None:
        if key in self.map:
            self._remove(self.map[key])
        node = _Node(key, value)
        self.map[key] = node
        self._append(node)
        if len(self.map) > self.cap:
            lru = self.head.next        # o primeiro depois da sentinela é o LRU
            self._remove(lru)
            del self.map[lru.key]       # por isso o nó guarda a chave!

c = LRUCache(2)
c.put(1, 1); c.put(2, 2)
assert c.get(1) == 1        # agora 2 é o LRU
c.put(3, 3)                 # remove 2
assert c.get(2) == -1
c.put(4, 4)                 # remove 1
assert c.get(1) == -1 and c.get(3) == 3 and c.get(4) == 4
```

Em Python há um atalho

`collections.OrderedDict` já é um dict + lista duplamente ligada: `move_to_end(key)` e `popitem(last=False)` são O(1). Numa entrevista, mencione o atalho, mas saiba implementar a versão manual, que é o que o entrevistador quer ver.

Armadilhas

Erros mais comuns com listas ligadas

- Acessar `x.next` quando `x` é `None` dá `AttributeError`. Sempre pense: "este ponteiro pode ser `None` aqui?". Por isso o laço do rápido e lento é `while fast and fast.next`.
- Perder o resto da lista ao fazer `curr.next = algo` antes de salvar `curr.next` em uma variável.
- Esquecer de terminar a lista com `None`, o que cria ciclos acidentais (em `reorder_list`, o `slow.next = None` é essencial).
- Retornar `head` depois de modificar a estrutura, quando a cabeça mudou. Retorne `dummy.next`.

Prática

Problema	Técnica
206. Reverse Linked List	três ponteiros
21. Merge Two Sorted Lists	dummy + tail
141 / 142. Linked List Cycle I e II	Floyd
19. Remove Nth Node From End	distância entre ponteiros
2. Add Two Numbers	carry
143. Reorder List	meio + reverter + intercalar
234. Palindrome Linked List	meio + reverter + comparar
138. Copy List with Random Pointer	dict nó original → cópia
146. LRU Cache	dict + lista dupla
25. Reverse Nodes in k-Group	reverter em blocos (difícil)

CAPÍTULO 6

Pilhas, filas e estruturas monotônicas

Pilha (stack): LIFO

Last In, First Out: o último a entrar é o primeiro a sair, como uma pilha de pratos. Em Python, use uma `list`: `append` empilha e `pop()` desempilha, ambos O(1). O topo é `stack[-1]`.

```
push 1, push 2, push 3        pop() → 3

   | 3 | ← topo                   |   |
   | 2 |                          | 2 | ← topo
   | 1 |                          | 1 |
   └───┘                          └───┘
```

Sinais de pilha: estruturas **aninhadas** (parênteses, tags, expressões), "desfazer", processar algo assim que o **par** dele aparece, e simular recursão de forma iterativa.

Parênteses válidos (problema 20)

Cada fechamento precisa casar com a abertura **mais recente ainda aberta**. "Mais recente" é exatamente o que uma pilha oferece.

```python
def is_valid(s: str) -> bool:
    par = {")": "(", "]": "[", "}": "{"}
    stack = []
    for c in s:
        if c in par:                          # fechamento
            if not stack or stack[-1] != par[c]:
                return False
            stack.pop()
        else:                                 # abertura
            stack.append(c)
    return not stack                          # sobrou abertura sem par?

assert is_valid("()[]{}") and is_valid("{[()]}")
assert not is_valid("(]") and not is_valid("([)]") and not is_valid("((")
```

Avaliar expressão em notação polonesa reversa (problema 150)

```python
def eval_rpn(tokens: list[str]) -> int:
    st = []
    for t in tokens:
        if t in "+-*/":
            b, a = st.pop(), st.pop()         # atenção à ordem: a op b
            if t == "+": st.append(a + b)
            elif t == "-": st.append(a - b)
            elif t == "*": st.append(a * b)
            else: st.append(int(a / b))       # trunca em direção a zero
        else:
            st.append(int(t))
    return st[0]

assert eval_rpn(["2", "1", "+", "3", "*"]) == 9
assert eval_rpn(["4", "13", "5", "/", "+"]) == 6
```

> **Divisão em Python**
>
> `//` arredonda para **baixo** (`-7 // 2 == -4`). Muitos problemas pedem truncamento em direção a zero (`-3`). Use `int(a / b)`.

Min Stack (problema 155): pilha com `min()` em O(1)

Truque: cada posição da pilha guarda também **o mínimo até ali**. Ao desempilhar, o mínimo anterior "volta" automaticamente.

```python
class MinStack:
    def __init__(self):
        self.st = []                  # pares (valor, mínimo até aqui)

    def push(self, x: int) -> None:
        m = min(x, self.st[-1][1]) if self.st else x
        self.st.append((x, m))

    def pop(self) -> None:
        self.st.pop()

    def top(self) -> int:
        return self.st[-1][0]

    def getMin(self) -> int:
        return self.st[-1][1]

ms = MinStack()
for x in [5, 2, 7, 1]:
    ms.push(x)
assert ms.getMin() == 1
ms.pop()
assert ms.getMin() == 2 and ms.top() == 7
```

Fila (queue): FIFO

First In, First Out, como uma fila de banco. Use `collections.deque`: `append` insere no fim e `popleft` remove do início, ambos O(1). Nunca use `list.pop(0)`, que é O(n).

```python
from collections import deque

q = deque()
q.append(1); q.append(2); q.append(3)
assert q.popleft() == 1
q.appendleft(0)                # deque também insere no início em O(1)
assert list(q) == [0, 2, 3]
assert q[0] == 0 and q[-1] == 3
```

Filas são o coração da **busca em largura (BFS)**, vista nos capítulos de árvores e grafos.

Pilha monotônica

Ideia

Uma pilha cujos elementos ficam **sempre em ordem** (crescente ou decrescente). Antes de empilhar `x`, desempilhe tudo que quebraria a ordem. O momento em que um elemento é desempilhado é exatamente o momento em que encontramos **o próximo elemento maior (ou menor) que ele**.

Sinais no enunciado: "próximo maior", "próximo menor", "quantos dias até ficar mais quente", "maior retângulo", "visibilidade", "span".

Próximo elemento maior: Daily Temperatures (problema 739)

Para cada dia, quantos dias até uma temperatura **maior**? Força bruta é $O(n^2)$. Com pilha monotônica **decrescente** de *índices* esperando seu "próximo maior":

```python
def daily_temperatures(t: list[int]) -> list[int]:
    res = [0] * len(t)
    st = []                                  # índices com temperaturas decrescentes
    for i, temp in enumerate(t):
        while st and t[st[-1]] < temp:       # temp é o "próximo maior" do topo
            j = st.pop()
            res[j] = i - j
        st.append(i)
    return res                               # quem sobrou na pilha fica com 0

assert daily_temperatures([73, 74, 75, 71, 69, 72, 76, 73]) == [1, 1, 4, 2, 1, 1, 0, 0]
```

Rastreio (pilha mostrada como `índice:temp`):

i	temp	Desempilha (e resolve)	Pilha depois
0	73	—	0:73

i	temp	Desempilha (e resolve)	Pilha depois
1	74	0 (res=1)	1:74
2	75	1 (res=1)	2:75
3	71	—	2:75, 3:71
4	69	—	2:75, 3:71, 4:69
5	72	4 (res=1), 3 (res=2)	2:75, 5:72
6	76	5 (res=1), 2 (res=4)	6:76
7	73	—	6:76, 7:73

Complexidade: O(n). Cada índice é empilhado uma vez e desempilhado no máximo uma vez.

Maior retângulo em histograma (problema 84), difícil

Para cada barra `h[i]`, o maior retângulo que usa **exatamente** sua altura se estende para a esquerda e para a direita até encontrar uma barra **menor**. Precisamos, para cada barra, do *menor anterior* e do *próximo menor*: pilha monotônica **crescente**.

Quando a barra `i` é menor que o topo, o topo descobriu seu limite direito (`i`). O limite esquerdo dele é o novo topo da pilha depois do pop, porque a pilha é crescente e esse é o menor anterior.

```
alturas: [2, 1, 5, 6, 2, 3]

6 │      ▇
5 │    ▇ ▇
4 │    ▇ ▇
3 │    ▇ ▇   ▇
2 │ ▇  ▇ ▇ ▇ ▇
1 │ ▇ ▇ ▇ ▇ ▇ ▇
  └────────────
    0 1 2 3 4 5     → maior retângulo: altura 5, barras 2..3, área 10
```

```python
def largest_rectangle(h: list[int]) -> int:
    st = []                       # índices com alturas crescentes
    best = 0
    for i, cur in enumerate(h + [0]):          # o 0 final força o esvaziamento da pilha
        while st and h[st[-1]] > cur:
            height = h[st.pop()]
            left = st[-1] if st else -1        # menor anterior
            width = i - left - 1               # entre left e i (exclusivos)
            best = max(best, height * width)
        st.append(i)
    return best

assert largest_rectangle([2, 1, 5, 6, 2, 3]) == 10
assert largest_rectangle([2, 4]) == 4
assert largest_rectangle([1, 1, 1, 1]) == 4
```

Esse problema é a base do **85. Maximal Rectangle**: para cada linha da matriz, monte o histograma de "1s consecutivos acima" e aplique `largest_rectangle`.

Trapping Rain Water (problema 42): duas soluções

A água sobre a posição `i` é `min(maior_à_esquerda, maior_à_direita) - h[i]`.

Solução com dois ponteiros, O(1) de espaço. Mantenha `left_max` e `right_max`. Se `left_max < right_max`, a água em `L` é determinada por `left_max`: existe algo à direita pelo menos tão alto quanto `right_max > left_max`. Então processe `L`. Caso contrário, processe `R`.

```
def trap(h: list[int]) -> int:
    L, R = 0, len(h) - 1
    left_max = right_max = water = 0
    while L < R:
        if h[L] < h[R]:
            left_max = max(left_max, h[L])
            water += left_max - h[L]
            L += 1
        else:
            right_max = max(right_max, h[R])
            water += right_max - h[R]
            R -= 1
    return water

assert trap([0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]) == 6
assert trap([4, 2, 0, 3, 2, 5]) == 9
```

Solução com pilha monotônica (preenche a água em "camadas horizontais"):

```
def trap_stack(h: list[int]) -> int:
    st, water = [], 0
    for i, cur in enumerate(h):
        while st and h[st[-1]] < cur:
            bottom = h[st.pop()]
            if not st:
                break                       # sem parede à esquerda
            left = st[-1]
            width = i - left - 1
            water += (min(h[left], cur) - bottom) * width
        st.append(i)
    return water

assert trap_stack([0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]) == 6
assert trap_stack([4, 2, 0, 3, 2, 5]) == 9
```

Deque monotônico: Sliding Window Maximum (problema 239)

Máximo de cada janela de tamanho k, em O(n). Mantenha um **deque de índices** com valores **decrescentes**:

- a **frente** é sempre o índice do máximo da janela;

- ao entrar `x`, remova do **fim** todos os menores que `x`, porque eles nunca mais serão máximo (`x` é maior e vai durar mais);
- remova da **frente** o índice que saiu da janela.

```python
from collections import deque

def max_sliding_window(nums: list[int], k: int) -> list[int]:
    dq = deque()                # índices; nums[dq] decrescente
    res = []
    for i, x in enumerate(nums):
        while dq and nums[dq[-1]] <= x:
            dq.pop()
        dq.append(i)
        if dq[0] <= i - k:      # frente saiu da janela
            dq.popleft()
        if i >= k - 1:
            res.append(nums[dq[0]])
    return res

assert max_sliding_window([1, 3, -1, -3, 5, 3, 6, 7], 3) == [3, 3, 5, 5, 6, 7]
assert max_sliding_window([1], 1) == [1]
```

Implementar fila com duas pilhas (problema 232)

Clássico de entrevista para mostrar análise amortizada. `entrada` recebe os pushes. Quando é preciso remover e `saida` está vazia, despeje tudo de `entrada` em `saida`: isso inverte a ordem, e o mais antigo fica no topo.

```python
class MyQueue:
    def __init__(self):
        self.entrada, self.saida = [], []

    def push(self, x):
        self.entrada.append(x)

    def _move(self):
        if not self.saida:
            while self.entrada:
                self.saida.append(self.entrada.pop())

    def pop(self):
        self._move()
        return self.saida.pop()

    def peek(self):
        self._move()
        return self.saida[-1]

    def empty(self):
        return not self.entrada and not self.saida

q = MyQueue()
q.push(1); q.push(2)
assert q.peek() == 1 and q.pop() == 1
```

```
q.push(3)
assert q.pop() == 2 and q.pop() == 3 and q.empty()
```

Cada elemento é movido **no máximo uma vez** de `entrada` para `saida`, então cada operação é O(1) amortizado.

Prática

Problema	Técnica
20. Valid Parentheses	pilha
155. Min Stack	pilha de pares
150. Evaluate Reverse Polish Notation	pilha
22. Generate Parentheses	backtracking com contadores
739. Daily Temperatures	pilha monotônica
496 / 503. Next Greater Element I e II	pilha monotônica (circular: percorra 2n)
84. Largest Rectangle in Histogram	pilha monotônica (difícil)
42. Trapping Rain Water	dois ponteiros ou pilha (difícil)
239. Sliding Window Maximum	deque monotônico (difícil)
853. Car Fleet	ordenar + pilha

CAPÍTULO 7

Busca binária

Ideia

Se o espaço de busca é **ordenado**, ou mais geralmente **monótono**, você pode olhar o meio e descartar **metade** do espaço a cada passo: O(log n). Para 10^9 elementos, são cerca de 30 passos.

A visão que destrava problemas difíceis: busca binária não é "procurar um número num array". É **encontrar a fronteira** de um predicado monótono:

```
índice:     0     1     2     3     4     5     6
ok(i):    False False False True  True  True  True
                              ▲
                    queremos o primeiro True
```

Se você consegue escrever uma função `ok(x)` que é `False, False, …, False, True, True, …, True` à medida que `x` cresce, você consegue aplicar busca binária, **mesmo sem nenhum array**.

O template único

Use sempre o mesmo template e você para de errar `+1` e `-1`. Ele retorna o **menor `x` em `[lo, hi)` com `ok(x) == True`**:

```python
def first_true(lo: int, hi: int, ok) -> int:
    """Menor x em [lo, hi) com ok(x) True. Se nenhum, retorna hi + 1."""
    hi += 1                         # trabalha no intervalo semiaberto [lo, hi)
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid):
            hi = mid                # mid pode ser a resposta: mantém
        else:
            lo = mid + 1            # mid não serve: descarta mid e tudo à esquerda
    return lo

assert first_true(0, 100, lambda x: x * x >= 50) == 8
assert first_true(0, 5, lambda x: False) == 6
```

Por que nunca entra em laço infinito: `mid` é sempre `< hi` (divisão arredonda para baixo), então `hi = mid` diminui o intervalo; `lo = mid + 1` também diminui. O intervalo encolhe a cada volta.

Invariante (o que se explica na entrevista): tudo à esquerda de `lo` é `False`; tudo em `hi` ou à direita é `True`. Quando `lo == hi`, essa é a fronteira.

Aplicações diretas

Busca exata (problema 704): primeiro índice com `nums[i] >= target`; depois confira se é igual.

```python
def search(nums: list[int], target: int) -> int:
    i = first_true(0, len(nums) - 1, lambda i: nums[i] >= target)
    return i if i < len(nums) and nums[i] == target else -1

assert search([-1, 0, 3, 5, 9, 12], 9) == 4
assert search([-1, 0, 3, 5, 9, 12], 2) == -1
```

Primeira e última posição (problema 34): a primeira é `first_true(nums[i] >= t)`; a última é `first_true(nums[i] > t) - 1`.

O módulo `bisect` já implementa isso:

Função	Retorna	Predicado equivalente
`bisect_left(a, x)`	primeiro índice com `a[i] >= x`	`ok(i) = a[i] >= x`
`bisect_right(a, x)`	primeiro índice com `a[i] > x`	`ok(i) = a[i] > x`

```python
from bisect import bisect_left, bisect_right

a = [1, 2, 2, 2, 3, 5]
assert bisect_left(a, 2) == 1         # primeira posição do 2
assert bisect_right(a, 2) == 4        # depois do último 2
assert bisect_right(a, 2) - bisect_left(a, 2) == 3   # quantidade de 2s
assert bisect_left(a, 4) == 5         # onde 4 seria inserido
```

Busca binária na resposta

Sinais

- "**mínimo** valor tal que é possível..." ou "**máximo** valor tal que...";
- a resposta está num intervalo numérico grande (até 10^9);
- é fácil **verificar** se um candidato funciona, mas difícil calcular a resposta diretamente.

Receita: (1) defina o intervalo `[lo, hi]` de respostas possíveis; (2) escreva `ok(x)` = "dá para fazer com x?"; (3) confirme que `ok` é monótono (se funciona com x, funciona com x + 1); (4) aplique o template.

Koko Eating Bananas (problema 875)

Koko come no máximo `k` bananas por hora, de uma pilha por vez, e tem `h` horas. Qual o **menor** `k`?

- Se `k` funciona, qualquer `k` maior também funciona: **monótono**.
- Horas necessárias com velocidade `k`: `Σ ceil(pilha / k)`.
- Intervalo: `k ∈ [1, max(piles)]`.

```python
def min_eating_speed(piles: list[int], h: int) -> int:
    def ok(k):
        horas = sum((p + k - 1) // k for p in piles)   # ceil sem float
        return horas <= h
    return first_true(1, max(piles), ok)

assert min_eating_speed([3, 6, 7, 11], 8) == 4
assert min_eating_speed([30, 11, 23, 4, 20], 5) == 30
assert min_eating_speed([30, 11, 23, 4, 20], 6) == 23
```

Complexidade: O(n · log(max)). O `ok` custa O(n) e é chamado $\log_2$(max) vezes.

> **ceil de inteiros sem float**
>
> `ceil(a / b)` com inteiros positivos é `(a + b - 1) // b`, ou `-(-a // b)`. Evite `math.ceil(a / b)` com números grandes, por causa de imprecisão de ponto flutuante.

Capacity to Ship Packages (problema 1011)

Menor capacidade do navio para entregar os pacotes, **em ordem**, em `days` dias. É a mesma estrutura: `ok(cap)` simula gulosamente, enchendo cada dia até não caber.

```python
def ship_within_days(weights: list[int], days: int) -> int:
    def ok(cap):
        d, carga = 1, 0
        for w in weights:
            if carga + w > cap:
                d += 1
                carga = 0
            carga += w
        return d <= days
    return first_true(max(weights), sum(weights), ok)   # precisa caber o maior pacote

assert ship_within_days([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 5) == 15
assert ship_within_days([3, 2, 2, 4, 1, 4], 3) == 6
```

Esse mesmo molde resolve: *Split Array Largest Sum* (410), *Minimize Max Distance to Gas Station*, *Magnetic Force Between Balls* (1552, que maximiza: aí o predicado é `True…True False…False`; use `first_true(not ok) - 1`).

Arrays rotacionados

Um array ordenado e rotacionado, como `[4, 5, 6, 7, 0, 1, 2]`, tem **duas metades ordenadas**. A observação-chave: ao cortar no meio, **pelo menos uma das metades está ordenada**, e dá para saber qual comparando com as pontas.

Mínimo em array rotacionado (problema 153)

Compare `nums[mid]` com `nums[-1]`: os elementos do "bloco da direita" (que contém o mínimo) são `<= nums[-1]`, e os do bloco da esquerda são `> nums[-1]`. Isso é um predicado monótono.

```python
def find_min(nums: list[int]) -> int:
    i = first_true(0, len(nums) - 1, lambda i: nums[i] <= nums[-1])
    return nums[i]

assert find_min([3, 4, 5, 1, 2]) == 1
assert find_min([4, 5, 6, 7, 0, 1, 2]) == 0
assert find_min([11, 13, 15, 17]) == 11
```

Buscar em array rotacionado (problema 33)

```python
def search_rotated(nums: list[int], target: int) -> int:
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
        if nums[lo] <= nums[mid]:                # metade esquerda ordenada
            if nums[lo] <= target < nums[mid]:
                hi = mid - 1                     # target está nela
            else:
                lo = mid + 1
        else:                                    # metade direita ordenada
            if nums[mid] < target <= nums[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1

assert search_rotated([4, 5, 6, 7, 0, 1, 2], 0) == 4
assert search_rotated([4, 5, 6, 7, 0, 1, 2], 3) == -1
assert search_rotated([1], 1) == 0
assert search_rotated([3, 1], 1) == 1
```

O raciocínio: numa metade ordenada é fácil testar se `target` está nela (basta comparar com as pontas). Se está, vá para ela; senão, vá para a outra.

Busca binária em matriz

Search a 2D Matrix (problema 74): se cada linha começa depois do fim da anterior, a matriz é um array ordenado "dobrado". O índice `k` corresponde a `(k // C, k % C)`.

```python
def search_matrix(m: list[list[int]], target: int) -> bool:
    R, C = len(m), len(m[0])
    k = first_true(0, R * C - 1, lambda k: m[k // C][k % C] >= target)
    return k < R * C and m[k // C][k % C] == target

M = [[1, 3, 5, 7], [10, 11, 16, 20], [23, 30, 34, 60]]
assert search_matrix(M, 3) and not search_matrix(M, 13)
```

Difícil: mediana de dois arrays ordenados (problema 4)

Objetivo: mediana de `A` e `B` ordenados em **O(log(min(m, n)))**.

Ideia: a mediana divide a união em duas metades de mesmo tamanho, com tudo da esquerda ≤ tudo da direita. Escolha quantos elementos `i` vêm de `A` para a metade esquerda; então `j = metade - i` vêm de `B`. O corte é válido quando:

```
          A[0..i-1]  |  A[i..]
          B[0..j-1]  |  B[j..]

válido ⇔  A[i-1] <= B[j]   e   B[j-1] <= A[i]
```

- Se `A[i-1] > B[j]`, pegamos elementos demais de A: diminua `i`.
- Se `B[j-1] > A[i]`, pegamos de menos: aumente `i`.

Isso é monótono em `i`, então dá para fazer busca binária em `i`, no array **menor**.

```python
def find_median_sorted_arrays(A: list[int], B: list[int]) -> float:
    if len(A) > len(B):
        A, B = B, A                          # busca no menor
    m, n = len(A), len(B)
    half = (m + n + 1) // 2                  # tamanho da metade esquerda
    INF = float("inf")
    lo, hi = 0, m
    while lo <= hi:
        i = (lo + hi) // 2
        j = half - i
        a_left  = A[i - 1] if i > 0 else -INF
        a_right = A[i]     if i < m else INF
        b_left  = B[j - 1] if j > 0 else -INF
        b_right = B[j]     if j < n else INF
        if a_left <= b_right and b_left <= a_right:     # corte válido
            if (m + n) % 2:
                return float(max(a_left, b_left))
            return (max(a_left, b_left) + min(a_right, b_right)) / 2
        if a_left > b_right:
            hi = i - 1
        else:
            lo = i + 1
    raise ValueError("entrada não ordenada")

assert find_median_sorted_arrays([1, 3], [2]) == 2.0
assert find_median_sorted_arrays([1, 2], [3, 4]) == 2.5
assert find_median_sorted_arrays([], [1]) == 1.0
assert find_median_sorted_arrays([1, 3, 8, 9, 15], [7, 11, 18, 19, 21, 25]) == 11.0
```

Os sentinelas `±∞` eliminam os casos de borda em que o corte fica na ponta de um array.

Armadilhas

Erros clássicos

- Misturar templates: `while lo <= hi` com `hi = mid` gera **laço infinito**. Escolha um template e use sempre o mesmo.
- Predicado não monótono: busca binária retorna lixo sem avisar. Antes de codar, verifique: "se x funciona, x + 1 também funciona?".
- Intervalo errado na busca na resposta: em *Ship Packages*, `lo` precisa ser `max(weights)`, senão `ok` mente.
- Em outras linguagens, `(lo + hi) / 2` pode estourar `int`; use `lo + (hi - lo) / 2`. Em Python não há overflow.

Prática

Problema	Técnica
704. Binary Search	template
35. Search Insert Position	`bisect_left`
34. First and Last Position	duas buscas
74. Search a 2D Matrix	matriz como array
875. Koko Eating Bananas	busca na resposta
1011. Capacity to Ship Packages	busca na resposta
410. Split Array Largest Sum	busca na resposta (difícil)
153. Find Minimum in Rotated Sorted Array	predicado `<= nums[-1]`
33. Search in Rotated Sorted Array	metade ordenada
981. Time Based Key-Value Store	`bisect` em listas de timestamps
4. Median of Two Sorted Arrays	busca no corte (difícil)

CAPÍTULO 8

Backtracking

Ideia

Backtracking constrói uma solução **uma escolha por vez**. Depois de cada escolha, explora recursivamente o resto. Quando termina (ou percebe que o caminho é inválido), **desfaz a escolha** e tenta a próxima. É uma busca em profundidade (DFS) na **árvore de decisões**.

```
Subconjuntos de [1, 2, 3]: em cada nível, decido "incluo ou não?"

                          []
              /                       \
           [1]                         []            ← decide o 1
         /       \                   /       \
     [1,2]       [1]             [2]         []      ← decide o 2
     /   \      /   \           /   \       /   \
[1,2,3][1,2][1,3] [1]      [2,3]  [2]   [3]   []    ← decide o 3 (folhas = respostas)
```

Sinais no enunciado: "**todas** as combinações / permutações / subconjuntos / partições", "gere todos", "encontre **qualquer** configuração válida" (N-Rainhas, Sudoku), restrições pequenas (n ≤ 15 a 20).

O template

```python
def backtrack(estado, escolhas):
    if estado_completo(estado):
        resultado.append(copia(estado))    # COPIE: estado continua mudando
        return
    for escolha in escolhas_validas(estado):
        aplicar(escolha, estado)           # 1. escolhe
        backtrack(estado, escolhas)        # 2. explora
        desfazer(escolha, estado)          # 3. desfaz (o "backtrack")
```

Três perguntas definem qualquer problema de backtracking:

1. **O que é o estado?** (o caminho até agora, as posições usadas)
2. **Quais são as escolhas em cada passo?** (e como evitar repetições)
3. **Quando o estado está completo, ou quando podemos podar?**

Subconjuntos (problema 78)

Formulação "a partir de `start`": cada nó da árvore **já é** um subconjunto válido. As escolhas são os elementos de `start` em diante, o que impede gerar `[2, 1]` depois de `[1, 2]`.

```python
def subsets(nums: list[int]) -> list[list[int]]:
    res, path = [], []
    def bt(start):
        res.append(path[:])               # todo nó é uma resposta
        for i in range(start, len(nums)):
            path.append(nums[i])          # escolhe
            bt(i + 1)                     # explora (só elementos depois de i)
            path.pop()                    # desfaz
    bt(0)
    return res

assert sorted(subsets([1, 2, 3])) == sorted(
    [[], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]])
```

Complexidade: 2^n subconjuntos, cada um copiado em O(n), total **$O(n \cdot 2^n)$**.

O bug número 1 do backtracking

`res.append(path)` em vez de `res.append(path[:])`. Você guarda a **mesma lista** várias vezes; ela continua sendo modificada e, no fim, todas as "respostas" são `[]`. Lembre do capítulo de fundamentos: append guarda uma referência.

Com duplicatas (problema 90)

Ordene e, **no mesmo nível** da árvore, pule valores iguais ao anterior. A condição `i > start` garante que só pulamos irmãos repetidos, não o uso do mesmo valor em níveis diferentes.

```python
def subsets_with_dup(nums: list[int]) -> list[list[int]]:
    nums.sort()
    res, path = [], []
    def bt(start):
        res.append(path[:])
        for i in range(start, len(nums)):
            if i > start and nums[i] == nums[i - 1]:
                continue                  # mesmo valor, mesmo nível: já explorado
            path.append(nums[i])
            bt(i + 1)
            path.pop()
    bt(0)
    return res

assert sorted(subsets_with_dup([1, 2, 2])) == [[], [1], [1, 2], [1, 2, 2], [2], [2, 2]]
```

Permutações (problema 46)

Agora a ordem importa: em cada posição, qualquer elemento **ainda não usado** pode entrar. O estado inclui um conjunto `used`.

```python
def permute(nums: list[int]) -> list[list[int]]:
    res, path = [], []
    used = [False] * len(nums)
    def bt():
        if len(path) == len(nums):
            res.append(path[:])
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            used[i] = True; path.append(nums[i])
            bt()
            used[i] = False; path.pop()
    bt()
    return res

assert sorted(permute([1, 2, 3])) == [
    [1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
```

Complexidade: O(n · n!). Com duplicatas (problema 47): ordene e pule `nums[i] == nums[i-1] and not used[i-1]`.

Combination Sum (problema 39): reuso e poda

Números podem ser reutilizados, então a chamada recursiva passa `i` (não `i + 1`). **Poda:** com os candidatos ordenados, se `candidates[i] > restante`, todos os seguintes também são grandes demais, então paramos o laço com `break`.

```python
def combination_sum(candidates: list[int], target: int) -> list[list[int]]:
    candidates.sort()
    res, path = [], []
    def bt(start, restante):
        if restante == 0:
            res.append(path[:])
            return
        for i in range(start, len(candidates)):
            c = candidates[i]
            if c > restante:
                break                       # poda: ordenado, nada depois serve
            path.append(c)
            bt(i, restante - c)             # i: pode reusar o mesmo número
            path.pop()
    bt(0, target)
    return res

assert combination_sum([2, 3, 6, 7], 7) == [[2, 2, 3], [7]]
assert combination_sum([2, 3, 5], 8) == [[2, 2, 2, 2], [2, 3, 3], [3, 5]]
```

Resumo das variações de índice

- `bt(i + 1)`: cada elemento no máximo uma vez (subconjuntos, combinações).
- `bt(i)`: elementos reutilizáveis (combination sum).
- laço de `0` com `used[]`: a ordem importa (permutações).
- `if i > start and nums[i] == nums[i-1]: continue`: elimina duplicatas na entrada.

Busca em grade: Word Search (problema 79)

Procure uma palavra em uma grade, movendo-se entre células adjacentes sem reutilizar células. O estado é a posição atual mais as células já usadas. Truque para economizar memória: **marque a célula na própria grade** e restaure ao voltar.

```python
def exist(board: list[list[str]], word: str) -> bool:
    R, C = len(board), len(board[0])

    def dfs(r, c, k):                          # k = índice do caractere esperado
        if k == len(word):
            return True
        if not (0 <= r < R and 0 <= c < C) or board[r][c] != word[k]:
            return False
        tmp, board[r][c] = board[r][c], "#"     # marca como usada
        found = (dfs(r + 1, c, k + 1) or dfs(r - 1, c, k + 1) or
                 dfs(r, c + 1, k + 1) or dfs(r, c - 1, k + 1))
        board[r][c] = tmp                       # desfaz
        return found

    return any(dfs(r, c, 0) for r in range(R) for c in range(C))

B = [["A", "B", "C", "E"], ["S", "F", "C", "S"], ["A", "D", "E", "E"]]
assert exist(B, "ABCCED") and exist(B, "SEE") and not exist(B, "ABCB")
```

Complexidade: $O(R \cdot C \cdot 3^L)$ para palavra de tamanho L: de cada célula há no máximo 3 direções novas, porque a de onde viemos está marcada.

N-Rainhas (problema 51)

Coloque `n` rainhas num tabuleiro `n × n` sem que duas se ataquem. Colocamos **uma rainha por linha**, então só precisamos escolher a coluna. Para checar ataques em O(1), guardamos três conjuntos:

- colunas ocupadas: `c`;
- diagonais "↘": todas as células têm o mesmo `r - c`;
- diagonais "↙": todas têm o mesmo `r + c`.

```
r - c (diagonal ↘)       r + c (diagonal ↙)
 0 -1 -2 -3               0  1  2  3
 1  0 -1 -2               1  2  3  4
 2  1  0 -1               2  3  4  5
 3  2  1  0               3  4  5  6
```

```python
def solve_n_queens(n: int) -> list[list[str]]:
    res = []
    cols, diag1, diag2 = set(), set(), set()
    queens = []                                  # queens[r] = coluna da rainha na linha r

    def bt(r):
        if r == n:
            res.append(["." * c + "Q" + "." * (n - c - 1) for c in queens])
            return
        for c in range(n):
            if c in cols or (r - c) in diag1 or (r + c) in diag2:
                continue                         # poda: posição atacada
            cols.add(c); diag1.add(r - c); diag2.add(r + c); queens.append(c)
            bt(r + 1)
            cols.remove(c); diag1.remove(r - c); diag2.remove(r + c); queens.pop()

    bt(0)
    return res

sols = solve_n_queens(4)
assert sols == [[".Q..", "...Q", "Q...", "..Q."], ["..Q.", "Q...", "...Q", ".Q.."]]
assert len(solve_n_queens(8)) == 92
```

A **poda** é o que torna isso viável: em vez de testar todas as n^n colocações, cortamos os ramos assim que há conflito.

Partição em palíndromos (problema 131)

Escolha onde cortar a string. A escolha em cada passo é "até onde vai o próximo pedaço", e só seguimos se o pedaço for palíndromo.

```python
def partition(s: str) -> list[list[str]]:
    res, path = [], []
    def bt(start):
        if start == len(s):
            res.append(path[:])
            return
        for end in range(start + 1, len(s) + 1):
            piece = s[start:end]
            if piece == piece[::-1]:             # só continua com palíndromos
                path.append(piece)
                bt(end)
                path.pop()
    bt(0)
    return res

assert partition("aab") == [["a", "a", "b"], ["aa", "b"]]
```

Sudoku Solver (problema 37): backtracking com retorno antecipado

Quando basta **uma** solução, a função retorna `True` assim que encontra uma e para de explorar.

```
def solve_sudoku(board: list[list[str]]) -> None:
    rows = [set() for _ in range(9)]
    cols = [set() for _ in range(9)]
    boxes = [set() for _ in range(9)]
    empty = []
    for r in range(9):
        for c in range(9):
            v = board[r][c]
            if v == ".":
                empty.append((r, c))
            else:
                rows[r].add(v); cols[c].add(v); boxes[(r // 3) * 3 + c // 3].add(v)

    def bt(k):
        if k == len(empty):
            return True
        r, c = empty[k]
        b = (r // 3) * 3 + c // 3
        for v in "123456789":
            if v in rows[r] or v in cols[c] or v in boxes[b]:
                continue
            board[r][c] = v
            rows[r].add(v); cols[c].add(v); boxes[b].add(v)
            if bt(k + 1):
                return True                    # achou: não desfaz nada
            rows[r].remove(v); cols[c].remove(v); boxes[b].remove(v)
        board[r][c] = "."
        return False

    bt(0)

puzzle = [list(r) for r in [
    "53..7....", "6..195...", ".98....6.", "8...6...3", "4..8.3..1",
    "7...2...6", ".6....28.", "...419..5", "....8..79"]]
solve_sudoku(puzzle)
assert "".join(puzzle[0]) == "534678912" and "".join(puzzle[8]) == "345286179"
```

Armadilhas

Checklist do backtracking

- Copiou o estado ao salvar (`path[:]`)?
- Todo `append`, `add` ou marcação tem o `pop`, `remove` ou desmarcação correspondente **depois** da chamada recursiva?
- O índice passado está certo (`i`, `i + 1` ou laço desde 0)?
- Há poda? Ordenar a entrada costuma habilitar `break` em vez de `continue`.
- Para contar soluções, em vez de listá-las, a recursão pode retornar um número. E se os subproblemas se repetem, isso vira **programação dinâmica**.

Prática

Problema	Variação
78. Subsets	`start`
90. Subsets II	duplicatas
46. Permutations	`used`
39. Combination Sum	reuso + poda
40. Combination Sum II	sem reuso + duplicatas
17. Letter Combinations of a Phone Number	produto cartesiano
22. Generate Parentheses	contadores `aberto` e `fechado`
79. Word Search	DFS em grade com marcação
131. Palindrome Partitioning	cortes
51. N-Queens	conjuntos de diagonais
37. Sudoku Solver	retorno antecipado (difícil)

CAPÍTULO 9

Árvores binárias e BST

Vocabulário

```
      3          ← raiz (root), profundidade 0
     /   \
    9     20      ← filhos de 3; 9 é folha (sem filhos)
         /  \
        15   7    ← profundidade 2; altura da árvore = 2 (em arestas)
```

- **Árvore binária**: cada nó tem no máximo 2 filhos (`left`, `right`).
- **Altura** de um nó: maior número de arestas até uma folha. **Profundidade**: arestas até a raiz.
- **Árvore balanceada**: altura O(log n). **Degenerada**: vira uma "lista", com altura O(n).
- **Subárvore**: qualquer nó com todos os seus descendentes. Toda subárvore é uma árvore, e por isso **recursão** é a ferramenta natural.
- No LeetCode, a árvore acima é escrita em nível como `[3, 9, 20, null, null, 15, 7]`.

O modelo mental de recursão em árvores

Quase todo problema de árvore se resolve respondendo: **"se eu já tivesse a resposta para as subárvores esquerda e direita, como combino as duas com o nó atual?"**

```python
def max_depth(root) -> int:
    if root is None:
        return 0                          # árvore vazia tem profundidade 0
    return 1 + max(max_depth(root.left), max_depth(root.right))

t = build_tree([3, 9, 20, None, None, 15, 7])
assert max_depth(t) == 3                  # contando nós no caminho mais longo
```

Há dois "fluxos" de informação:

- **De baixo para cima (pós-ordem)**: o nó usa os resultados dos filhos (altura, soma, "é balanceada?"). É o mais comum.
- **De cima para baixo (pré-ordem)**: o nó passa informação para os filhos via parâmetros (profundidade atual, limites mínimo e máximo, soma acumulada no caminho).

Percursos (traversals)

Percurso	Ordem	Uso típico
Pré-ordem	nó, esquerda, direita	copiar ou serializar a árvore, passar informação para baixo

Percurso	Ordem	Uso típico
Em ordem	esquerda, nó, direita	**BST: visita em ordem crescente**
Pós-ordem	esquerda, direita, nó	calcular a partir dos filhos (altura, deletar)
Por nível (BFS)	nível por nível	menor profundidade, visão lateral, largura

```
def preorder(root):
    return [root.val] + preorder(root.left) + preorder(root.right) if root else []

def inorder(root):
    return inorder(root.left) + [root.val] + inorder(root.right) if root else []

def postorder(root):
    return postorder(root.left) + postorder(root.right) + [root.val] if root else []

t = build_tree([1, 2, 3, 4, 5])
#         1
#       /   \
#      2     3
#     / \
#    4   5
assert preorder(t) == [1, 2, 4, 5, 3]
assert inorder(t) == [4, 2, 5, 1, 3]
assert postorder(t) == [4, 5, 2, 3, 1]
```

(Essas versões curtas concatenam listas, o que custa $O(n^2)$ no pior caso. Para produção, use uma lista de resultado compartilhada.)

Em ordem iterativa (pilha explícita)

Evita o limite de recursão e é cobrada em entrevistas. Ideia: desça para a esquerda empilhando; quando não der mais, desempilhe, visite e vá para a direita.

```
def inorder_iter(root):
    res, st, cur = [], [], root
    while cur or st:
        while cur:                  # desce tudo para a esquerda
            st.append(cur)
            cur = cur.left
        cur = st.pop()              # o mais à esquerda ainda não visitado
        res.append(cur.val)
        cur = cur.right             # agora a subárvore direita
    return res

assert inorder_iter(build_tree([1, 2, 3, 4, 5])) == [4, 2, 5, 1, 3]
```

Por nível: BFS com fila (problema 102)

O truque para separar os níveis: no início de cada volta, `len(q)` é exatamente o número de nós **do nível atual**.

```python
from collections import deque

def level_order(root) -> list[list[int]]:
    if not root:
        return []
    res, q = [], deque([root])
    while q:
        nivel = []
        for _ in range(len(q)):            # processa só o nível atual
            node = q.popleft()
            nivel.append(node.val)
            if node.left:  q.append(node.left)
            if node.right: q.append(node.right)
        res.append(nivel)
    return res

assert level_order(build_tree([3, 9, 20, None, None, 15, 7])) == [[3], [9, 20], [15, 7]]
```

Variações imediatas: **visão da direita** (199), o último de cada nível; **zigue-zague** (103), inverter os níveis ímpares; **menor profundidade** (111), a primeira folha encontrada na BFS.

Padrão: retornar um valor e atualizar uma resposta global

Em vários problemas, o valor que a recursão **retorna** para o pai é diferente da **resposta** do problema.

Diâmetro (problema 543)

O diâmetro é o maior caminho entre dois nós quaisquer. O caminho mais longo que **passa** por um nó usa `altura(esq) + altura(dir)`. Mas para o pai, o nó só pode oferecer **um** braço: `1 + max(esq, dir)`.

```python
def diameter(root) -> int:
    best = 0
    def height(node):                  # retorna: altura em nós
        nonlocal best
        if not node:
            return 0
        l, r = height(node.left), height(node.right)
        best = max(best, l + r)        # caminho que "dobra" neste nó (em arestas)
        return 1 + max(l, r)           # para o pai: só um braço
    height(root)
    return best

assert diameter(build_tree([1, 2, 3, 4, 5])) == 3     # 4-2-1-3 ou 5-2-1-3
```

Soma máxima de caminho (problema 124), difícil

Mesmo padrão, mas com valores negativos: um braço negativo é **descartado** (troca-se por 0).

```python
def max_path_sum(root) -> int:
    best = float("-inf")
    def gain(node):                 # retorna: melhor soma de um braço que desce de node
        nonlocal best
        if not node:
            return 0
        l = max(gain(node.left), 0)     # braço negativo? não usa
        r = max(gain(node.right), 0)
        best = max(best, node.val + l + r)   # caminho que dobra aqui
        return node.val + max(l, r)
    gain(root)
    return best

assert max_path_sum(build_tree([1, 2, 3])) == 6
assert max_path_sum(build_tree([-10, 9, 20, None, None, 15, 7])) == 42   # 15-20-7
assert max_path_sum(build_tree([-3])) == -3
```

Árvore balanceada (problema 110), com sinal de erro

Retorne a altura, ou `-1` para dizer "já desbalanceou". Isso evita recalcular alturas, o que daria $O(n^2)$.

```python
def is_balanced(root) -> bool:
    def h(node):
        if not node:
            return 0
        l = h(node.left)
        if l == -1: return -1
        r = h(node.right)
        if r == -1 or abs(l - r) > 1: return -1
        return 1 + max(l, r)
    return h(root) != -1

assert is_balanced(build_tree([3, 9, 20, None, None, 15, 7]))
assert not is_balanced(build_tree([1, 2, 2, 3, 3, None, None, 4, 4]))
```

Informação de cima para baixo

Path Sum (problema 112): existe caminho raiz-folha com soma alvo?

```python
def has_path_sum(root, target) -> bool:
    if not root:
        return False
    if not root.left and not root.right:            # folha
        return root.val == target
    rest = target - root.val                        # passa o que falta para baixo
    return has_path_sum(root.left, rest) or has_path_sum(root.right, rest)

t = build_tree([5, 4, 8, 11, None, 13, 4, 7, 2, None, None, None, 1])
assert has_path_sum(t, 22) and not has_path_sum(t, 100)
```

Árvore Binária de Busca (BST)

Propriedade: para **todo** nó, todos os valores da subárvore esquerda são menores e todos da direita são maiores. Consequências:

- **Em ordem dá a sequência ordenada.**
- Busca, inserção e remoção custam O(h): O(log n) se balanceada, O(n) se degenerada. (Árvores autobalanceadas, como AVL e rubro-negra, garantem O(log n); em Python, use `sortedcontainers.SortedList` quando disponível.)

```python
def bst_search(root, x):
    while root and root.val != x:
        root = root.left if x < root.val else root.right
    return root

def bst_insert(root, x):
    if not root:
        return TreeNode(x)
    if x < root.val:
        root.left = bst_insert(root.left, x)
    else:
        root.right = bst_insert(root.right, x)
    return root

root = None
for v in [5, 3, 8, 1, 4, 9]:
    root = bst_insert(root, v)
assert inorder(root) == [1, 3, 4, 5, 8, 9]
assert bst_search(root, 4).val == 4 and bst_search(root, 7) is None
```

Remover de uma BST (problema 450)

Três casos para o nó a remover:

1. **Folha**: simplesmente some (retorna `None`).
2. **Um filho**: o filho ocupa o lugar dele.
3. **Dois filhos**: troque o valor pelo **sucessor** (menor valor da subárvore direita) e remova o sucessor da subárvore direita.

```python
def bst_delete(root, key):
    if not root:
        return None
    if key < root.val:
        root.left = bst_delete(root.left, key)
    elif key > root.val:
        root.right = bst_delete(root.right, key)
    else:
        if not root.left:  return root.right       # casos 1 e 2
        if not root.right: return root.left
        succ = root.right
        while succ.left:                           # caso 3: menor da direita
            succ = succ.left
        root.val = succ.val
```

```
        root.right = bst_delete(root.right, succ.val)
    return root

root = bst_delete(root, 3)       # 3 tem dois filhos (1 e 4)
assert inorder(root) == [1, 4, 5, 8, 9]
```

Validar BST (problema 98)

> **O erro clássico**
>
> Checar só `left.val < node.val < right.val` **não basta**. Na árvore `[5, 4, 6, null, null, 3, 7]`, cada pai é maior que o filho esquerdo, mas o `3` está à direita do `5`. A propriedade vale para a **subárvore inteira**.

Solução: passe para baixo o **intervalo permitido** `(lo, hi)`.

```
def is_valid_bst(root) -> bool:
    def ok(node, lo, hi):
        if not node:
            return True
        if not (lo < node.val < hi):
            return False
        return ok(node.left, lo, node.val) and ok(node.right, node.val, hi)
    return ok(root, float("-inf"), float("inf"))

assert is_valid_bst(build_tree([2, 1, 3]))
assert not is_valid_bst(build_tree([5, 4, 6, None, None, 3, 7]))
```

K-ésimo menor (problema 230)

Em ordem visita crescente: pare no k-ésimo visitado.

```
def kth_smallest(root, k: int) -> int:
    st, cur = [], root
    while True:
        while cur:
            st.append(cur); cur = cur.left
        cur = st.pop()
        k -= 1
        if k == 0:
            return cur.val
        cur = cur.right

assert kth_smallest(build_tree([5, 3, 6, 2, 4, None, None, 1]), 3) == 3
```

Ancestral Comum Mais Baixo (LCA)

Em BST (problema 235): se `p` e `q` são ambos menores que o nó, desça para a esquerda; ambos maiores, desça para a direita; senão eles se separam aqui, e este nó é o LCA.

Em árvore binária qualquer (problema 236): retorne o nó se ele é `p` ou `q`. Se a busca encontrou algo **nas duas** subárvores, o nó atual é o LCA; senão, propague o que achou.

```python
def lca(root, p, q):
    if root is None or root is p or root is q:
        return root
    l = lca(root.left, p, q)
    r = lca(root.right, p, q)
    if l and r:
        return root          # p num lado, q no outro
    return l or r            # os dois no mesmo lado (ou nenhum)

t = build_tree([3, 5, 1, 6, 2, 0, 8, None, None, 7, 4])
n5, n1, n4 = t.left, t.right, t.left.right.right
assert lca(t, n5, n1) is t
assert lca(t, n5, n4) is n5          # um nó é ancestral de si mesmo
```

Construir árvore a partir de percursos (problema 105)

Pré-ordem diz **quem é a raiz** (o primeiro elemento). Em ordem diz **quem fica à esquerda e à direita** dela. Um dict `valor → índice` na lista em ordem evita buscas O(n), e o total fica O(n).

```python
def build_from(preorder: list[int], inorder_vals: list[int]):
    pos = {v: i for i, v in enumerate(inorder_vals)}
    it = iter(preorder)                     # consome a pré-ordem na ordem certa

    def build(lo, hi):                      # monta a subárvore de inorder[lo..hi]
        if lo > hi:
            return None
        root = TreeNode(next(it))
        m = pos[root.val]
        root.left = build(lo, m - 1)        # esquerda ANTES: segue a pré-ordem
        root.right = build(m + 1, hi)
        return root

    return build(0, len(inorder_vals) - 1)

t = build_from([3, 9, 20, 15, 7], [9, 3, 15, 20, 7])
assert level_order(t) == [[3], [9, 20], [15, 7]]
```

Serializar e desserializar (problema 297), difícil

Pré-ordem **com marcadores de nulo** identifica a árvore de forma única.

```python
def serialize(root) -> str:
    out = []
    def dfs(node):
        if not node:
            out.append("#")
            return
        out.append(str(node.val))
        dfs(node.left)
        dfs(node.right)
```

```
    dfs(root)
    return ",".join(out)

def deserialize(data: str):
    it = iter(data.split(","))
    def build():
        v = next(it)
        if v == "#":
            return None
        node = TreeNode(int(v))
        node.left = build()
        node.right = build()
        return node
    return build()

t = build_tree([1, 2, 3, None, None, 4, 5])
s = serialize(t)
assert s == "1,2,#,#,3,4,#,#,5,#,#"
assert serialize(deserialize(s)) == s
```

Complexidade em árvores

Operação	Tempo	Espaço
DFS (qualquer percurso)	O(n)	O(h) de pilha: O(log n) balanceada, O(n) degenerada
BFS	O(n)	O(w): largura máxima, até n/2 no último nível
BST busca, inserção ou remoção	O(h)	O(h) recursivo, O(1) iterativo

Prática

Problema	Padrão
104. Maximum Depth	baixo para cima
226. Invert Binary Tree	trocar filhos recursivamente
100 / 572. Same Tree, Subtree of Another Tree	comparação recursiva
102. Level Order Traversal	BFS por nível
199. Right Side View	BFS, último do nível
543. Diameter	retorno ≠ resposta
110. Balanced Binary Tree	sinal −1
98. Validate BST	limites de cima para baixo
230. Kth Smallest in BST	em ordem iterativa
236. LCA of Binary Tree	propagação

Problema	Padrão
105. Construct from Preorder and Inorder	dict de posições
124. Maximum Path Sum	ganho com descarte (difícil)
297. Serialize and Deserialize	pré-ordem com nulos (difícil)

CAPÍTULO 10

Heaps e filas de prioridade

Ideia

Um **heap** (min-heap) é uma árvore binária **completa** em que todo pai é ≤ seus filhos. Assim, o **menor elemento está sempre na raiz**. Ele não é totalmente ordenado: só garante a relação pai-filho. Essa ordem parcial é barata de manter.

```
          1                 armazenado num array (sem ponteiros!):
        /   \
       3     2              índice:  0  1  2  3  4  5
      / \   /               valor:  [1, 3, 2, 7, 4, 5]
     7   4 5
                            filhos de i: 2i+1 e 2i+2      pai de i: (i-1)//2
```

Operação	Custo	Como
ver o mínimo	O(1)	`heap[0]`
inserir (`heappush`)	O(log n)	coloca no fim e **sobe** trocando com o pai enquanto for menor
remover o mínimo (`heappop`)	O(log n)	move o último para a raiz e **desce** trocando com o menor filho
construir de uma lista (`heapify`)	**O(n)**	desce a partir do meio (a maioria dos nós está perto das folhas)

Sinais no enunciado: "**K** maiores / menores / mais frequentes / mais próximos", "sempre pegar o menor ou maior atual", "mesclar K listas ordenadas", "mediana em fluxo contínuo", agendamento, Dijkstra.

`heapq` em Python

`heapq` opera sobre uma `list` comum e é **sempre min-heap**.

```python
import heapq

h = []
for x in [5, 1, 8, 3]:
    heapq.heappush(h, x)
assert h[0] == 1                    # menor no topo
assert heapq.heappop(h) == 1
assert heapq.heappop(h) == 3

a = [9, 4, 7, 1]
heapq.heapify(a)                    # O(n), in-place
```

```python
assert a[0] == 1

# MAX-heap: guarde valores negados
mx = []
for x in [5, 1, 8, 3]:
    heapq.heappush(mx, -x)
assert -mx[0] == 8

# Tuplas: compara pelo 1º campo, depois pelo 2º, ...
tarefas = []
heapq.heappush(tarefas, (2, "escrever"))
heapq.heappush(tarefas, (1, "ler"))
assert heapq.heappop(tarefas) == (1, "ler")

assert heapq.nlargest(2, [5, 1, 8, 3]) == [8, 5]
assert heapq.nsmallest(2, [5, 1, 8, 3]) == [1, 3]
```

Empate em tuplas com objetos

`heappush(h, (dist, node))` quebra com `TypeError` se dois `dist` empatam e `node` não é comparável (como `ListNode`). Insira um desempate único: `(dist, contador, node)`, com `contador` incrementado a cada push.

Implementação manual (entenda o sobe e desce)

```python
class MinHeap:
    def __init__(self):
        self.a = []

    def push(self, x):
        a = self.a
        a.append(x)
        i = len(a) - 1
        while i > 0 and a[(i - 1) // 2] > a[i]:          # sobe
            p = (i - 1) // 2
            a[i], a[p] = a[p], a[i]
            i = p

    def pop(self):
        a = self.a
        top = a[0]
        last = a.pop()
        if a:
            a[0] = last
            i, n = 0, len(a)
            while True:                                  # desce
                l, r, m = 2 * i + 1, 2 * i + 2, i
                if l < n and a[l] < a[m]: m = l
                if r < n and a[r] < a[m]: m = r
                if m == i:
                    break
                a[i], a[m] = a[m], a[i]
                i = m
        return top
```

```
h = MinHeap()
for x in [5, 3, 8, 1, 9, 2]:
    h.push(x)
assert [h.pop() for _ in range(6)] == [1, 2, 3, 5, 8, 9]
```

Padrão 1: Top K com heap de tamanho K

K-ésimo maior elemento (problema 215). Mantenha um **min-heap com os K maiores vistos**. O topo é o menor deles, ou seja, o K-ésimo maior. Ao ver um número maior que o topo, ele entra e o topo sai.

```
import heapq

def find_kth_largest(nums: list[int], k: int) -> int:
    h = nums[:k]
    heapq.heapify(h)
    for x in nums[k:]:
        if x > h[0]:
            heapq.heapreplace(h, x)      # pop + push numa operação
    return h[0]

assert find_kth_largest([3, 2, 1, 5, 6, 4], 2) == 5
assert find_kth_largest([3, 2, 3, 1, 2, 4, 5, 5, 6], 4) == 4
```

Complexidade: O(n log k) de tempo e O(k) de espaço. É melhor que ordenar (O(n log n)) quando k ⩘ n. (Existe o *Quickselect*, O(n) em média; veja o capítulo de ordenação.)

> **Contraintuitivo, mas importante**
>
> Para os K **maiores**, use um **min**-heap (para descartar o menor dos candidatos). Para os K **menores**, use um **max**-heap.

K pontos mais próximos da origem (problema 973):

```
import heapq

def k_closest(points: list[list[int]], k: int) -> list[list[int]]:
    h = []                                       # max-heap pela distância (negada)
    for x, y in points:
        d = x * x + y * y                        # não precisa de sqrt para comparar
        heapq.heappush(h, (-d, x, y))
        if len(h) > k:
            heapq.heappop(h)                     # remove o mais distante
    return [[x, y] for _, x, y in h]

assert sorted(k_closest([[1, 3], [-2, 2]], 1)) == [[-2, 2]]
assert sorted(k_closest([[3, 3], [5, -1], [-2, 4]], 2)) == [[-2, 4], [3, 3]]
```

Padrão 2: mesclar K sequências ordenadas (problema 23)

O próximo elemento da saída é o **menor entre as K cabeças**. Mantenha as cabeças num heap: pop do menor, push do seguinte daquela mesma lista.

```python
import heapq

def merge_k_lists(lists):
    h = []
    for i, node in enumerate(lists):
        if node:
            heapq.heappush(h, (node.val, i, node))   # i desempata: ListNode não é comparável
    dummy = tail = ListNode()
    while h:
        _, i, node = heapq.heappop(h)
        tail.next = node
        tail = node
        if node.next:
            heapq.heappush(h, (node.next.val, i, node.next))
    return dummy.next

ls = [build_list([1, 4, 5]), build_list([1, 3, 4]), build_list([2, 6])]
assert to_list(merge_k_lists(ls)) == [1, 1, 2, 3, 4, 4, 5, 6]
```

Complexidade: O(N log K), com N = total de nós e K = número de listas.

Padrão 3: dois heaps (mediana em fluxo, problema 295)

Divida os números em duas metades:

- `low`: **max-heap** com a metade menor (o topo é o maior dos pequenos);
- `high`: **min-heap** com a metade maior (o topo é o menor dos grandes).

Invariantes: todo elemento de `low` ≤ todo elemento de `high`, e `len(low)` é igual a `len(high)` ou `len(high) + 1`. A mediana está nos topos.

```
     low (max-heap)        high (min-heap)
   [1, 2, 3, ⟨5⟩]   |   [⟨7⟩, 8, 9]
                ▲           ▲
       tamanhos 4 e 3 → mediana = topo de low = 5
```

```python
import heapq

class MedianFinder:
    def __init__(self):
        self.low = []     # max-heap (valores negados)
        self.high = []    # min-heap

    def addNum(self, x: int) -> None:
        heapq.heappush(self.low, -x)
        # garante low ≤ high: o maior de low vai para high
        heapq.heappush(self.high, -heapq.heappop(self.low))
        # rebalanceia: low pode ter no máximo 1 a mais
```

```
        if len(self.high) > len(self.low):
            heapq.heappush(self.low, -heapq.heappop(self.high))

    def findMedian(self) -> float:
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2

mf = MedianFinder()
mf.addNum(1); mf.addNum(2)
assert mf.findMedian() == 1.5
mf.addNum(3)
assert mf.findMedian() == 2.0
for x in [10, 0, -5]:
    mf.addNum(x)
assert mf.findMedian() == 1.5       # [-5, 0, 1, 2, 3, 10]
```

Cada `addNum` custa O(log n) e `findMedian` custa O(1).

Padrão 4: simulação com heap (Task Scheduler, problema 621)

Tarefas iguais precisam de `n` intervalos entre si. Guloso: a cada momento, execute a tarefa disponível com **mais execuções restantes**, porque ela é o gargalo. Um max-heap escolhe a tarefa, e uma fila guarda as que estão "esfriando".

```
import heapq
from collections import Counter, deque

def least_interval(tasks: list[str], n: int) -> int:
    h = [-c for c in Counter(tasks).values()]
    heapq.heapify(h)
    cooling = deque()                  # (restantes negativo, instante em que libera)
    t = 0
    while h or cooling:
        t += 1
        if h:
            c = heapq.heappop(h) + 1   # executa uma (negativo: +1 diminui o módulo)
            if c:
                cooling.append((c, t + n))
        if cooling and cooling[0][1] == t:
            heapq.heappush(h, cooling.popleft()[0])
    return t

assert least_interval(["A", "A", "A", "B", "B", "B"], 2) == 8   # A B _ A B _ A B
assert least_interval(["A", "A", "A", "B", "B", "B"], 0) == 6
```

Prática

Problema	Padrão
703. Kth Largest Element in a Stream	min-heap de tamanho k
1046. Last Stone Weight	max-heap

Problema	Padrão
215. Kth Largest Element in an Array	min-heap k / quickselect
973. K Closest Points to Origin	max-heap k
347. Top K Frequent Elements	Counter + heap
23. Merge k Sorted Lists	heap de cabeças (difícil)
621. Task Scheduler	heap + fila
295. Find Median from Data Stream	dois heaps (difícil)
355. Design Twitter	merge de K feeds

CAPÍTULO 11

Grafos

O que é um grafo

Um grafo é um conjunto de **vértices** (nós) ligados por **arestas**. Árvores e listas ligadas são casos particulares. Muitos problemas são grafos disfarçados: grades (cada célula é um vértice), dependências entre cursos, transformações de palavras, estados de um quebra-cabeça.

Tipo	Significado
Não direcionado	a aresta vale nos dois sentidos (amizade)
Direcionado	a aresta tem sentido `u → v` (pré-requisito, link)
Ponderado	cada aresta tem um custo (distância, preço)
Acíclico (DAG)	direcionado e sem ciclos (dependências)
Conexo	existe caminho entre quaisquer dois vértices

Representações

```
Arestas: 0-1, 0-2, 1-2, 2-3

Lista de adjacência (o padrão)      Matriz de adjacência
0: [1, 2]                              0 1 2 3
1: [0, 2]                           0 [0 1 1 0]
2: [0, 1, 3]                        1 [1 0 1 0]
3: [2]                              2 [1 1 0 1]
                                    3 [0 0 1 0]
```

	Lista de adjacência	Matriz
Espaço	O(V + E)	$O(V^2)$
Listar vizinhos de u	O(grau(u))	O(V)
Existe aresta u-v?	O(grau(u))	O(1)
Usar quando	quase sempre (grafos esparsos)	grafo denso, V pequeno

O LeetCode costuma dar a lista de arestas. O primeiro passo é quase sempre montar a lista de adjacência:

```python
from collections import defaultdict

def build_graph(edges, directed=False):
    g = defaultdict(list)
    for u, v in edges:
        g[u].append(v)
        if not directed:
            g[v].append(u)
    return g

g = build_graph([(0, 1), (0, 2), (1, 2), (2, 3)])
assert sorted(g[2]) == [0, 1, 3]
```

DFS: busca em profundidade

Vai o **mais fundo possível** por um caminho antes de voltar. Implementação: recursão, ou pilha explícita. **Sempre marque os visitados**: num grafo com ciclos, sem `visited` a busca não termina.

```python
def dfs_rec(g, start):
    visited, order = set(), []
    def dfs(u):
        visited.add(u)
        order.append(u)
        for v in g[u]:
            if v not in visited:
                dfs(v)
    dfs(start)
    return order

def dfs_iter(g, start):
    visited, order, st = set(), [], [start]
    while st:
        u = st.pop()
        if u in visited:
            continue
        visited.add(u)
        order.append(u)
        for v in reversed(g[u]):        # reversed só para imitar a ordem recursiva
            if v not in visited:
                st.append(v)
    return order

g = build_graph([(0, 1), (0, 2), (1, 3), (2, 3)])
assert dfs_rec(g, 0) == dfs_iter(g, 0) == [0, 1, 3, 2]
```

Usos: componentes conexos, alcançabilidade, detecção de ciclos, ordenação topológica, backtracking.

BFS: busca em largura

Explora em **camadas**: primeiro todos a distância 1, depois todos a distância 2, e assim por diante. Por isso **BFS encontra o caminho mais curto em grafos sem peso**.

```python
from collections import deque

def bfs_dist(g, start):
    dist = {start: 0}
    q = deque([start])
    while q:
        u = q.popleft()
        for v in g[u]:
            if v not in dist:            # dist também funciona como "visitado"
                dist[v] = dist[u] + 1
                q.append(v)
    return dist

g = build_graph([(0, 1), (0, 2), (1, 3), (2, 3), (3, 4)])
assert bfs_dist(g, 0) == {0: 0, 1: 1, 2: 1, 3: 2, 4: 3}
```

Marque como visitado ao ENFILEIRAR, não ao desenfileirar

Se você marcar só quando tira da fila, o mesmo vértice pode ser enfileirado várias vezes (por vizinhos diferentes), e o BFS degrada muito em grafos densos e grades.

Complexidade de DFS e BFS: O(V + E) de tempo e O(V) de espaço. Em grade R × C: O(R · C).

Grades como grafos

Número de ilhas (problema 200)

Cada célula `"1"` não visitada inicia uma nova ilha. Um DFS ou BFS a partir dela "afunda" a ilha inteira, marcando-a.

```python
def num_islands(grid: list[list[str]]) -> int:
    R, C = len(grid), len(grid[0])
    def sink(r, c):
        if r < 0 or r >= R or c < 0 or c >= C or grid[r][c] != "1":
            return
        grid[r][c] = "0"                      # marca visitado alterando a grade
        sink(r + 1, c); sink(r - 1, c); sink(r, c + 1); sink(r, c - 1)
    count = 0
    for r in range(R):
        for c in range(C):
            if grid[r][c] == "1":
                count += 1
                sink(r, c)
    return count

grid = [list("11000"), list("11000"), list("00100"), list("00011")]
assert num_islands(grid) == 3
```

BFS com múltiplas origens: Rotting Oranges (problema 994)

Todas as laranjas podres começam a apodrecer as vizinhas **ao mesmo tempo**. Coloque **todas as fontes na fila** desde o início; cada camada da BFS é um minuto.

```python
from collections import deque

def oranges_rotting(grid: list[list[int]]) -> int:
    R, C = len(grid), len(grid[0])
    q, fresh = deque(), 0
    for r in range(R):
        for c in range(C):
            if grid[r][c] == 2:
                q.append((r, c))
            elif grid[r][c] == 1:
                fresh += 1
    minutes = 0
    while q and fresh:
        for _ in range(len(q)):              # uma camada = um minuto
            r, c = q.popleft()
            for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
                if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] == 1:
                    grid[nr][nc] = 2
                    fresh -= 1
                    q.append((nr, nc))
        minutes += 1
    return minutes if fresh == 0 else -1

assert oranges_rotting([[2, 1, 1], [1, 1, 0], [0, 1, 1]]) == 4
assert oranges_rotting([[2, 1, 1], [0, 1, 1], [1, 0, 1]]) == -1
```

O mesmo truque resolve *Walls and Gates* (286) e *01 Matrix* (542): distância até a fonte mais próxima.

BFS em grafo implícito: Word Ladder (problema 127), difícil

Cada palavra é um vértice; duas palavras são vizinhas se diferem em uma letra. Gerar vizinhos comparando todos os pares custa $O(N^2 \cdot L)$. O truque é o **padrão com coringa**: `"hot"` pertence aos baldes `"*ot"`, `"h*t"` e `"ho*"`, e palavras no mesmo balde são vizinhas.

```python
from collections import deque, defaultdict

def ladder_length(begin: str, end: str, words: list[str]) -> int:
    if end not in words:
        return 0
    L = len(begin)
    buckets = defaultdict(list)
    for w in words:
        for i in range(L):
            buckets[w[:i] + "*" + w[i + 1:]].append(w)
    q = deque([(begin, 1)])
    seen = {begin}
    while q:
        w, d = q.popleft()
        if w == end:
            return d
        for i in range(L):
            key = w[:i] + "*" + w[i + 1:]
            for nxt in buckets[key]:
                if nxt not in seen:
                    seen.add(nxt)
                    q.append((nxt, d + 1))
```

```
            buckets[key] = []                # balde já explorado: esvazia
    return 0

assert ladder_length("hit", "cog", ["hot", "dot", "dog", "lot", "log", "cog"]) == 5
assert ladder_length("hit", "cog", ["hot", "dot", "dog", "lot", "log"]) == 0
```

Ciclos e ordenação topológica

Ordenação topológica: algoritmo de Kahn (BFS)

Num **DAG**, uma ordem topológica lista os vértices de modo que toda aresta `u → v` tenha `u` antes de `v` (faça os pré-requisitos primeiro).

Kahn:

1. Calcule o **grau de entrada** (quantas arestas chegam) de cada vértice.
2. Enfileire todos com grau 0: não dependem de ninguém.
3. Ao processar `u`, "remova" suas arestas: decremente o grau de cada vizinho; quem chegar a 0 entra na fila.
4. Se no fim nem todos foram processados, **há um ciclo**: os vértices do ciclo nunca chegam a grau 0.

Course Schedule II (problema 210)

`prerequisites[i] = [a, b]` significa "para fazer `a`, faça `b` antes", ou seja, a aresta `b → a`.

```
from collections import deque

def find_order(n: int, prerequisites: list[list[int]]) -> list[int]:
    g = [[] for _ in range(n)]
    indeg = [0] * n
    for a, b in prerequisites:
        g[b].append(a)
        indeg[a] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in g[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return order if len(order) == n else []      # [] = há ciclo

assert find_order(2, [[1, 0]]) == [0, 1]
res = find_order(4, [[1, 0], [2, 0], [3, 1], [3, 2]])
assert res[0] == 0 and res[-1] == 3
assert find_order(2, [[1, 0], [0, 1]]) == []     # ciclo
```

Detectar ciclo com DFS de 3 cores

Para grafos **direcionados**, só "visitado sim/não" não basta. Use três estados:

- **branco (0)**: não visitado;
- **cinza (1)**: na pilha de recursão atual, ou seja, no caminho que estamos explorando;
- **preto (2)**: totalmente explorado e sem ciclo a partir dele.

Encontrar um vizinho **cinza** significa voltar a um vértice do caminho atual: é um ciclo. Um vizinho preto é seguro.

```python
def has_cycle(n: int, edges: list[tuple[int, int]]) -> bool:
    g = [[] for _ in range(n)]
    for u, v in edges:
        g[u].append(v)
    color = [0] * n
    def dfs(u):
        color[u] = 1
        for v in g[u]:
            if color[v] == 1:
                return True               # aresta de volta: ciclo
            if color[v] == 0 and dfs(v):
                return True
        color[u] = 2
        return False
    return any(color[u] == 0 and dfs(u) for u in range(n))

assert has_cycle(3, [(0, 1), (1, 2), (2, 0)])
assert not has_cycle(4, [(0, 1), (0, 2), (1, 3), (2, 3)])   # "losango" não é ciclo
```

Union-Find (Disjoint Set Union)

Ideia

Mantém uma coleção de **conjuntos disjuntos** com duas operações quase O(1):

- `find(x)`: qual o representante (raiz) do conjunto de `x`?
- `union(a, b)`: junta os conjuntos de `a` e `b`.

Cada conjunto é uma árvore em que cada nó aponta para o pai, e a raiz é o representante. Duas otimizações tornam o custo praticamente constante, O(α(n)), onde α é a inversa de Ackermann (≤ 4 para qualquer n real):

- **Compressão de caminho**: no `find`, faça cada nó do caminho apontar direto para a raiz.
- **União por tamanho ou rank**: pendure a árvore menor na maior.

```python
class DSU:
    def __init__(self, n: int):
        self.parent = list(range(n))
        self.size = [1] * n
        self.components = n
```

```
    def find(self, x: int) -> int:
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]   # compressão (halving)
            x = self.parent[x]
        return x

    def union(self, a: int, b: int) -> bool:
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False                       # já estavam juntos
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra                   # menor embaixo da maior
        self.size[ra] += self.size[rb]
        self.components -= 1
        return True

d = DSU(5)
d.union(0, 1); d.union(3, 4)
assert d.find(0) == d.find(1) and d.find(0) != d.find(3)
assert d.components == 3                       # {0,1}, {2}, {3,4}
```

Quando usar: conectividade **dinâmica** (arestas chegando aos poucos), contar componentes, detectar ciclo em grafo **não direcionado** (unir dois vértices que já estão no mesmo conjunto fecha um ciclo), algoritmo de Kruskal.

Redundant Connection (problema 684): a primeira aresta cujo `union` retorna `False` é a que fecha o ciclo.

```
def find_redundant_connection(edges: list[list[int]]) -> list[int]:
    d = DSU(len(edges) + 1)
    for u, v in edges:
        if not d.union(u, v):
            return [u, v]
    return []

assert find_redundant_connection([[1, 2], [1, 3], [2, 3]]) == [2, 3]
assert find_redundant_connection([[1, 2], [2, 3], [3, 4], [1, 4], [1, 5]]) == [1, 4]
```

Caminho mínimo com pesos

Dijkstra

Para pesos não negativos. Generaliza a BFS: em vez de uma fila, usa um **min-heap por distância**. Sempre expande o vértice **não finalizado mais próximo**. Como os pesos são ≥ 0, nenhum caminho futuro pode chegar a ele por menos: a distância dele está **finalizada**.

```
    1        Partindo de A:
  A ─── B    heap: (0,A)
  │     │    pop A(0): relaxa B=1, C=4              heap: (1,B) (4,C)
 4│     │2   pop B(1): relaxa D=1+2=3               heap: (3,D) (4,C)
  │     │    pop D(3): relaxa C = min(4, 3+1) = 4   (não melhora)
  C ─── D    pop C(4): fim
    1        dist = {A:0, B:1, D:3, C:4}
```

```python
import heapq

def dijkstra(g: dict, src) -> dict:
    """g[u] = lista de (v, peso). Retorna distância mínima de src a cada vértice
alcançável."""
    dist = {src: 0}
    h = [(0, src)]
    while h:
        d, u = heapq.heappop(h)
        if d > dist[u]:
            continue                        # entrada obsoleta: já achamos algo melhor
        for v, w in g.get(u, []):
            nd = d + w
            if nd < dist.get(v, float("inf")):
                dist[v] = nd                # relaxamento
                heapq.heappush(h, (nd, v))
    return dist

g = {"A": [("B", 1), ("C", 4)], "B": [("A", 1), ("D", 2)],
     "C": [("A", 4), ("D", 1)], "D": [("B", 2), ("C", 1)]}
assert dijkstra(g, "A") == {"A": 0, "B": 1, "C": 4, "D": 3}
```

Complexidade: O((V + E) log V). A linha `if d > dist[u]: continue` é a "remoção preguiçosa": em vez de atualizar uma entrada no heap, que o `heapq` não suporta, inserimos uma nova e ignoramos a velha quando ela sair.

Network Delay Time (problema 743): tempo para o sinal chegar a todos = **maior** distância do Dijkstra.

```python
def network_delay_time(times: list[list[int]], n: int, k: int) -> int:
    g = {}
    for u, v, w in times:
        g.setdefault(u, []).append((v, w))
    dist = dijkstra(g, k)
    return max(dist.values()) if len(dist) == n else -1

assert network_delay_time([[2, 1, 1], [2, 3, 1], [3, 4, 1]], 4, 2) == 2
assert network_delay_time([[1, 2, 1]], 2, 2) == -1
```

Dijkstra não funciona com pesos negativos

Ele assume que uma distância finalizada não melhora mais. Uma aresta negativa encontrada depois quebra isso. Para pesos negativos, use **Bellman-Ford**.

Bellman-Ford e o limite de arestas

Relaxa **todas** as arestas `V - 1` vezes. Depois da rodada `i`, ele conhece todos os caminhos mínimos com até `i` arestas. Custa O(V · E), mas aceita pesos negativos (e detecta ciclo negativo se a rodada `V` ainda melhora algo).

Essa propriedade de "até `i` arestas" resolve diretamente **Cheapest Flights Within K Stops (problema 787)**: K paradas são K + 1 arestas, então rodamos K + 1 rodadas. O detalhe crucial é relaxar a partir de uma **cópia** da rodada anterior, para não encadear várias arestas na mesma rodada.

```python
def find_cheapest_price(n, flights, src, dst, k) -> int:
    INF = float("inf")
    dist = [INF] * n
    dist[src] = 0
    for _ in range(k + 1):
        prev = dist[:]                      # distâncias com até i arestas
        for u, v, w in flights:
            if prev[u] + w < dist[v]:
                dist[v] = prev[u] + w
    return dist[dst] if dist[dst] < INF else -1

fl = [[0, 1, 100], [1, 2, 100], [2, 0, 100], [1, 3, 600], [2, 3, 200]]
assert find_cheapest_price(4, fl, 0, 3, 1) == 700     # 0→1→3
assert find_cheapest_price(4, fl, 0, 3, 2) == 400     # 0→1→2→3
```

Floyd-Warshall: todos os pares

DP em O(V³): `dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])`, permitindo o vértice `k` como intermediário. O laço de `k` **precisa ser o mais externo**. Use quando V ≤ ~400 e você precisa da distância entre todos os pares.

```python
def floyd_warshall(n, edges):
    INF = float("inf")
    d = [[0 if i == j else INF for j in range(n)] for i in range(n)]
    for u, v, w in edges:
        d[u][v] = min(d[u][v], w)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if d[i][k] + d[k][j] < d[i][j]:
                    d[i][j] = d[i][k] + d[k][j]
    return d

d = floyd_warshall(3, [(0, 1, 4), (1, 2, 1), (0, 2, 7)])
assert d[0][2] == 5
```

Árvore Geradora Mínima (MST)

Conectar todos os vértices com custo total mínimo usando `V - 1` arestas.

Kruskal: ordene as arestas por peso; adicione cada uma se ela **não formar ciclo** (checado com Union-Find). É guloso, e funciona pela *propriedade do corte*: a aresta mais leve que cruza qualquer divisão dos vértices pertence a alguma MST.

Min Cost to Connect All Points (problema 1584): custo = distância de Manhattan entre pontos.

```python
def min_cost_connect_points(points: list[list[int]]) -> int:
    n = len(points)
    edges = []
    for i in range(n):
        for j in range(i + 1, n):
            (x1, y1), (x2, y2) = points[i], points[j]
            edges.append((abs(x1 - x2) + abs(y1 - y2), i, j))
    edges.sort()
    d, total, used = DSU(n), 0, 0
    for w, i, j in edges:
        if d.union(i, j):
            total += w
            used += 1
            if used == n - 1:
                break
    return total

assert min_cost_connect_points([[0, 0], [2, 2], [3, 10], [5, 2], [7, 0]]) == 20
```

Kruskal custa O(E log E). Para grafos densos como este (E ≈ V²), o **Prim** com heap, ou o Prim O(V²) com array, é mais eficiente.

Grafo bipartido (problema 785)

Dá para pintar os vértices com 2 cores sem que vizinhos tenham a mesma cor? Faça uma BFS pintando alternadamente; um conflito significa que não é bipartido (existe ciclo ímpar).

```python
from collections import deque

def is_bipartite(graph: list[list[int]]) -> bool:
    color = [-1] * len(graph)
    for s in range(len(graph)):              # o grafo pode ser desconexo
        if color[s] != -1:
            continue
        color[s] = 0
        q = deque([s])
        while q:
            u = q.popleft()
            for v in graph[u]:
                if color[v] == -1:
                    color[v] = 1 - color[u]
                    q.append(v)
                elif color[v] == color[u]:
                    return False
    return True

assert is_bipartite([[1, 3], [0, 2], [1, 3], [0, 2]])
assert not is_bipartite([[1, 2, 3], [0, 2], [0, 1, 3], [0, 2]])
```

Qual algoritmo usar?

Situação	Algoritmo	Complexidade
Alcançabilidade, componentes	DFS / BFS / Union-Find	O(V + E)
Caminho mínimo **sem peso** (ou peso igual)	BFS	O(V + E)
Caminho mínimo com pesos 0 ou 1	BFS 0-1 (deque: peso 0 na frente, peso 1 no fim)	O(V + E)
Caminho mínimo com pesos ≥ 0	Dijkstra	O((V + E) log V)
Pesos negativos / limite de arestas	Bellman-Ford	O(V · E)
Todos os pares, V pequeno	Floyd-Warshall	$O(V^3)$
Ordem de dependências / ciclo em direcionado	Kahn ou DFS de 3 cores	O(V + E)
Ciclo em não direcionado / conectividade incremental	Union-Find	≈ O(E)
Conectar tudo com custo mínimo	Kruskal / Prim	O(E log E)
2-coloração	BFS / DFS bipartido	O(V + E)

Prática

Problema	Algoritmo
200. Number of Islands	DFS em grade
695. Max Area of Island	DFS retornando tamanho
133. Clone Graph	DFS + dict original → cópia
417. Pacific Atlantic Water Flow	DFS reverso a partir das bordas
994. Rotting Oranges	BFS multi-origem
207 / 210. Course Schedule I e II	topológica
684. Redundant Connection	Union-Find
323. Number of Connected Components	Union-Find / DFS
127. Word Ladder	BFS com coringas (difícil)
743. Network Delay Time	Dijkstra
787. Cheapest Flights Within K Stops	Bellman-Ford limitado

Problema	Algoritmo
1584. Min Cost to Connect All Points	Kruskal / Prim
778. Swim in Rising Water	Dijkstra minimax ou busca binária + BFS (difícil)
269. Alien Dictionary	topológica a partir de pares de palavras (difícil)

CAPÍTULO 12

Programação dinâmica

Ideia

Programação dinâmica (DP) é **recursão + memória**. Ela se aplica quando o problema tem duas propriedades:

1. **Subestrutura ótima**: a resposta do problema é construída a partir das respostas de subproblemas menores.
2. **Subproblemas sobrepostos**: os mesmos subproblemas aparecem muitas vezes. É isso que separa DP de "dividir e conquistar" (no merge sort, as metades não se repetem).

Em vez de recalcular, **guarde cada resposta na primeira vez** e reutilize-a.

```
fib(5) ingênuo: a mesma conta várias vezes

                    fib(5)
                 /          \
            fib(4)           fib(3)      ← fib(3) calculado 2 vezes
           /     \           /     \
       fib(3)   fib(2)   fib(2)  fib(1)  ← fib(2) calculado 3 vezes
       /   \
   fib(2) fib(1)                         → O(2ⁿ) chamadas

com memória: cada fib(i) é calculado UMA vez → O(n)
```

Sinais no enunciado:

- "**número de maneiras** de...", "**mínimo / máximo** custo, lucro, tamanho...", "**é possível**...?";
- decisões em sequência em que **cada escolha afeta as seguintes** (pegar ou não pegar, cortar aqui ou ali);
- força bruta é exponencial (backtracking), mas o "estado" relevante é pequeno.

O roteiro em 5 passos

Esse roteiro serve para **todo** problema de DP. Escreva as respostas em comentários antes de codar.

1. **Estado**: o que `dp[i]` (ou `dp[i][j]`) significa, **em palavras e com precisão**. É o passo mais importante. Exemplo: "`dp[i]` = maior soma roubando apenas das casas `0..i`".
2. **Transição**: como `dp[i]` depende de estados menores. Pense na **última decisão**: o que acontece com o elemento `i`?
3. **Casos base**: os menores estados, que se respondem sem transição.
4. **Ordem de cálculo**: cada estado só pode ser calculado depois daqueles de que depende.

5. **Resposta**: qual estado (ou máximo entre estados) responde ao problema.

Top-down vs bottom-up

```python
from functools import cache

# 1. Top-down (memoização): a recursão natural + cache
@cache
def fib_memo(n: int) -> int:
    if n < 2:
        return n
    return fib_memo(n - 1) + fib_memo(n - 2)

# 2. Bottom-up (tabulação): preenche a tabela dos menores para os maiores
def fib_tab(n: int) -> int:
    if n < 2:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]

# 3. Bottom-up com espaço O(1): só os 2 últimos estados importam
def fib(n: int) -> int:
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a

assert fib_memo(30) == fib_tab(30) == fib(30) == 832040
```

	Top-down (memo)	Bottom-up (tabela)
Como escrever	recursão + `@cache`	laços preenchendo `dp`
Vantagem	mais fácil de derivar; calcula só os estados necessários	sem limite de recursão; permite otimizar espaço
Desvantagem	limite de recursão do Python; overhead de chamadas	é preciso descobrir a ordem certa

Estratégia recomendada

1. Escreva a **força bruta recursiva** (o backtracking "pego ou não pego").
2. Identifique os parâmetros que mudam: eles são o **estado**.
3. Adicione `@cache`: pronto, é DP top-down.
4. Se precisar, converta para bottom-up e otimize o espaço.

DP 1D

House Robber (problema 198)

Casas em linha; não pode roubar duas adjacentes. Maximize o total.

- **Estado:** `dp[i]` = maior total usando as casas `0..i`.
- **Transição** (última decisão: roubo ou não a casa `i`?):
  - não roubo `i`: `dp[i - 1]`;
  - roubo `i`: então não posso a `i - 1`, e fica `dp[i - 2] + nums[i]`.
  - `dp[i] = max(dp[i - 1], dp[i - 2] + nums[i])`.
- **Base:** `dp[-1] = dp[-2] = 0`, ou seja, sem casas o total é 0.

```python
def rob(nums: list[int]) -> int:
    prev2 = prev1 = 0                   # dp[i-2], dp[i-1]
    for x in nums:
        prev2, prev1 = prev1, max(prev1, prev2 + x)
    return prev1

assert rob([1, 2, 3, 1]) == 4           # 1 + 3
assert rob([2, 7, 9, 3, 1]) == 12       # 2 + 9 + 1
```

Rastreio `[2, 7, 9, 3, 1]`:

i	x	não roubar (`dp[i-1]`)	roubar (`dp[i-2] + x`)	dp[i]
0	2	0	0 + 2	2
1	7	2	0 + 7	7
2	9	7	2 + 9	11
3	3	11	7 + 3	11
4	1	11	11 + 1	12

Coin Change: mínimo de moedas (problema 322)

- **Estado:** `dp[a]` = menor número de moedas que somam exatamente `a`.
- **Transição** (última moeda usada foi `c`): `dp[a] = 1 + min(dp[a - c])` para toda moeda `c ≤ a`.
- **Base:** `dp[0] = 0`. Valores impossíveis ficam com ∞.

```python
def coin_change(coins: list[int], amount: int) -> int:
    INF = float("inf")
    dp = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and dp[a - c] + 1 < dp[a]:
                dp[a] = dp[a - c] + 1
    return dp[amount] if dp[amount] != INF else -1

assert coin_change([1, 2, 5], 11) == 3     # 5 + 5 + 1
assert coin_change([2], 3) == -1
assert coin_change([1, 3, 4], 6) == 2      # 3 + 3 (o guloso 4+1+1 erraria!)
```

O último teste mostra por que um algoritmo **guloso** ("sempre a maior moeda") falha e a DP é necessária.

Coin Change II: número de combinações (problema 518), a ordem dos laços

Quantas **combinações** (sem contar ordem) de moedas somam `amount`?

```python
def change(amount: int, coins: list[int]) -> int:
    dp = [1] + [0] * amount            # 1 maneira de formar 0: não usar nada
    for c in coins:                    # laço EXTERNO nas moedas
        for a in range(c, amount + 1):
            dp[a] += dp[a - c]
    return dp[amount]

def permutations_count(amount: int, coins: list[int]) -> int:
    dp = [1] + [0] * amount
    for a in range(1, amount + 1):     # laço EXTERNO nos valores
        for c in coins:
            if c <= a:
                dp[a] += dp[a - c]
    return dp[amount]

assert change(5, [1, 2, 5]) == 4                # 5, 2+2+1, 2+1+1+1, 1×5
assert permutations_count(4, [1, 2]) == 5       # 1111, 112, 121, 211, 22
assert change(4, [1, 2]) == 3                   # 1111, 112, 22
```

> **Por que a ordem dos laços muda a resposta?**
>
> Com **moedas fora**, quando processamos a moeda `c`, só existem combinações com as moedas anteriores; cada combinação é contada em uma única ordem (crescente por moeda). Com **valores fora**, para cada valor qualquer moeda pode ser "a última", então `1+2` e `2+1` contam separadamente: são **permutações** (problema 377, *Combination Sum IV*).

Word Break (problema 139)

- **Estado:** `dp[i]` = o prefixo `s[:i]` pode ser segmentado em palavras do dicionário.
- **Transição:** `dp[i]` é verdadeiro se existe `j < i` com `dp[j]` verdadeiro **e** `s[j:i]` no dicionário.

```python
def word_break(s: str, words: list[str]) -> bool:
    ws = set(words)
    maxlen = max(map(len, ws), default=0)
    dp = [True] + [False] * len(s)
    for i in range(1, len(s) + 1):
        for j in range(max(0, i - maxlen), i):     # só palavras de tamanho possível
            if dp[j] and s[j:i] in ws:
                dp[i] = True
                break
    return dp[len(s)]

assert word_break("leetcode", ["leet", "code"])
assert word_break("applepenapple", ["apple", "pen"])
assert not word_break("catsandog", ["cats", "dog", "sand", "and", "cat"])
```

Longest Increasing Subsequence (problema 300)

O(n²): `dp[i]` = tamanho da maior subsequência crescente que **termina** em `i`. Então `dp[i] = 1 + max(dp[j])` para `j < i` com `nums[j] < nums[i]`.

```python
def lis_quadratic(nums: list[int]) -> int:
    dp = [1] * len(nums)
    for i in range(len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp, default=0)

assert lis_quadratic([10, 9, 2, 5, 3, 7, 101, 18]) == 4   # 2, 3, 7, 18
```

O(n log n), patience sorting: mantenha `tails[k]` = o **menor final possível** de uma subsequência crescente de tamanho `k + 1`. `tails` é sempre crescente, então dá para usar busca binária. Para cada `x`:

- se `x` é maior que todos, ele estende a maior subsequência (`append`);
- senão, substitui o primeiro `tails[k] >= x`: uma subsequência de mesmo tamanho com final menor é sempre melhor para o futuro.

```python
from bisect import bisect_left

def lis(nums: list[int]) -> int:
    tails = []
    for x in nums:
        k = bisect_left(tails, x)
        if k == len(tails):
            tails.append(x)
        else:
            tails[k] = x
    return len(tails)

assert lis([10, 9, 2, 5, 3, 7, 101, 18]) == 4
assert lis([0, 1, 0, 3, 2, 3]) == 4
assert lis([7, 7, 7]) == 1
```

```
x:     10    9     2     5     3     7     101    18
tails: [10] [9]   [2]   [2,5] [2,3] [2,3,7] [2,3,7,101] [2,3,7,18]
                                              tamanho final = 4
```

`tails` **não** é a subsequência em si, apenas guarda os melhores finais. O tamanho, porém, é exato.

DP 2D

Caminhos em grade (problema 62)

`dp[r][c]` = número de caminhos até `(r, c)`, andando só para a direita e para baixo. Só se chega de cima ou da esquerda: `dp[r][c] = dp[r-1][c] + dp[r][c-1]`. Otimizado para uma linha:

```python
def unique_paths(m: int, n: int) -> int:
    row = [1] * n                       # primeira linha: 1 caminho para cada célula
    for _ in range(1, m):
        for c in range(1, n):
            row[c] += row[c - 1]        # row[c] (de cima) + row[c-1] (da esquerda)
    return row[-1]

assert unique_paths(3, 7) == 28
assert unique_paths(3, 2) == 3
```

Longest Common Subsequence (problema 1143)

- **Estado:** `dp[i][j]` = LCS de `a[:i]` e `b[:j]` (os `i` primeiros de `a` e os `j` primeiros de `b`).
- **Transição** (olhe os últimos caracteres `a[i-1]` e `b[j-1]`):
  - se são **iguais**, fazem parte da LCS: `dp[i][j] = dp[i-1][j-1] + 1`;
  - se são **diferentes**, um deles não está na LCS: `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`.
- **Base:** `dp[0][*] = dp[*][0] = 0` (string vazia).

```python
def lcs(a: str, b: str) -> int:
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[n][m]

assert lcs("abcde", "ace") == 3
assert lcs("abc", "def") == 0
```

Tabela para `a = "abcde"` e `b = "ace"`:

```
      ""  a   c   e
  ""   0  0   0   0
  a    0  1↖  1   1       ↖ = caracteres iguais (diagonal + 1)
  b    0  1   1   1       ← ou ↑ = copia o maior vizinho
  c    0  1   2↖  2
  d    0  1   2   2
  e    0  1   2   3↖     → resposta dp[5][3] = 3 ("ace")
```

Reconstruir a subsequência: comece em `dp[n][m]` e volte. Se os caracteres são iguais, inclua o caractere e vá na diagonal; senão, vá para o vizinho com o maior valor.

```python
def lcs_string(a: str, b: str) -> str:
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            dp[i][j] = dp[i-1][j-1] + 1 if a[i-1] == b[j-1] else max(dp[i-1][j], dp[i][j-1])
    out, i, j = [], n, m
    while i and j:
        if a[i - 1] == b[j - 1]:
            out.append(a[i - 1]); i -= 1; j -= 1
        elif dp[i - 1][j] >= dp[i][j - 1]:
            i -= 1
        else:
            j -= 1
    return "".join(reversed(out))

assert lcs_string("abcde", "ace") == "ace"
```

Edit Distance (problema 72), clássico difícil

Menor número de operações (inserir, remover ou substituir um caractere) para transformar `a` em `b`.

- **Estado:** `dp[i][j]` = distância entre `a[:i]` e `b[:j]`.
- **Transição:**
  - se `a[i-1] == b[j-1]`, nada a fazer: `dp[i-1][j-1]`;
  - senão, `1 + min(` substituir: `dp[i-1][j-1]`, remover `a[i-1]`: `dp[i-1][j]`, inserir `b[j-1]`: `dp[i][j-1]` `)`.
- **Base:** `dp[i][0] = i` (remover tudo); `dp[0][j] = j` (inserir tudo).

```python
def min_distance(a: str, b: str) -> int:
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n + 1): dp[i][0] = i
    for j in range(m + 1): dp[0][j] = j
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j - 1],   # substituir
                                   dp[i - 1][j],       # remover
                                   dp[i][j - 1])       # inserir
    return dp[n][m]

assert min_distance("horse", "ros") == 3     # horse → rorse → rose → ros
assert min_distance("intention", "execution") == 5
assert min_distance("", "abc") == 3
```

Otimização de espaço em DP 2D

Se a linha `i` só depende da linha `i - 1`, guarde **duas linhas** (ou uma, com cuidado com a diagonal). O espaço cai de O(n · m) para O(m).

Mochila 0/1 (0/1 Knapsack)

`n` itens, cada um com peso `w[i]` e valor `v[i]`; capacidade `W`. Cada item é usado **no máximo uma vez**.

- **Estado:** `dp[i][c]` = maior valor usando os primeiros `i` itens com capacidade `c`.
- **Transição:** `dp[i][c] = max(dp[i-1][c], dp[i-1][c - w] + v)`, ou seja, não pegar ou pegar o item `i`.

Otimização clássica para 1D: percorra a capacidade **de trás para frente**, para que `dp[c - w]` ainda seja o valor da **linha anterior** (o item ainda não usado).

```python
def knapsack(weights: list[int], values: list[int], W: int) -> int:
    dp = [0] * (W + 1)
    for w, v in zip(weights, values):
        for c in range(W, w - 1, -1):          # DE TRÁS PARA FRENTE: cada item 1 vez
            dp[c] = max(dp[c], dp[c - w] + v)
    return dp[W]

assert knapsack([1, 3, 4, 5], [1, 4, 5, 7], 7) == 9      # itens de peso 3 e 4
```

Para frente vs para trás

Percorrendo `c` **para frente**, `dp[c - w]` já pode incluir o item atual, e ele seria usado várias vezes. Isso é a **mochila ilimitada** (Coin Change). Percorrendo **para trás**, cada item é usado no máximo uma vez: **mochila 0/1**. É a mesma diferença entre os problemas 322 e 416.

Partition Equal Subset Sum (problema 416): existe um subconjunto com soma `total / 2`? É uma mochila 0/1 de booleanos:

```python
def can_partition(nums: list[int]) -> bool:
    total = sum(nums)
    if total % 2:
        return False
    alvo = total // 2
    dp = [True] + [False] * alvo
    for x in nums:
        for s in range(alvo, x - 1, -1):
            dp[s] = dp[s] or dp[s - x]
    return dp[alvo]

assert can_partition([1, 5, 11, 5])          # [1, 5, 5] e [11]
assert not can_partition([1, 2, 3, 5])
```

DP em intervalos

O estado é um **intervalo** `[i, j]`, e a resposta vem de dividi-lo num ponto `k`. A ordem de cálculo é por **tamanho crescente de intervalo**. Costuma ser $O(n^3)$.

Burst Balloons (problema 312), difícil

Estourar o balão `k` rende `nums[esq] · nums[k] · nums[dir]`, com os vizinhos **atuais**. O truque que destrava: em vez de pensar em qual balão estourar **primeiro** (os vizinhos mudam), pense em qual balão é estourado **por último** no intervalo `(i, j)`. Se `k` é o último, quando ele estoura seus vizinhos são exatamente `i` e `j`, as bordas, e os dois lados são independentes.

- **Estado:** `dp[i][j]` = máximo de moedas estourando todos os balões **estritamente entre** `i` e `j`.
- **Transição:** `dp[i][j] = max(dp[i][k] + nums[i]·nums[k]·nums[j] + dp[k][j])` para `i < k < j`.

```python
def max_coins(nums: list[int]) -> int:
    a = [1] + nums + [1]                      # bordas virtuais valem 1
    n = len(a)
    dp = [[0] * n for _ in range(n)]
    for length in range(2, n):                # distância entre i e j
        for i in range(n - length):
            j = i + length
            for k in range(i + 1, j):         # k = último a estourar em (i, j)
                dp[i][j] = max(dp[i][j], dp[i][k] + a[i] * a[k] * a[j] + dp[k][j])
    return dp[0][n - 1]

assert max_coins([3, 1, 5, 8]) == 167
assert max_coins([1, 5]) == 10
```

Palíndromos

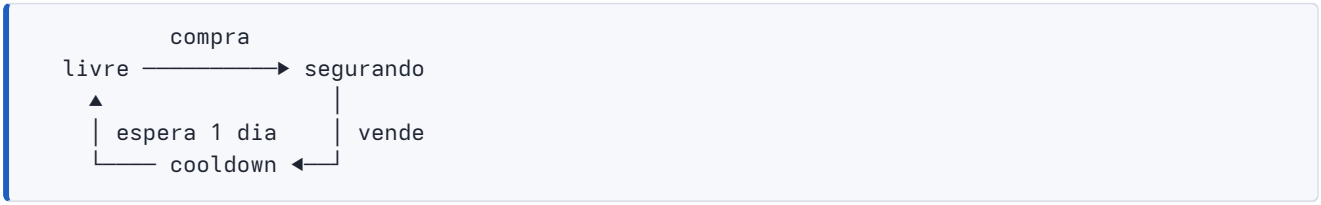
Longest Palindromic Substring (problema 5): a DP em intervalos funciona (`pal[i][j] = s[i] == s[j] and pal[i+1][j-1]`), mas **expandir a partir do centro** é mais simples e usa O(1) de espaço:

```python
def longest_palindrome(s: str) -> str:
    best = ""
    for center in range(len(s)):
        for L, R in ((center, center), (center, center + 1)):   # ímpar e par
            while L >= 0 and R < len(s) and s[L] == s[R]:
                L -= 1; R += 1
            if R - L - 1 > len(best):
                best = s[L + 1 : R]
    return best

assert longest_palindrome("babad") in ("bab", "aba")
assert longest_palindrome("cbbd") == "bb"
```

DP com máquina de estados

Quando, em cada dia ou passo, você está em um de poucos **modos**, o estado é `(i, modo)`. **Best Time to Buy and Sell Stock with Cooldown (problema 309):** depois de vender, é preciso esperar um dia.




```python
def max_profit_cooldown(prices: list[int]) -> int:
    hold, sold, rest = float("-inf"), 0, 0    # lucro ao fim do dia em cada modo
    for p in prices:
        hold, sold, rest = (
            max(hold, rest - p),     # já segurava, ou comprou hoje (vindo de "livre")
            hold + p,                # vendeu hoje
            max(rest, sold),         # livre: já estava livre, ou saiu do cooldown
        )
    return max(sold, rest)

assert max_profit_cooldown([1, 2, 3, 0, 2]) == 3   # compra, vende, cooldown, compra, vende
```

DP com bitmask (introdução)

Quando o estado é **"quais itens já foram usados"** e n ≤ 20, represente o conjunto como um inteiro de n bits. Por exemplo, `dp[mask][i]` = menor custo para visitar o conjunto `mask` terminando em `i` (o Caixeiro Viajante, em $O(2^n \cdot n^2)$).

```python
def tsp(dist: list[list[int]]) -> int:
    n = len(dist)
    INF = float("inf")
    dp = [[INF] * n for _ in range(1 << n)]
    dp[1][0] = 0                                  # começou na cidade 0
    for mask in range(1 << n):
        for i in range(n):
            if dp[mask][i] == INF or not (mask >> i) & 1:
                continue
            for j in range(n):
                if not (mask >> j) & 1:           # j ainda não visitada
                    nm = mask | (1 << j)
                    dp[nm][j] = min(dp[nm][j], dp[mask][i] + dist[i][j])
    full = (1 << n) - 1
    return min(dp[full][i] + dist[i][0] for i in range(n))

D = [[0, 10, 15, 20], [10, 0, 35, 25], [15, 35, 0, 30], [20, 25, 30, 0]]
assert tsp(D) == 80                               # 0→1→3→2→0
```

Como identificar o tipo de DP

Formato da entrada ou pergunta	Estado típico
uma sequência, decisão por elemento	`dp[i]` = melhor considerando até `i` (ou terminando em `i`)
duas strings ou sequências	`dp[i][j]` = resposta para os prefixos `a[:i]` e `b[:j]`
itens + capacidade	`dp[c]` (mochila)
grade	`dp[r][c]`
intervalo / "fundir" / "estourar"	`dp[i][j]` por tamanho, escolhendo `k`
poucos modos por passo	`dp[i][modo]`
subconjuntos com n ≤ 20	`dp[mask]`

Armadilhas

Erros comuns em DP

- Estado vago ("`dp[i]` é a resposta até i"): seja preciso. A maior subsequência **terminando** em i é diferente da maior **até** i.
- Esquecer os casos base, ou inicializar mínimos com 0 em vez de ∞.
- Ordem dos laços errada (combinações vs permutações; 0/1 vs ilimitada).
- `@cache` com argumentos mutáveis (lista) dá `TypeError: unhashable`. Passe índices ou tuplas.
- `@cache` em métodos de classe no LeetCode: defina a função interna **dentro** do método, para não reaproveitar cache entre testes.

Prática (em ordem de dificuldade)

Problema	Tipo
70. Climbing Stairs	1D (Fibonacci)
198 / 213. House Robber I e II	1D (II: circular, rode duas vezes)
322. Coin Change	mochila ilimitada
518. Coin Change II	combinações
139. Word Break	1D com dicionário
300. Longest Increasing Subsequence	1D / patience
152. Maximum Product Subarray	guarda máximo e mínimo
62. Unique Paths	grade
1143. Longest Common Subsequence	duas strings
416. Partition Equal Subset Sum	mochila 0/1
494. Target Sum	mochila 0/1 com contagem
309. Stock with Cooldown	máquina de estados
72. Edit Distance	duas strings (difícil)
115. Distinct Subsequences	duas strings (difícil)
10. Regular Expression Matching	duas strings (difícil)
312. Burst Balloons	intervalos (difícil)

CAPÍTULO 13

Algoritmos gulosos e intervalos

Ideia do guloso

Um algoritmo **guloso** faz, em cada passo, a escolha que parece **melhor agora**, sem voltar atrás. Quando funciona, é simples e rápido (muitas vezes uma ordenação e uma passada, O(n log n)). O problema é que **frequentemente não funciona**: no capítulo de DP, "sempre a maior moeda" falhou em `[1, 3, 4]` com alvo 6.

Como saber se o guloso está certo: você precisa de um argumento. Os dois mais usados:

- **Troca (exchange argument):** pegue uma solução ótima qualquer que difere da gulosa. Mostre que dá para **trocar** uma escolha dela pela escolha gulosa sem piorar. Então existe uma solução ótima que concorda com o guloso.
- **"Fica à frente" (stays ahead):** mostre que, depois de cada passo, o guloso está pelo menos tão bem quanto qualquer outra solução.

> **Na prática**
>
> Tente um contraexemplo pequeno em 2 minutos. Se não achar e houver um argumento de troca plausível, implemente o guloso. Se achar, provavelmente é DP.

Jump Game (problemas 55 e 45)

55. Dá para chegar ao fim? Mantenha o **alcance máximo** `reach`. Se em algum momento `i > reach`, você ficou preso.

```python
def can_jump(nums: list[int]) -> bool:
    reach = 0
    for i, x in enumerate(nums):
        if i > reach:
            return False
        reach = max(reach, i + x)
    return True

assert can_jump([2, 3, 1, 1, 4])
assert not can_jump([3, 2, 1, 0, 4])
```

45. Menor número de saltos. É uma BFS por camadas, sem fila: a camada atual vai de `start` até `end`, e a próxima camada vai até o `farthest` alcançável a partir dela.

```python
def jump(nums: list[int]) -> int:
    jumps = end = farthest = 0
    for i in range(len(nums) - 1):        # não precisa saltar a partir do último
        farthest = max(farthest, i + nums[i])
        if i == end:                      # fim da camada atual: precisa saltar
            jumps += 1
            end = farthest
    return jumps

assert jump([2, 3, 1, 1, 4]) == 2
assert jump([2, 3, 0, 1, 4]) == 2
assert jump([0]) == 0
```

Gas Station (problema 134)

Dois fatos levam à solução O(n):

1. Se `sum(gas) < sum(cost)`, é impossível. Caso contrário, **existe** uma resposta.
2. Se, partindo de `s`, o tanque fica negativo ao chegar em `i`, então **nenhum** posto entre `s` e `i` serve como início: chegamos a cada um deles com tanque ≥ 0, e mesmo assim não passamos de `i`. Recomece em `i + 1`.

```python
def can_complete_circuit(gas: list[int], cost: list[int]) -> int:
    if sum(gas) < sum(cost):
        return -1
    start = tank = 0
    for i in range(len(gas)):
        tank += gas[i] - cost[i]
        if tank < 0:
            start, tank = i + 1, 0
    return start

assert can_complete_circuit([1, 2, 3, 4, 5], [3, 4, 5, 1, 2]) == 3
assert can_complete_circuit([2, 3, 4], [3, 4, 3]) == -1
```

Problemas de intervalos

Intervalos `[início, fim]` aparecem em agendas, reuniões e recursos. **Quase sempre o primeiro passo é ordenar**, pelo início ou pelo fim. Saber qual dos dois é o ponto central.

Dois intervalos `[a, b]` e `[c, d]` (com `a <= c`) se **sobrepõem** se `c <= b` (ou `c < b`, se encostar não conta).

```
sobrepõem:    [a─────b]              não sobrepõem:  [a──b]
                  [c─────d]                                [c──d]
```

Merge Intervals (problema 56): ordenar pelo início

Ordenado pelo início, cada intervalo ou **se funde com o último** da resposta ou começa um novo.

```python
def merge(intervals: list[list[int]]) -> list[list[int]]:
    intervals.sort(key=lambda x: x[0])
    res = []
    for s, e in intervals:
        if res and s <= res[-1][1]:            # sobrepõe o último
            res[-1][1] = max(res[-1][1], e)    # max: o último pode conter este inteiro
        else:
            res.append([s, e])
    return res

assert merge([[1, 3], [2, 6], [8, 10], [15, 18]]) == [[1, 6], [8, 10], [15, 18]]
assert merge([[1, 4], [2, 3]]) == [[1, 4]]
assert merge([[1, 4], [4, 5]]) == [[1, 5]]
```

Insert Interval (problema 57)

A lista já está ordenada e sem sobreposição. Três fases: os que terminam **antes** do novo, os que **sobrepõem** (funde tudo num só) e os que começam **depois**. O(n).

```python
def insert(intervals: list[list[int]], new: list[int]) -> list[list[int]]:
    res, i, n = [], 0, len(intervals)
    s, e = new
    while i < n and intervals[i][1] < s:           # antes
        res.append(intervals[i]); i += 1
    while i < n and intervals[i][0] <= e:          # sobrepõem
        s = min(s, intervals[i][0])
        e = max(e, intervals[i][1])
        i += 1
    res.append([s, e])
    res.extend(intervals[i:])                      # depois
    return res

assert insert([[1, 3], [6, 9]], [2, 5]) == [[1, 5], [6, 9]]
assert insert([[1, 2], [3, 5], [6, 7], [8, 10], [12, 16]], [4, 8]) == [[1, 2], [3, 10], [12,
16]]
```

Non-overlapping Intervals (problema 435): ordenar pelo FIM

Menor número de remoções para não haver sobreposição. Equivale a **manter o máximo de intervalos compatíveis** (seleção de atividades).

Guloso: ordene pelo **fim** e sempre mantenha o intervalo que **termina primeiro**. **Argumento de troca:** numa solução ótima, troque o primeiro intervalo dela pelo que termina primeiro no geral. Ele termina antes ou junto, então não conflita com nada que o original não conflitasse.

```python
def erase_overlap_intervals(intervals: list[list[int]]) -> int:
    intervals.sort(key=lambda x: x[1])
    kept, end = 0, float("-inf")
    for s, e in intervals:
        if s >= end:            # compatível com o último mantido
            kept += 1
            end = e
    return len(intervals) - kept

assert erase_overlap_intervals([[1, 2], [2, 3], [3, 4], [1, 3]]) == 1
assert erase_overlap_intervals([[1, 2], [1, 2], [1, 2]]) == 2
```

Por que pelo fim e não pelo início?

Ordenando pelo início, `[[1, 100], [2, 3], [4, 5]]` faria você manter `[1, 100]` e remover dois. Pelo fim, você mantém `[2, 3]` e `[4, 5]` e remove só um. Quem **termina cedo deixa mais espaço** para o resto.

Meeting Rooms II (problema 253): quantas salas?

É o **número máximo de reuniões simultâneas**. Duas soluções clássicas.

Heap de horários de término: ordene pelo início. Para cada reunião, se a sala que libera primeiro (topo do heap) já liberou, reutilize-a; senão, abra outra. O tamanho do heap é o número de salas.

```python
import heapq

def min_meeting_rooms(intervals: list[list[int]]) -> int:
    intervals.sort()
    ends = []                               # min-heap dos términos em uso
    for s, e in intervals:
        if ends and ends[0] <= s:
            heapq.heapreplace(ends, e)      # reutiliza a sala que liberou
        else:
            heapq.heappush(ends, e)         # nova sala
    return len(ends)

assert min_meeting_rooms([[0, 30], [5, 10], [15, 20]]) == 2
assert min_meeting_rooms([[7, 10], [2, 4]]) == 1
```

Varredura (sweep line): transforme cada intervalo em dois eventos, `+1` no início e `-1` no fim, ordene e acumule. O pico do acumulado é a resposta. Com `(tempo, delta)`, o `-1` vem antes do `+1` no mesmo instante, então uma reunião que termina às 10 libera a sala para a que começa às 10.

```python
def min_meeting_rooms_sweep(intervals: list[list[int]]) -> int:
    events = []
    for s, e in intervals:
        events.append((s, 1))
        events.append((e, -1))
    events.sort()
    cur = best = 0
    for _, d in events:
        cur += d
        best = max(best, cur)
    return best

assert min_meeting_rooms_sweep([[0, 30], [5, 10], [15, 20]]) == 2
assert min_meeting_rooms_sweep([[1, 5], [5, 10]]) == 1
```

A **sweep line** é uma técnica geral: "quantos ativos ao mesmo tempo", "ponto mais coberto", "skyline". Converta intervalos em eventos ordenados.

Outros gulosos clássicos

Partition Labels (problema 763)

Divida a string no máximo de pedaços, de modo que cada letra apareça em um só pedaço. Guarde a **última ocorrência** de cada letra; o pedaço atual só pode terminar quando `i` alcançar a maior "última ocorrência" vista dentro dele.

```python
def partition_labels(s: str) -> list[int]:
    last = {c: i for i, c in enumerate(s)}
    res, start, end = [], 0, 0
    for i, c in enumerate(s):
        end = max(end, last[c])
        if i == end:
            res.append(end - start + 1)
            start = i + 1
    return res

assert partition_labels("ababcbacadefegdehijhklij") == [9, 7, 8]
```

Candy (problema 135), difícil

Cada criança recebe ≥ 1 doce, e quem tem nota maior que um vizinho recebe mais que ele. Minimize o total. **Duas passadas**: da esquerda para a direita, satisfaça a regra com o vizinho da esquerda; da direita para a esquerda, com o vizinho da direita, mantendo o máximo das duas exigências.

```python
def candy(ratings: list[int]) -> int:
    n = len(ratings)
    c = [1] * n
    for i in range(1, n):
        if ratings[i] > ratings[i - 1]:
            c[i] = c[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if ratings[i] > ratings[i + 1]:
            c[i] = max(c[i], c[i + 1] + 1)
    return sum(c)

assert candy([1, 0, 2]) == 5
assert candy([1, 2, 2]) == 4
```

"Duas passadas, uma em cada direção" é um padrão que se repete: *Trapping Rain Water*, *Product Except Self*, *Shortest Distance to a Character*.

Prática

Problema	Técnica
122. Best Time to Buy and Sell Stock II	somar toda alta
55 / 45. Jump Game I e II	alcance / BFS por camadas
134. Gas Station	reinício guloso
846. Hand of Straights	Counter + menor primeiro
763. Partition Labels	última ocorrência
56. Merge Intervals	ordenar pelo início
57. Insert Interval	três fases
435. Non-overlapping Intervals	ordenar pelo fim
252 / 253. Meeting Rooms I e II	ordenar / heap / sweep
1851. Minimum Interval to Include Each Query	ordenar queries + heap (difícil)
135. Candy	duas passadas (difícil)

CAPÍTULO 14

Estruturas avançadas: Trie, Fenwick e Segment Tree

Trie (árvore de prefixos)

Ideia

Uma árvore em que cada **aresta é um caractere** e cada caminho da raiz é um **prefixo**. Palavras com o mesmo prefixo compartilham o caminho. Um marcador `fim` indica que o caminho até ali forma uma palavra completa.

```
palavras: "car", "cat", "cart", "dog"

            (raiz)
           /      \
          c        d
          |        |
          a        o
         / \       |
        r*  t*     g*
        |
        t*                  * = fim de palavra
```

Operação	Trie	Set de strings
inserir / buscar palavra de tamanho L	O(L)	O(L) (para calcular o hash)
existe alguma palavra com prefixo p?	**O(|p|)**	O(n · |p|)
listar palavras com prefixo	O(|p| + saída)	O(n · L)

Sinais: prefixos, autocomplete, "começa com", buscar **muitas palavras ao mesmo tempo** numa grade, e XOR máximo (trie de bits).

```python
class TrieNode:
    __slots__ = ("children", "end")
    def __init__(self):
        self.children = {}          # caractere -> TrieNode
        self.end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
```

```
        node.end = True

    def _walk(self, s: str):
        node = self.root
        for ch in s:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    def search(self, word: str) -> bool:
        node = self._walk(word)
        return node is not None and node.end

    def startsWith(self, prefix: str) -> bool:
        return self._walk(prefix) is not None

t = Trie()
for w in ["car", "cat", "cart", "dog"]:
    t.insert(w)
assert t.search("car") and not t.search("ca")
assert t.startsWith("ca") and not t.startsWith("cu")
```

Busca com curinga (problema 211)

`"."` casa com qualquer letra: no ponto, tente **todos os filhos** (DFS).

```
class WordDictionary:
    def __init__(self):
        self.root = TrieNode()

    def addWord(self, word: str) -> None:
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.end = True

    def search(self, word: str) -> bool:
        def dfs(node, i):
            if i == len(word):
                return node.end
            ch = word[i]
            if ch == ".":
                return any(dfs(child, i + 1) for child in node.children.values())
            nxt = node.children.get(ch)
            return nxt is not None and dfs(nxt, i + 1)
        return dfs(self.root, 0)

wd = WordDictionary()
for w in ["bad", "dad", "mad"]:
    wd.addWord(w)
assert wd.search(".ad") and wd.search("b..") and not wd.search("pad")
```

Word Search II (problema 212), difícil

Encontrar **várias** palavras numa grade. Rodar o *Word Search* para cada palavra repete muito trabalho. Em vez disso, coloque todas as palavras numa trie e faça **um único DFS** por célula,

descendo na trie junto com a grade. Se o caminho atual não é prefixo de nenhuma palavra, poda imediatamente.

```python
def find_words(board: list[list[str]], words: list[str]) -> list[str]:
    root = {}
    for w in words:                        # trie com dicts; "$" guarda a palavra no fim
        node = root
        for ch in w:
            node = node.setdefault(ch, {})
        node["$"] = w
    R, C = len(board), len(board[0])
    found = []

    def dfs(r, c, parent):
        ch = board[r][c]
        node = parent[ch]
        if "$" in node:
            found.append(node.pop("$"))  # pop: evita achar a mesma palavra 2 vezes
        board[r][c] = "#"
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < R and 0 <= nc < C and board[nr][nc] in node:
                dfs(nr, nc, node)
        board[r][c] = ch
        if not node:                       # poda: ramo esgotado, remove da trie
            parent.pop(ch)

    for r in range(R):
        for c in range(C):
            if board[r][c] in root:
                dfs(r, c, root)
    return found

B = [list("oaan"), list("etae"), list("ihkr"), list("iflv")]
assert sorted(find_words(B, ["oath", "pea", "eat", "rain"])) == ["eat", "oath"]
```

Consultas de intervalo com atualizações

Problema: um array sofre **atualizações** de posições e **consultas** de soma em intervalos, intercaladas.

Estrutura	Atualizar	Consultar soma
array simples	O(1)	O(n)
prefix sum	O(n) (recalcular)	O(1)
Fenwick / Segment Tree	**O(log n)**	**O(log n)**

Com 10^5 operações de cada tipo, só a última linha passa no tempo.

Fenwick Tree (Binary Indexed Tree)

A posição `i` (base 1) guarda a soma de um bloco que **termina em i** e cujo tamanho é o **bit menos significativo** de `i`, ou seja, `i & -i`:

```
i (binário)   i & -i   bloco guardado em tree[i]
1  (0001)      1       a[1]
2  (0010)      2       a[1..2]
3  (0011)      1       a[3]
4  (0100)      4       a[1..4]
6  (0110)      2       a[5..6]
8  (1000)      8       a[1..8]
```

- **Prefixo até i:** some `tree[i]` e pule para `i -= i & -i` (remove o último bit).
- **Atualizar i:** some em `tree[i]` e suba para `i += i & -i` (os blocos que contêm i).

```python
class Fenwick:
    def __init__(self, n: int):
        self.n = n
        self.tree = [0] * (n + 1)          # base 1

    def add(self, i: int, delta: int) -> None:     # a[i] += delta (i base 0)
        i += 1
        while i <= self.n:
            self.tree[i] += delta
            i += i & -i

    def prefix(self, i: int) -> int:               # soma de a[0..i-1]
        s = 0
        while i > 0:
            s += self.tree[i]
            i -= i & -i
        return s

    def range_sum(self, l: int, r: int) -> int:    # soma de a[l..r]
        return self.prefix(r + 1) - self.prefix(l)

a = [5, 2, 9, -3, 6]
fw = Fenwick(len(a))
for i, x in enumerate(a):
    fw.add(i, x)
assert fw.range_sum(1, 3) == 8           # 2 + 9 - 3
fw.add(2, -9)                            # a[2] = 0
assert fw.range_sum(0, 4) == 10
```

Aplicação clássica: contar inversões ou *Count of Smaller Numbers After Self* (problema 315). Percorra da direita para a esquerda; para cada `x`, quantos menores já apareceram? `prefix(rank(x))`. Depois, registre `x`.

```python
def count_smaller(nums: list[int]) -> list[int]:
    ranks = {v: i for i, v in enumerate(sorted(set(nums)))}   # compressão de coordenadas
    fw = Fenwick(len(ranks))
    res = []
    for x in reversed(nums):
        res.append(fw.prefix(ranks[x]))   # quantos com rank < rank(x) já vistos
        fw.add(ranks[x], 1)
    return res[::-1]

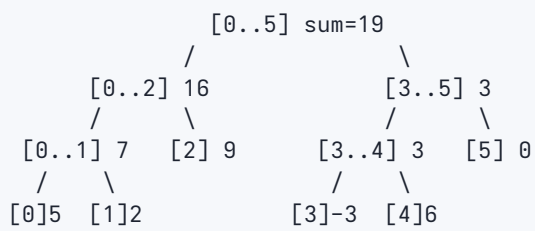
assert count_smaller([5, 2, 6, 1]) == [2, 1, 1, 0]
assert count_smaller([-1, -1]) == [0, 0]
```

Compressão de coordenadas

Se os valores são enormes (até 10^9) mas há poucos ($n \le 10^5$), troque cada valor pela sua **posição na lista ordenada de valores distintos**. A ordem se mantém e o domínio cai para `0..n-1`.

Segment Tree

Mais geral que a Fenwick: funciona para **qualquer operação associativa** (soma, mínimo, máximo, mdc), e não só para as que têm inversa. Cada nó guarda o resultado de um **segmento**; a raiz cobre tudo e cada filho cobre uma metade.



Uma consulta `[l, r]` visita O(log n) nós: os que estão **totalmente dentro** devolvem seu valor; os **totalmente fora** devolvem o neutro; os **parciais** descem. Versão iterativa compacta, com a árvore num array de tamanho `2n` (folhas em `n..2n-1`):

```python
class SegTree:
    def __init__(self, data, op=min, neutral=float("inf")):
        self.n = n = len(data)
        self.op, self.neutral = op, neutral
        self.t = [neutral] * n + list(data)
        for i in range(n - 1, 0, -1):
            self.t[i] = op(self.t[2 * i], self.t[2 * i + 1])

    def update(self, i, value):                  # a[i] = value
        i += self.n
        self.t[i] = value
        while i > 1:
            i //= 2
            self.t[i] = self.op(self.t[2 * i], self.t[2 * i + 1])

    def query(self, l, r):                       # op sobre a[l..r]
        res = self.neutral
        l += self.n; r += self.n + 1
        while l < r:
            if l & 1:
                res = self.op(res, self.t[l]); l += 1
            if r & 1:
                r -= 1; res = self.op(res, self.t[r])
            l //= 2; r //= 2
        return res

st = SegTree([5, 2, 9, -3, 6, 0])
assert st.query(0, 2) == 2 and st.query(1, 5) == -3
st.update(3, 10)
assert st.query(1, 5) == 0
```

```
ss = SegTree([5, 2, 9, -3, 6, 0], op=lambda a, b: a + b, neutral=0)
assert ss.query(0, 5) == 19
```

Com **propagação preguiçosa** (*lazy propagation*), a segment tree também suporta **atualizações em intervalos** ("some 5 em `a[l..r]`") em O(log n). É raro no LeetCode, mas comum em competições.

Prática

Problema	Estrutura
208. Implement Trie	trie
211. Design Add and Search Words	trie + DFS
212. Word Search II	trie + backtracking (difícil)
421. Maximum XOR of Two Numbers	trie de bits
307. Range Sum Query - Mutable	Fenwick / Segment Tree
315. Count of Smaller Numbers After Self	Fenwick + compressão (difícil)
493. Reverse Pairs	Fenwick ou merge sort (difícil)
218. The Skyline Problem	sweep + heap (difícil)

CAPÍTULO 15

Strings, bits e matemática

Busca de padrão em string

Problema: encontrar `pattern` (tamanho m) dentro de `text` (tamanho n). A busca ingênua compara a partir de cada posição: O(n · m) no pior caso (`text = "aaaa…ab"`, `pattern = "aa…ab"`). O `str.find` do Python é muito otimizado, e numa entrevista vale citá-lo. Mas KMP e Rabin-Karp são as ideias que resolvem variações difíceis.

KMP (Knuth-Morris-Pratt)

Ideia: quando há uma diferença depois de casar `k` caracteres, não volte o texto. Esses `k` caracteres **já são conhecidos** (são `pattern[:k]`). Pule para o **maior prefixo do padrão que também é sufixo** do trecho casado.

A tabela `lps[i]` (*longest proper prefix which is also suffix*) guarda, para `pattern[:i+1]`, o tamanho desse prefixo-sufixo:

```
pattern: a  b  a  b  c  a  b  a  b  d
lps:     0  0  1  2  0  1  2  3  4  0

lps[3] = 2: em "abab", "ab" é prefixo e também sufixo
```

```python
def build_lps(p: str) -> list[int]:
    lps = [0] * len(p)
    k = 0                                   # tamanho do prefixo-sufixo atual
    for i in range(1, len(p)):
        while k and p[i] != p[k]:
            k = lps[k - 1]                  # tenta o próximo prefixo-sufixo menor
        if p[i] == p[k]:
            k += 1
        lps[i] = k
    return lps

def kmp_search(text: str, p: str) -> list[int]:
    lps, res, k = build_lps(p), [], 0
    for i, ch in enumerate(text):
        while k and ch != p[k]:
            k = lps[k - 1]                  # não volta no texto: reaproveita o casado
        if ch == p[k]:
            k += 1
        if k == len(p):                     # casou inteiro
            res.append(i - k + 1)
            k = lps[k - 1]                  # continua buscando ocorrências sobrepostas
    return res

assert build_lps("ababcababd") == [0, 0, 1, 2, 0, 1, 2, 3, 4, 0]
assert kmp_search("abababcab", "abab") == [0, 2]
assert kmp_search("aaaaa", "aa") == [0, 1, 2, 3]
```

Complexidade: O(n + m). `k` sobe no máximo n vezes no total, então também desce no máximo n vezes.

Truque extra: a tabela `lps` sozinha resolve vários problemas. *Shortest Palindrome* (214): calcule `lps` de `s + "#" + s[::-1]`; o último valor é o maior prefixo palíndromo de `s`. *Repeated Substring Pattern* (459): `s` é repetição se `n % (n - lps[-1]) == 0` e `lps[-1] > 0`.

Rabin-Karp (rolling hash)

Ideia: compare **hashes** em vez de strings. O hash da janela seguinte é calculado a partir do anterior em O(1): remove o caractere que sai e adiciona o que entra, como uma janela deslizante numérica.

`hash(s) = s[0]·B^(m-1) + s[1]·B^(m-2) + … + s[m-1]` (mod M)

```python
def rabin_karp(text: str, p: str) -> list[int]:
    n, m = len(text), len(p)
    if m > n:
        return []
    B, M = 256, (1 << 61) - 1                       # base e um primo grande
    hp = ht = 0
    for i in range(m):
        hp = (hp * B + ord(p[i])) % M
        ht = (ht * B + ord(text[i])) % M
    power = pow(B, m - 1, M)                        # B^(m-1) mod M
    res = []
    for i in range(n - m + 1):
        if ht == hp and text[i:i + m] == p:         # confirma: hashes podem colidir
            res.append(i)
        if i + m < n:
            ht = (ht - ord(text[i]) * power) % M    # remove quem sai
            ht = (ht * B + ord(text[i + m])) % M    # adiciona quem entra
    return res

assert rabin_karp("abababcab", "abab") == [0, 2]
assert rabin_karp("hello", "ll") == [2]
```

O rolling hash brilha quando é preciso comparar **muitas substrings de mesmo tamanho**, como em *Longest Duplicate Substring* (1044): busca binária no tamanho + set de hashes das janelas.

Manipulação de bits

Operadores

Operador	Nome	Exemplo (`a = 0b1100`, `b = 0b1010`)
`a & b`	E	`0b1000`
`a | b`	OU	`0b1110`
`a ^ b`	OU exclusivo (XOR)	`0b0110`

Operador	Nome	Exemplo (`a = 0b1100`, `b = 0b1010`)
`~a`	NÃO	`-a - 1` em Python
`a << k`	desloca à esquerda	multiplica por 2^k
`a >> k`	desloca à direita	divide por 2^k (piso)

Truques essenciais

```python
x = 0b10110
assert (x >> 2) & 1 == 1               # testa o bit 2
assert x | (1 << 0) == 0b10111         # liga o bit 0
assert x & ~(1 << 1) == 0b10100        # desliga o bit 1
assert x ^ (1 << 4) == 0b00110         # inverte o bit 4
assert x & (x - 1) == 0b10100          # apaga o bit 1 mais baixo
assert x & -x == 0b00010               # isola o bit 1 mais baixo (usado na Fenwick)
assert bin(x).count("1") == 3          # quantidade de bits 1 (ou x.bit_count())

def is_power_of_two(n: int) -> bool:
    return n > 0 and n & (n - 1) == 0  # potência de 2 tem exatamente um bit 1

assert is_power_of_two(64) and not is_power_of_two(6)
```

As propriedades do XOR

`a ^ a = 0`, `a ^ 0 = a`, e o XOR é comutativo e associativo. Logo, **pares se cancelam**.

```python
from functools import reduce
from operator import xor

def single_number(nums: list[int]) -> int:        # problema 136
    return reduce(xor, nums)                      # todos os pares se anulam

def missing_number(nums: list[int]) -> int:       # problema 268
    res = len(nums)
    for i, x in enumerate(nums):
        res ^= i ^ x                              # índices e valores se anulam, sobra o
ausente
    return res

assert single_number([4, 1, 2, 1, 2]) == 4
assert missing_number([3, 0, 1]) == 2
```

Contar bits de 0 a n (problema 338): DP com bits

`bits(i) = bits(i >> 1) + (i & 1)`: o número de 1s de `i` é o de `i` sem o último bit, mais o último bit.

```python
def count_bits(n: int) -> list[int]:
    dp = [0] * (n + 1)
    for i in range(1, n + 1):
        dp[i] = dp[i >> 1] + (i & 1)
    return dp

assert count_bits(5) == [0, 1, 1, 2, 1, 2]
```

Bitmask para enumerar subconjuntos

Com n itens, cada inteiro de `0` a `2ⁿ - 1` é um subconjunto: o bit `i` ligado significa que o item `i` está dentro.

```python
def subsets_bitmask(nums: list[int]) -> list[list[int]]:
    n = len(nums)
    return [[nums[i] for i in range(n) if mask >> i & 1] for mask in range(1 << n)]

assert sorted(subsets_bitmask([1, 2])) == [[], [1], [1, 2], [2]]
```

Matemática que aparece

MDC (algoritmo de Euclides) e MMC

`mdc(a, b) = mdc(b, a % b)`, com `mdc(a, 0) = a`. Custa O(log min(a, b)).

```python
def gcd(a: int, b: int) -> int:
    while b:
        a, b = b, a % b
    return a

def lcm(a: int, b: int) -> int:
    return a // gcd(a, b) * b

assert gcd(48, 18) == 6 and lcm(4, 6) == 12
import math
assert math.gcd(48, 18) == 6          # já existe na biblioteca padrão
```

Exponenciação rápida (problema 50)

`xⁿ = (x^(n/2))²` se n é par; `x · xⁿ⁻¹` se é ímpar. Isso dá O(log n) multiplicações em vez de n.

```python
def my_pow(x: float, n: int) -> float:
    if n < 0:
        x, n = 1 / x, -n
    res = 1.0
    while n:
        if n & 1:          # bit atual de n ligado: multiplica
            res *= x
        x *= x             # x, x², x⁴, x⁸, ...
        n >>= 1
    return res

assert my_pow(2.0, 10) == 1024.0
assert abs(my_pow(2.0, -2) - 0.25) < 1e-12
assert pow(3, 200, 10**9 + 7) == pow(3, 200) % (10**9 + 7)   # pow modular embutido
```

Aritmética modular

Quando o enunciado diz "retorne a resposta módulo $10^9 + 7$", aplique `% MOD` **a cada passo** de somas e multiplicações: `(a + b) % M` e `(a * b) % M` podem ser feitos parcialmente. Em Python não há overflow, mas números gigantes ficam lentos. **Divisão** modular exige o inverso: `a / b ≡ a · b^(M-2) (mod M)` quando M é primo (pequeno teorema de Fermat), ou seja, `a * pow(b, M - 2, M) % M`.

Crivo de Eratóstenes (problema 204)

Primos até n em O(n log log n): para cada primo p, risque seus múltiplos a partir de p^2, porque os menores já foram riscados por primos menores.

```python
def count_primes(n: int) -> int:
    if n < 3:
        return 0
    is_p = [True] * n
    is_p[0] = is_p[1] = False
    p = 2
    while p * p < n:
        if is_p[p]:
            is_p[p * p :: p] = [False] * len(range(p * p, n, p))
        p += 1
    return sum(is_p)

assert count_primes(10) == 4          # 2, 3, 5, 7
assert count_primes(100) == 25
```

Prática

Problema	Técnica
28. Find the Index of the First Occurrence	KMP
214. Shortest Palindrome	lps de `s#reverso` (difícil)
1044. Longest Duplicate Substring	busca binária + rolling hash (difícil)

Problema	Técnica
136. Single Number	XOR
191. Number of 1 Bits	`x & (x - 1)`
338. Counting Bits	DP com bits
268. Missing Number	XOR
371. Sum of Two Integers	soma com `^` e carry `&`
50. Pow(x, n)	exponenciação rápida
204. Count Primes	crivo

CAPÍTULO 16

Ordenação e dividir para conquistar

Você quase nunca vai implementar uma ordenação na prática: `sorted()` (Timsort, O(n log n) e **estável**) é melhor. Mas as **ideias** dos algoritmos de ordenação resolvem problemas difíceis: o *merge* conta inversões, a *partição* acha o k-ésimo elemento e o *counting sort* ordena em O(n).

Ordenação em Python

```
pessoas = [("ana", 30), ("bia", 25), ("caio", 30)]

assert sorted(pessoas, key=lambda p: p[1]) == [("bia", 25), ("ana", 30), ("caio", 30)]
# idade decrescente, depois nome crescente: negue o campo numérico
assert sorted(pessoas, key=lambda p: (-p[1], p[0])) == [("ana", 30), ("caio", 30), ("bia",
25)]

# comparação customizada: "Largest Number" (problema 179)
from functools import cmp_to_key
def largest_number(nums: list[int]) -> str:
    s = list(map(str, nums))
    s.sort(key=cmp_to_key(lambda a, b: -1 if a + b > b + a else (1 if a + b < b + a else 0)))
    return "0" if s[0] == "0" else "".join(s)

assert largest_number([3, 30, 34, 5, 9]) == "9534330"
```

Estável significa que elementos com chaves iguais mantêm a ordem original. Isso permite ordenar por vários critérios em passadas: primeiro pelo critério secundário, depois pelo principal.

Ordenações simples: $O(n^2)$

São lentas para entradas grandes, mas são o melhor lugar para aprender a raciocinar sobre **invariantes**: o que é verdade sobre o array depois de cada volta do laço.

Bubble Sort

Ideia: percorra o array trocando pares vizinhos fora de ordem. A cada passada, o **maior elemento restante "borbulha" até o fim**.

Invariante: depois da passada `i`, os `i + 1` últimos elementos são os maiores, já na posição final.

```
[5, 1, 4, 2]   passada 1: 5>1 troca, 5>4 troca, 5>2 troca → [1, 4, 2, 5]   (5 fixo)
[1, 4, 2, 5]   passada 2: 1<4 ok,    4>2 troca             → [1, 2, 4, 5]   (4 fixo)
[1, 2, 4, 5]   passada 3: nenhuma troca → já está ordenado, para cedo
```

```python
def bubble_sort(a: list[int]) -> list[int]:
    a = a[:]
    n = len(a)
    for i in range(n - 1):
        trocou = False
        for j in range(n - 1 - i):          # os i últimos já estão no lugar
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                trocou = True
        if not trocou:                      # nenhuma troca: já ordenado
            break
    return a

assert bubble_sort([5, 1, 4, 2]) == [1, 2, 4, 5]
assert bubble_sort([]) == [] and bubble_sort([1]) == [1]
```

Selection Sort

Ideia: para cada posição `i`, **encontre o menor** elemento do trecho não ordenado e coloque-o em `i`.

Invariante: `a[0..i-1]` contém os `i` menores elementos, em ordem.

```
[5, 1, 4, 2]   i=0: menor de [5,1,4,2] é 1 → troca com a[0] → [1, 5, 4, 2]
[1, 5, 4, 2]   i=1: menor de [5,4,2]   é 2 → troca com a[1] → [1, 2, 4, 5]
[1, 2, 4, 5]   i=2: menor de [4,5]     é 4 → já no lugar
```

```python
def selection_sort(a: list[int]) -> list[int]:
    a = a[:]
    n = len(a)
    for i in range(n - 1):
        m = i
        for j in range(i + 1, n):           # acha o índice do menor restante
            if a[j] < a[m]:
                m = j
        a[i], a[m] = a[m], a[i]
    return a

assert selection_sort([5, 1, 4, 2]) == [1, 2, 4, 5]
```

Faz no máximo `n - 1` trocas, o que é útil quando escrever na memória é caro. Mas sempre faz $O(n^2)$ comparações, mesmo com o array já ordenado. Não é estável.

Insertion Sort

Ideia: como ordenar cartas na mão. Pegue o próximo elemento e **deslize-o para a esquerda** até encontrar o lugar certo na parte já ordenada.

Invariante: `a[0..i-1]` está ordenado (mas ainda não contém necessariamente os menores de todos).

```
[5, 1, 4, 2]   pega 1: desliza o 5 → [1, 5, 4, 2]
[1, 5, 4, 2]   pega 4: desliza o 5 → [1, 4, 5, 2]
[1, 4, 5, 2]   pega 2: desliza 5 e 4 → [1, 2, 4, 5]
```

```python
def insertion_sort(a: list[int]) -> list[int]:
    a = a[:]
    for i in range(1, len(a)):
        x = a[i]
        j = i - 1
        while j >= 0 and a[j] > x:          # desloca os maiores uma casa à direita
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = x                        # encaixa x no buraco
    return a

assert insertion_sort([5, 1, 4, 2]) == [1, 2, 4, 5]
assert insertion_sort([1, 2, 3]) == [1, 2, 3]
```

É **O(n) em arrays quase ordenados** (o `while` quase não roda) e é estável. Por isso o Timsort, o algoritmo do `sorted()`, usa insertion sort em trechos pequenos.

	Bubble	Selection	Insertion
Melhor caso	O(n) (com parada antecipada)	O(n²)	O(n)
Pior caso	O(n²)	O(n²)	O(n²)
Trocas no pior caso	O(n²)	O(n)	O(n²) deslocamentos
Estável	sim	não	sim

Merge Sort

Dividir para conquistar: divide ao meio, ordena cada metade recursivamente e **mescla** as duas metades ordenadas em O(n).

```
          [5, 2, 4, 6, 1, 3]
          /                \
     [5, 2, 4]          [6, 1, 3]          dividir
     /      \           /      \
   [5]    [2, 4]       [6]    [1, 3]
           ...                 ...
   [5]    [2, 4]       [6]    [1, 3]        ordenar (recursão)
     \      /           \      /
     [2, 4, 5]          [1, 3, 6]          mesclar
          \                /
          [1, 2, 3, 4, 5, 6]
```

```python
def merge_sort(a: list[int]) -> list[int]:
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left, right = merge_sort(a[:mid]), merge_sort(a[mid:])
    res, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:          # <= mantém a estabilidade
            res.append(left[i]); i += 1
        else:
            res.append(right[j]); j += 1
    res.extend(left[i:]); res.extend(right[j:])
    return res

assert merge_sort([5, 2, 4, 6, 1, 3]) == [1, 2, 3, 4, 5, 6]
```

Complexidade: O(n log n) **sempre** (log n níveis × O(n) por nível), e O(n) de espaço extra.

Contar inversões com merge sort

Uma inversão é um par `i < j` com `a[i] > a[j]`. Durante o merge, quando um elemento da **direita** é menor que `left[i]`, ele também é menor que **todos os restantes da esquerda** (que estão ordenados): são `len(left) - i` inversões de uma vez.

```python
def count_inversions(a: list[int]) -> int:
    def sort(a):
        if len(a) <= 1:
            return a, 0
        mid = len(a) // 2
        left, x = sort(a[:mid])
        right, y = sort(a[mid:])
        res, i, j, inv = [], 0, 0, x + y
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                res.append(left[i]); i += 1
            else:
                res.append(right[j]); j += 1
                inv += len(left) - i         # right[j] < todos os restantes de left
        res.extend(left[i:]); res.extend(right[j:])
        return res, inv
    return sort(a)[1]

assert count_inversions([2, 4, 1, 3, 5]) == 3     # (2,1), (4,1), (4,3)
assert count_inversions([5, 4, 3, 2, 1]) == 10
```

Quick Sort e Quickselect

Partição: escolha um **pivô** e reorganize o array para que os menores fiquem à esquerda e os maiores à direita. O pivô termina na **posição final** que teria no array ordenado.

```python
import random

def partition(a, lo, hi):
    """Lomuto: usa a[hi] como pivô; retorna sua posição final."""
```

```
    pivot = a[hi]
    i = lo                                  # a[lo..i-1] < pivot
    for j in range(lo, hi):
        if a[j] < pivot:
            a[i], a[j] = a[j], a[i]
            i += 1
    a[i], a[hi] = a[hi], a[i]
    return i

def quick_sort(a, lo=0, hi=None):
    if hi is None:
        hi = len(a) - 1
    if lo < hi:
        r = random.randint(lo, hi)          # pivô aleatório evita o pior caso O(n²)
        a[r], a[hi] = a[hi], a[r]
        p = partition(a, lo, hi)
        quick_sort(a, lo, p - 1)
        quick_sort(a, p + 1, hi)
    return a

assert quick_sort([3, 6, 1, 8, 2, 9, 2]) == [1, 2, 2, 3, 6, 8, 9]
```

Complexidade: O(n log n) em média e O(n²) no pior caso (pivô sempre o menor ou o maior; o pivô aleatório torna isso improvável). Espaço O(log n) de pilha, in-place.

Quickselect, o k-ésimo menor em O(n) médio: depois de particionar, o pivô está na posição `p`. Se `p == k`, achou; senão, recurse **só** no lado que contém `k`. A média fica `n + n/2 + n/4 + … ≈ 2n`.

```
import random

def quickselect(a: list[int], k: int) -> int:
    """k-ésimo menor (k base 0)."""
    a = a[:]
    lo, hi = 0, len(a) - 1
    while True:
        r = random.randint(lo, hi)
        a[r], a[hi] = a[hi], a[r]
        p = partition(a, lo, hi)
        if p == k:
            return a[p]
        if p < k:
            lo = p + 1
        else:
            hi = p - 1

nums = [3, 2, 1, 5, 6, 4]
assert quickselect(nums, len(nums) - 2) == 5      # 2º maior = (n-2)-ésimo menor
```

Muitos elementos repetidos

A partição de Lomuto degrada para O(n²) quando quase tudo é igual ao pivô. A solução é a **partição em 3 vias** (menores, iguais, maiores), a mesma ideia do problema *Sort Colors*, abaixo.

Partição em 3 vias: Sort Colors (problema 75)

Ordenar `0`, `1` e `2` em uma passada, in-place (bandeira holandesa). Três regiões: `[0, low)` são 0s, `[low, mid)` são 1s, `(high, fim]` são 2s, e `[mid, high]` é o que falta analisar.

```python
def sort_colors(nums: list[int]) -> None:
    low, mid, high = 0, 0, len(nums) - 1
    while mid <= high:
        if nums[mid] == 0:
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1; mid += 1
        elif nums[mid] == 1:
            mid += 1
        else:
            nums[mid], nums[high] = nums[high], nums[mid]
            high -= 1                    # não avança mid: o valor trocado ainda não foi
visto

a = [2, 0, 2, 1, 1, 0]
sort_colors(a)
assert a == [0, 0, 1, 1, 2, 2]
```

Ordenações lineares (sem comparação)

O limite O(n log n) vale para ordenações **por comparação**. Se os valores são inteiros num intervalo pequeno, dá para fazer melhor.

Counting sort: conte quantas vezes cada valor aparece e reescreva em ordem. O(n + k), com k = intervalo de valores.

```python
def counting_sort(a: list[int], max_val: int) -> list[int]:
    count = [0] * (max_val + 1)
    for x in a:
        count[x] += 1
    res = []
    for v, c in enumerate(count):
        res.extend([v] * c)
    return res

assert counting_sort([4, 2, 2, 8, 3, 3, 1], 8) == [1, 2, 2, 3, 3, 4, 8]
```

Bucket sort distribui os valores em baldes por faixa (como no *Top K Frequent*, em que o índice era a frequência). **Radix sort** ordena dígito por dígito com uma ordenação estável.

Resumo

Algoritmo	Melhor	Médio	Pior	Espaço	Estável
Insertion sort	O(n)	O(n²)	O(n²)	O(1)	sim
Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)	sim

Algoritmo	Melhor	Médio	Pior	Espaço	Estável
Quick sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	não
Heap sort	O(n log n)	O(n log n)	O(n log n)	O(1)	não
Counting sort	O(n + k)	O(n + k)	O(n + k)	O(k)	sim
Timsort (`sorted`)	O(n)	O(n log n)	O(n log n)	O(n)	sim

Prática

Problema	Técnica
912. Sort an Array	merge sort / quick sort
75. Sort Colors	partição em 3 vias
179. Largest Number	comparador customizado
215. Kth Largest Element	quickselect
148. Sort List	merge sort em lista ligada
493. Reverse Pairs	merge sort contando (difícil)
164. Maximum Gap	bucket sort (difícil)

CAPÍTULO 17

Avançado II: grafos e árvores além do básico

Este capítulo aprofunda técnicas que aparecem nos problemas **difíceis** do LeetCode: Prim, BFS 0-1, DP em árvores (incluindo *rerooting*), Segment Tree com propagação preguiçosa, componentes fortemente conexos e pontes. Ele pressupõe os capítulos de Árvores, Grafos, DP e Estruturas avançadas.

Prim: árvore geradora mínima a partir de um vértice

Ideia: comece de um vértice qualquer e **cresça a árvore** uma aresta por vez, sempre pela aresta **mais barata** que liga a árvore a um vértice de fora. É o mesmo argumento do corte do Kruskal: a aresta mais leve que cruza o corte (árvore | resto) está em alguma MST.

Comparando com o Kruskal: o Kruskal olha **todas as arestas** em ordem global e usa Union-Find; o Prim cresce **uma única árvore** e usa um heap, como o Dijkstra. A diferença em relação ao Dijkstra é a prioridade: no Prim ela é **o peso da aresta**, e não a distância acumulada desde a origem.

```python
import heapq

def prim(n: int, adj: list[list[tuple[int, int]]]) -> int:
    """adj[u] = [(v, peso), ...]. Retorna o custo da MST (grafo conexo)."""
    visited = [False] * n
    h = [(0, 0)]                        # (peso da aresta que traz o vértice, vértice)
    total = used = 0
    while h and used < n:
        w, u = heapq.heappop(h)
        if visited[u]:
            continue                    # já entrou por uma aresta mais barata
        visited[u] = True
        total += w
        used += 1
        for v, wt in adj[u]:
            if not visited[v]:
                heapq.heappush(h, (wt, v))
    return total

#   0 --1-- 1
#   | \     |
#   4   3   2
#   |     \ |
#   3 --5-- 2
adj = [[] for _ in range(4)]
for u, v, w in [(0, 1, 1), (1, 2, 2), (0, 2, 3), (0, 3, 4), (2, 3, 5)]:
    adj[u].append((v, w)); adj[v].append((u, w))
assert prim(4, adj) == 7              # arestas 0-1 (1), 1-2 (2), 0-3 (4)
```

Complexidade: O(E log V) com heap.

Prim $O(V^2)$ para grafos densos

Em *Min Cost to Connect All Points* (1584), **todo par** de pontos é uma aresta: $E \approx V^2/2$. Gerar e ordenar todas as arestas (Kruskal) custa $O(V^2 \log V)$. O Prim **sem heap** custa $O(V^2)$: guarde `dist[v]` = aresta mais barata de `v` até a árvore e, a cada passo, escolha o menor com uma varredura linear.

```python
def min_cost_connect_points(points: list[list[int]]) -> int:
    n = len(points)
    INF = float("inf")
    dist = [INF] * n                  # custo para ligar cada ponto à árvore
    dist[0] = 0
    in_tree = [False] * n
    total = 0
    for _ in range(n):
        u = min((i for i in range(n) if not in_tree[i]), key=dist.__getitem__)
        in_tree[u] = True
        total += dist[u]
        ux, uy = points[u]
        for v in range(n):            # u entrou: talvez ofereça arestas melhores
            if not in_tree[v]:
                d = abs(ux - points[v][0]) + abs(uy - points[v][1])
                if d < dist[v]:
                    dist[v] = d
    return total

assert min_cost_connect_points([[0, 0], [2, 2], [3, 10], [5, 2], [7, 0]]) == 20
assert min_cost_connect_points([[3, 12], [-2, 5], [-4, 1]]) == 18
```

BFS 0-1

Problema: caminho mínimo quando os pesos são **apenas 0 ou 1**. O Dijkstra resolve, mas em $O(E \log V)$. Dá para fazer em **$O(V + E)$**.

Ideia: use uma `deque`. Ao relaxar uma aresta de peso **0**, o vizinho tem a **mesma** distância do vértice atual, então coloque-o **na frente**. Aresta de peso **1**: coloque **no fim**. Assim a deque fica ordenada por distância, como a fila de prioridade do Dijkstra, mas sem heap.

```
deque sempre na forma:  [ d, d, d, ..., d+1, d+1 ]
                          ▲ peso 0 entra aqui     ▲ peso 1 entra aqui
```

Minimum Cost to Make at Least One Valid Path (problema 1368), difícil

Cada célula tem uma seta (1 = direita, 2 = esquerda, 3 = baixo, 4 = cima). Seguir a seta custa 0; ir em outra direção (ou seja, "mudar a seta") custa 1. Qual o custo mínimo de `(0, 0)` até o canto inferior direito?

```python
from collections import deque

def min_cost(grid: list[list[int]]) -> int:
    R, C = len(grid), len(grid[0])
    DIRS = {1: (0, 1), 2: (0, -1), 3: (1, 0), 4: (-1, 0)}
```

```
    INF = float("inf")
    dist = [[INF] * C for _ in range(R)]
    dist[0][0] = 0
    dq = deque([(0, 0)])
    while dq:
        r, c = dq.popleft()
        d = dist[r][c]
        for seta, (dr, dc) in DIRS.items():
            nr, nc = r + dr, c + dc
            if not (0 <= nr < R and 0 <= nc < C):
                continue
            w = 0 if grid[r][c] == seta else 1       # seguir a seta é grátis
            if d + w < dist[nr][nc]:
                dist[nr][nc] = d + w
                if w == 0:
                    dq.appendleft((nr, nc))          # mesma distância: frente
                else:
                    dq.append((nr, nc))              # distância + 1: fim
    return dist[R - 1][C - 1]

assert min_cost([[1, 1, 1, 1], [2, 2, 2, 2], [1, 1, 1, 1], [2, 2, 2, 2]]) == 3
assert min_cost([[1, 1, 3], [3, 2, 2], [1, 1, 4]]) == 0
assert min_cost([[1, 2], [4, 3]]) == 1
```

O mesmo molde resolve *Minimum Obstacle Removal to Reach Corner* (2290): célula livre custa 0, obstáculo custa 1.

DP em árvores

A recursão em árvores (capítulo de Árvores) **já é** uma DP: cada subárvore é um subproblema, e a pós-ordem é a ordem de cálculo. A diferença nos problemas difíceis é que a função precisa devolver **mais de um valor**, um para cada "estado" do nó.

House Robber III (problema 337)

Casas formam uma árvore; não dá para roubar pai e filho juntos. Para cada nó, devolva **dois** valores:

- `com`: melhor total da subárvore **roubando** este nó. Então os filhos não podem ser roubados: `val + sem(esq) + sem(dir)`.
- `sem`: melhor total **sem** roubar este nó. Cada filho fica livre: `max(esq) + max(dir)`.

```python
def rob_tree(root) -> int:
    def dfs(node):                          # retorna (com, sem)
        if not node:
            return 0, 0
        lc, ls = dfs(node.left)
        rc, rs = dfs(node.right)
        com = node.val + ls + rs
        sem = max(lc, ls) + max(rc, rs)
        return com, sem
    return max(dfs(root))

assert rob_tree(build_tree([3, 2, 3, None, 3, None, 1])) == 7      # 3 + 3 + 1
assert rob_tree(build_tree([3, 4, 5, 1, 3, None, 1])) == 9         # 4 + 5
```

Por que a versão ingênua é exponencial

`rob(node) = max(node.val + rob(netos), rob(filhos))` recalcula os netos muitas vezes, porque eles aparecem pelo avô e pelo pai. Devolver os dois estados de uma vez faz cada nó ser visitado **uma única vez**: O(n).

Binary Tree Cameras (problema 968), difícil: três estados

Câmeras num nó vigiam o nó, o pai e os filhos. Mínimo de câmeras para vigiar todos. Estados devolvidos para o pai:

- `0`: **não vigiado** (o pai precisa pôr câmera);
- `1`: **tem câmera**;
- `2`: **vigiado sem câmera**.

Guloso de baixo para cima: **nunca ponha câmera numa folha**; é sempre melhor pôr no pai dela, que cobre mais nós. Um `None` conta como "vigiado" (estado 2), para não forçar câmeras.

```python
def min_camera_cover(root) -> int:
    cams = 0
    def dfs(node):
        nonlocal cams
        if not node:
            return 2                        # nulo: não precisa de vigilância
        l, r = dfs(node.left), dfs(node.right)
        if l == 0 or r == 0:                # algum filho descoberto: câmera aqui
            cams += 1
            return 1
        if l == 1 or r == 1:                # um filho tem câmera: estou vigiado
            return 2
        return 0                            # filhos vigiados sem câmera: estou descoberto
    if dfs(root) == 0:                      # a raiz ficou descoberta
        cams += 1
    return cams

assert min_camera_cover(build_tree([0, 0, None, 0, 0])) == 1
assert min_camera_cover(build_tree([0, 0, None, 0, None, 0, None, None, 0])) == 2
```

Rerooting: Sum of Distances in Tree (problema 834), difícil

Para **cada** nó de uma árvore (não binária, dada por arestas), calcule a soma das distâncias até todos os outros. Rodar uma BFS por nó custa O(n²). Com *rerooting*, o custo é **O(n)**.

Ideia em duas passadas:

1. **Pós-ordem** a partir da raiz 0: calcule `count[u]` (tamanho da subárvore de `u`) e `ans[0]` (soma das profundidades).
2. **Pré-ordem**: derive a resposta de cada filho a partir da do pai. Ao mover a raiz de `p` para seu filho `c`:
   - os `count[c]` nós da subárvore de `c` ficam **1 mais perto**;
   - os outros `n - count[c]` nós ficam **1 mais longe**.

   Logo: `ans[c] = ans[p] - count[c] + (n - count[c])`.

```
      0              ans[0] = 1+1+2+2+2 = 8
     / \             mover a raiz de 0 para 2 (count[2] = 4):
    1   2              4 nós (2,3,4,5) ficam 1 mais perto  → -4
       /|\             2 nós (0,1) ficam 1 mais longe      → +2
      3 4 5          ans[2] = 8 - 4 + 2 = 6
```

```python
def sum_of_distances_in_tree(n: int, edges: list[list[int]]) -> list[int]:
    adj = [[] for _ in range(n)]
    for u, v in edges:
        adj[u].append(v); adj[v].append(u)

    # ordem de visita (pré-ordem) iterativa, guardando o pai: evita recursão profunda
    parent = [-1] * n
    order, st, seen = [], [0], [False] * n
    seen[0] = True
    while st:
        u = st.pop()
        order.append(u)
        for v in adj[u]:
            if not seen[v]:
                seen[v] = True
                parent[v] = u
                st.append(v)

    count = [1] * n
    depth_sum = [0] * n                 # soma das distâncias de u até sua subárvore
    for u in reversed(order):           # pós-ordem: filhos antes do pai
        p = parent[u]
        if p != -1:
            count[p] += count[u]
            depth_sum[p] += depth_sum[u] + count[u]

    ans = [0] * n
    ans[0] = depth_sum[0]
    for u in order:                     # pré-ordem: pai antes dos filhos
        p = parent[u]
        if p != -1:
            ans[u] = ans[p] - count[u] + (n - count[u])
    return ans
```

```
assert sum_of_distances_in_tree(6, [[0, 1], [0, 2], [2, 3], [2, 4], [2, 5]]) == [8, 12, 6,
10, 10, 10]
assert sum_of_distances_in_tree(1, []) == [0]
```

A linha `depth_sum[p] += depth_sum[u] + count[u]` diz que todos os `count[u]` nós da subárvore de `u` ficam 1 mais longe quando vistos a partir de `p`.

Quando pensar em rerooting

"Calcule X **para cada nó como se fosse a raiz**." Faça uma passada de baixo para cima com a raiz fixa e outra de cima para baixo, ajustando a resposta ao mover a raiz para um filho.

Segment Tree com propagação preguiçosa (lazy)

A Segment Tree do capítulo de Estruturas avançadas atualiza **uma posição** por vez. E se a operação for "**some 5 em todo o intervalo `[l, r]`**"? Atualizar cada folha custaria O(n) por operação.

Ideia: quando um nó cobre um segmento **inteiramente dentro** da atualização, atualize **só ele** e anote no campo `lazy` "devo `+v` aos meus filhos". A dívida só é repassada (*push*) quando alguém precisar **descer** por aquele nó. Cada operação continua O(log n).

```
add(0, 3, +5) num array de 8 posições:

            [0..7]                 o nó [0..3] está inteiro dentro de [0..3]:
           /      \                  sum[0..3] += 5 × 4
      [0..3]*    [4..7]             lazy[0..3] += 5      ← dívida com os filhos
      /    \                       os filhos [0..1], [2..3] NÃO são tocados agora
  [0..1]  [2..3]
```

```
class LazySegTree:
    """Soma em intervalo + adição em intervalo, ambos O(log n)."""
    def __init__(self, data: list[int]):
        self.n = len(data)
        self.sum = [0] * (4 * self.n)
        self.lazy = [0] * (4 * self.n)
        self._build(data, 1, 0, self.n - 1)

    def _build(self, data, node, l, r):
        if l == r:
            self.sum[node] = data[l]
            return
        m = (l + r) // 2
        self._build(data, 2 * node, l, m)
        self._build(data, 2 * node + 1, m + 1, r)
        self.sum[node] = self.sum[2 * node] + self.sum[2 * node + 1]

    def _apply(self, node, l, r, v):      # soma v em todo o segmento do nó
        self.sum[node] += v * (r - l + 1)
        self.lazy[node] += v
```

```
    def _push(self, node, l, r):          # repassa a dívida aos filhos
        if self.lazy[node]:
            m = (l + r) // 2
            self._apply(2 * node, l, m, self.lazy[node])
            self._apply(2 * node + 1, m + 1, r, self.lazy[node])
            self.lazy[node] = 0

    def add(self, ql, qr, v, node=1, l=0, r=None):
        if r is None:
            r = self.n - 1
        if qr < l or r < ql:              # totalmente fora
            return
        if ql <= l and r <= qr:           # totalmente dentro: anota e para
            self._apply(node, l, r, v)
            return
        self._push(node, l, r)            # parcial: desce, então paga a dívida
        m = (l + r) // 2
        self.add(ql, qr, v, 2 * node, l, m)
        self.add(ql, qr, v, 2 * node + 1, m + 1, r)
        self.sum[node] = self.sum[2 * node] + self.sum[2 * node + 1]

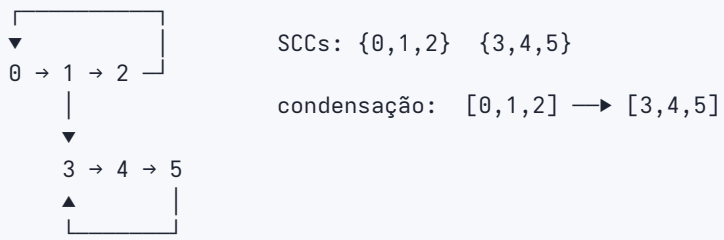
    def query(self, ql, qr, node=1, l=0, r=None):
        if r is None:
            r = self.n - 1
        if qr < l or r < ql:
            return 0
        if ql <= l and r <= qr:
            return self.sum[node]
        self._push(node, l, r)
        m = (l + r) // 2
        return self.query(ql, qr, 2 * node, l, m) + self.query(ql, qr, 2 * node + 1, m + 1,
r)

# teste contra força bruta com operações aleatórias
import random
rng = random.Random(42)
base = [rng.randint(-10, 10) for _ in range(50)]
st, ref = LazySegTree(base), base[:]
for _ in range(2000):
    l = rng.randrange(50); r = rng.randrange(l, 50)
    if rng.random() < 0.5:
        v = rng.randint(-5, 5)
        st.add(l, r, v)
        for i in range(l, r + 1):
            ref[i] += v
    else:
        assert st.query(l, r) == sum(ref[l:r + 1])
```

Trocando `_apply` e a combinação dos filhos, o mesmo esqueleto serve para "atribuir valor em intervalo + mínimo em intervalo", "inverter bits em intervalo + contar 1s", e assim por diante. A regra é sempre a mesma: o nó guarda o **resumo do segmento** e a **operação pendente** para os filhos.

Componentes fortemente conexos (SCC)

Num grafo **direcionado**, um SCC é um conjunto máximo de vértices em que **todo vértice alcança todos os outros**. Colapsando cada SCC num único vértice, o grafo vira um **DAG** (a *condensação*), em que topológica e DP voltam a funcionar.



Kosaraju: duas DFS

1. Faça DFS no grafo e anote a **ordem de término** (o momento em que cada vértice termina).
2. **Inverta todas as arestas**. Processe os vértices em ordem **decrescente de término**; cada DFS no grafo invertido marca exatamente um SCC.

Por que funciona: o vértice que termina por último está num SCC "fonte" da condensação. No grafo invertido, esse SCC vira "sorvedouro": a DFS a partir dele não consegue sair, então pega só o próprio SCC. Removido esse, o argumento se repete.

```python
def kosaraju(n: int, edges: list[tuple[int, int]]) -> list[list[int]]:
    g = [[] for _ in range(n)]
    rg = [[] for _ in range(n)]
    for u, v in edges:
        g[u].append(v)
        rg[v].append(u)

    # 1ª passada: ordem de término (DFS iterativa com índice do próximo vizinho)
    visited, finish = [False] * n, []
    for s in range(n):
        if visited[s]:
            continue
        visited[s] = True
        st = [(s, 0)]
        while st:
            u, i = st.pop()
            if i < len(g[u]):
                st.append((u, i + 1))            # volta a u depois deste vizinho
                v = g[u][i]
                if not visited[v]:
                    visited[v] = True
                    st.append((v, 0))
            else:
                finish.append(u)                 # todos os vizinhos de u terminaram

    # 2ª passada: grafo invertido, em ordem decrescente de término
    comp = [-1] * n
    sccs = []
    for s in reversed(finish):
```

```
        if comp[s] != -1:
            continue
        comp[s] = len(sccs)
        grupo, st = [], [s]
        while st:
            u = st.pop()
            grupo.append(u)
            for v in rg[u]:
                if comp[v] == -1:
                    comp[v] = comp[s]
                    st.append(v)
        sccs.append(sorted(grupo))
    return sccs

E = [(0, 1), (1, 2), (2, 0), (1, 3), (3, 4), (4, 5), (5, 3)]
assert sorted(kosaraju(6, E)) == [[0, 1, 2], [3, 4, 5]]
assert sorted(kosaraju(3, [(0, 1), (1, 2)])) == [[0], [1], [2]]   # DAG: cada vértice sozinho
```

Complexidade: O(V + E). (O algoritmo de **Tarjan** faz o mesmo com uma só DFS usando *low-link*, a mesma ideia da seção seguinte.)

Pontes: Critical Connections (problema 1192), difícil

Uma **ponte** é uma aresta que, se removida, **desconecta** o grafo (não direcionado). Ferramenta: DFS com dois números por vértice.

- `disc[u]`: o momento em que a DFS **descobriu** `u`.
- `low[u]`: o **menor** `disc` alcançável a partir da subárvore de `u` usando no máximo **uma aresta de volta** (que não seja a aresta para o pai).

A aresta de árvore `u → v` é ponte se **`low[v] > disc[u]`**: nada na subárvore de `v` consegue voltar até `u` ou acima sem passar por essa aresta.

```
  0 ─── 1 ─── 3        disc:  0→0, 1→1, 2→2, 3→3
   \   /               low[2] = 0 (2 volta direto ao 0)
     2                 low[1] = 0      low[3] = 3
                       aresta 1-3: low[3]=3 > disc[1]=1 → PONTE
                       aresta 1-2: low[2]=0 ≤ disc[1]=1 → não é ponte
```

```
import sys

def critical_connections(n: int, connections: list[list[int]]) -> list[list[int]]:
    adj = [[] for _ in range(n)]
    for u, v in connections:
        adj[u].append(v); adj[v].append(u)
    disc = [-1] * n
    low = [0] * n
    timer = 0
    bridges = []
    sys.setrecursionlimit(max(1000, 2 * n + 10))

    def dfs(u, parent):
        nonlocal timer
```

```
        disc[u] = low[u] = timer
        timer += 1
        for v in adj[u]:
            if v == parent:
                continue                        # não usa a aresta de volta ao pai
            if disc[v] == -1:                   # aresta de árvore
                dfs(v, u)
                low[u] = min(low[u], low[v])
                if low[v] > disc[u]:
                    bridges.append([u, v])
            else:                               # aresta de volta
                low[u] = min(low[u], disc[v])

    for s in range(n):
        if disc[s] == -1:
            dfs(s, -1)
    return bridges

res = critical_connections(4, [[0, 1], [1, 2], [2, 0], [1, 3]])
assert sorted(map(sorted, res)) == [[1, 3]]
assert critical_connections(2, [[0, 1]]) == [[0, 1]]
```

Trocando a condição para `low[v] >= disc[u]` (com tratamento especial da raiz), obtém-se os **pontos de articulação**: os vértices cuja remoção desconecta o grafo.

Estruturas ordenadas (o substituto das árvores balanceadas)

Árvores AVL e rubro-negras mantêm uma BST com altura O(log n), mas implementá-las numa entrevista é raro. O que importa é saber **quando** você precisa de uma: inserir, remover **e** consultar "o menor ≥ x", "o k-ésimo" ou "quantos < x", tudo em O(log n), com dados mudando.

No LeetCode, o módulo `sortedcontainers` está disponível:

```
from sortedcontainers import SortedList

sl = SortedList([5, 1, 3])
sl.add(4)                 # O(log n)
sl.remove(1)              # O(log n)
sl[0], sl[-1]             # menor e maior
sl.bisect_left(4)         # quantos < 4
sl[sl.bisect_left(4)]     # menor elemento >= 4
```

Problemas em que isso destrava a solução: *Contains Duplicate III* (220), *Sliding Window Median* (480), *Count of Range Sum* (327), *My Calendar* (729). Sem `sortedcontainers`, as alternativas são: dois heaps com remoção preguiçosa, Fenwick com compressão de coordenadas, ou `bisect.insort` numa lista (O(n) por inserção, aceitável para n pequeno).

Prática

Problema	Técnica
1584. Min Cost to Connect All Points	Prim $O(V^2)$
1368. Minimum Cost to Make at Least One Valid Path	BFS 0-1 (difícil)
2290. Minimum Obstacle Removal to Reach Corner	BFS 0-1 (difícil)
337. House Robber III	DP em árvore (2 estados)
968. Binary Tree Cameras	DP / guloso em árvore (difícil)
834. Sum of Distances in Tree	rerooting (difícil)
2385. Amount of Time for Binary Tree to Be Infected	árvore como grafo + BFS
699. Falling Squares	segment tree com lazy + compressão (difícil)
1192. Critical Connections in a Network	pontes (difícil)
480. Sliding Window Median	estrutura ordenada / dois heaps (difícil)

CAPÍTULO 18

Método para resolver e explicar problemas

Saber as técnicas não basta: diante de um problema novo, você precisa **escolher** a técnica e **justificar** a escolha. Este capítulo é o roteiro.

O roteiro em 6 passos

1. Entender (2 a 5 minutos)

- Reescreva o problema **com suas palavras**. Qual é a entrada? Qual é a saída exata?
- Leia as **restrições**: tamanho de `n`, intervalo dos valores (há negativos? zeros?), duplicatas? já vem ordenado?
- Faça **2 exemplos à mão**, incluindo um caso de borda (vazio, um elemento, todos iguais, valores negativos).

2. Força bruta primeiro

Descreva a solução mais óbvia, mesmo que lenta, e diga a complexidade dela. Isso (a) garante que você entendeu o problema; (b) dá uma resposta de reserva; (c) mostra **onde está o desperdício** que a solução ótima vai eliminar.

3. Otimizar: perguntas que destravam

Pergunta	Leva a
Estou recalculando a mesma coisa?	hash, prefix sum, memoização (DP)
Estou procurando algo repetidamente numa lista?	`set` / `dict` (O(1))
Ordenar ajudaria?	two pointers, busca binária, guloso, intervalos
A resposta é monótona ("se x serve, x+1 também")?	busca binária na resposta
Há uma janela contígua?	sliding window
Preciso do maior ou menor **atual** repetidamente?	heap
Preciso do "próximo maior ou menor"?	pilha monotônica
Há relações entre elementos (dependência, vizinhança)?	grafo: BFS, DFS, topológica, Union-Find
Preciso de **todas** as soluções?	backtracking
Escolhas em sequência com sobreposição?	DP

4. Validar a ideia antes de codar

Rode o algoritmo **à mão** num exemplo pequeno. Diga a complexidade de tempo e de espaço. Se for guloso, procure um contraexemplo. Só então escreva código.

5. Codar limpo

- Nomes claros (`left`, `right`, `window_count`, e não `a1`, `x2`).
- Funções auxiliares para partes independentes (`ok(x)`, `neighbors(r, c)`).
- Trate os casos de borda **no início**.

6. Testar como um revisor

Rastreie o código (não a ideia) linha a linha num exemplo. Depois teste: entrada vazia, um elemento, todos iguais, valores extremos, a resposta no começo ou no fim.

Tabela: sinal no enunciado → técnica

Sinal no enunciado	Técnica provável	Capítulo
array **ordenado** + par ou trinca	two pointers	Arrays
subarray/substring contígua + "mais longa / mais curta / no máximo K"	sliding window	Arrays
soma de subarray com **negativos** / "contar subarrays com soma k"	prefix sum + hash	Arrays
"já vi isso antes?", complemento, frequências	hash map / set	Hash
parênteses, aninhamento, desfazer	pilha	Pilhas
"próximo maior / menor", histograma	pilha monotônica	Pilhas
máximo de cada janela	deque monotônico	Pilhas
ordenado / "mínimo valor tal que é possível" / n até 10^9	busca binária	Busca binária
"todas as combinações / permutações / subconjuntos"	backtracking	Backtracking
árvore: altura, caminho, subárvore	DFS recursiva (pós-ordem)	Árvores
árvore: "por nível", "mais próximo da raiz"	BFS	Árvores
BST	em ordem = ordenado; limites `(lo, hi)`	Árvores
"K maiores / menores / mais frequentes", mediana em fluxo	heap	Heaps
grade, ilhas, regiões	DFS / BFS em grade	Grafos

Sinal no enunciado	Técnica provável	Capítulo
menor número de passos sem peso	BFS	Grafos
dependências, ordem de tarefas	ordenação topológica	Grafos
conectividade, "estão no mesmo grupo?"	Union-Find	Grafos
menor custo com pesos	Dijkstra (≥ 0) / Bellman-Ford	Grafos
"número de maneiras", "mínimo / máximo" com escolhas	programação dinâmica	DP
duas strings comparadas	DP 2D `dp[i][j]`	DP
intervalos (reuniões, fusões)	ordenar + guloso / sweep line	Gulosos
prefixos de palavras	trie	Avançadas
consultas de intervalo + atualizações	Fenwick / Segment Tree	Avançadas
"aparece uma vez, os outros duas"	XOR	Bits
n ≤ 20 e "conjunto de usados"	bitmask (backtracking ou DP)	DP / Bits

Como explicar uma solução

Uma boa explicação segue sempre esta estrutura e cabe em 1 a 2 minutos:

1. **Observação-chave**: a frase que torna o problema fácil. *"Como o array está ordenado, se a soma é pequena demais, o elemento da esquerda nunca faz par com ninguém."*
2. **Algoritmo**: os passos em alto nível, sem ler o código.
3. **Por que está correto**: o invariante, ou o argumento de troca.
4. **Complexidade**: tempo e espaço, **com o motivo**. *"O(n), porque cada índice entra e sai da janela no máximo uma vez."*
5. **Casos de borda**: quais você tratou e como.

Exemplo completo: Longest Increasing Path in a Matrix (problema 329)

> Dada uma matriz `m × n` de inteiros, retorne o comprimento do maior caminho **estritamente crescente**, movendo-se nas 4 direções.

Passo 1: entender

- Entrada: grade com até 200 × 200 = 40.000 células. Saída: um inteiro.

- O caminho pode começar em qualquer célula. "Estritamente crescente": vizinhos iguais não podem ser seguidos.
- Exemplo: `[[9,9,4],[6,6,8],[2,1,1]]` dá 4 (o caminho `1 → 2 → 6 → 9`).

Passo 2: força bruta

De cada célula, uma DFS que tenta todos os vizinhos maiores e devolve o maior caminho. Sem memória, cada DFS pode explorar um número exponencial de caminhos. **Desperdício:** o maior caminho *a partir de uma célula* é calculado várias vezes, uma para cada caminho que passa por ela.

Passo 3: otimizar

Observação-chave: o maior caminho a partir de `(r, c)` **não depende de como chegamos lá**. E, como o caminho é estritamente crescente, **não existe ciclo**: o grafo "célula → vizinho maior" é um DAG. Logo:

`best(r, c) = 1 + max(best(vizinho))` sobre os vizinhos com valor maior.

Isso é uma DP sobre um DAG, e a memoização garante que cada célula é calculada **uma vez**.

> **Por que não precisamos de `visited` aqui?**
>
> Em DFS comum, `visited` evita ciclos. Aqui o caminho é estritamente crescente: nunca voltamos a uma célula do caminho atual, porque ela tem valor menor. A ausência de ciclos é **exatamente** o que permite a memoização. Com ciclos, `best` de uma célula dependeria do caminho até ela.

Passo 4: validar à mão

Para `[[9,9,4],[6,6,8],[2,1,1]]`: `best(9) = 1` (sem vizinho maior); `best(6 em (1,0)) = 1 + best(9) = 2`; `best(2) = 1 + best(6) = 3`; `best(1 em (2,1)) = 1 + best(2) = 4`. A resposta é 4. ✔

Passo 5: código

```python
from functools import cache

def longest_increasing_path(matrix: list[list[int]]) -> int:
    R, C = len(matrix), len(matrix[0])

    @cache
    def best(r: int, c: int) -> int:
        """Maior caminho crescente que COMEÇA em (r, c)."""
        res = 1
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < R and 0 <= nc < C and matrix[nr][nc] > matrix[r][c]:
                res = max(res, 1 + best(nr, nc))
        return res

    return max(best(r, c) for r in range(R) for c in range(C))
```

```python
assert longest_increasing_path([[9, 9, 4], [6, 6, 8], [2, 1, 1]]) == 4
assert longest_increasing_path([[3, 4, 5], [3, 2, 6], [2, 2, 1]]) == 4
assert longest_increasing_path([[1]]) == 1
```

Passo 6: testar e explicar

- Uma célula: 1 ✔. Todos iguais: 1 ✔ (nenhum vizinho estritamente maior).
- Profundidade de recursão: até R · C = 40.000 no pior caso (um caminho em "cobra"). Isso pode estourar o limite do Python. Mitigação: `sys.setrecursionlimit`, ou a versão iterativa. Ordene as células por valor **decrescente** e calcule `best` nessa ordem, porque os vizinhos maiores já estarão prontos. É a ordem topológica do DAG.

```python
def longest_increasing_path_iter(matrix: list[list[int]]) -> int:
    R, C = len(matrix), len(matrix[0])
    cells = sorted(((matrix[r][c], r, c) for r in range(R) for c in range(C)), reverse=True)
    best = [[1] * C for _ in range(R)]
    for v, r, c in cells:                       # do maior valor para o menor
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < R and 0 <= nc < C and matrix[nr][nc] > v:
                best[r][c] = max(best[r][c], 1 + best[nr][nc])
    return max(map(max, best))

assert longest_increasing_path_iter([[9, 9, 4], [6, 6, 8], [2, 1, 1]]) == 4
assert longest_increasing_path_iter([[3, 4, 5], [3, 2, 6], [2, 2, 1]]) == 4
```

A explicação final (como seria falada):

> "A observação-chave é que, como o caminho é estritamente crescente, o grafo de movimentos não tem ciclos. Então o maior caminho a partir de uma célula depende só dela, e não de como chegamos. Defino `best(r, c)` como 1 mais o maior `best` entre os vizinhos maiores, e memoizo. Cada célula é calculada uma vez e olha 4 vizinhos: tempo O(R·C) e espaço O(R·C) para o cache e a pilha. Para evitar recursão profunda, posso processar as células em ordem decrescente de valor, que é uma ordem topológica."

Repare que essa solução **combinou** três capítulos: grade como grafo (Grafos), DAG e ordem topológica (Grafos) e memoização (DP). Problemas difíceis raramente exigem uma técnica nova; eles exigem **reconhecer e combinar** técnicas conhecidas.

Checklist de bugs antes de enviar

Revise sempre

- **Índices:** `range(n)` vs `range(n + 1)`; `R` inclusivo ou exclusivo; `mid ± 1` na busca binária.
- **Vazio e `None`:** `nums = []`, `root = None`, `head.next` quando `head` é o último.
- **Referências:** `path[:]` ao salvar; `[[0] * C for _ in range(R)]`, e nunca `[[0] * C] * R`.
- **Visitado:** marcado ao **enfileirar** na BFS; restaurado no backtracking.
- **Inicialização:** mínimo começa em `inf`, máximo em `-inf` (ou no primeiro elemento), contagem em 0.
- **Laço infinito:** o ponteiro sempre avança? O intervalo sempre encolhe?
- **Overflow e precisão:** `ceil` com inteiros; `% MOD` a cada passo; não comparar `float` com `==`.
- **Mutação da entrada:** o problema permite modificar a entrada? (Às vezes os testes reusam o objeto.)

Plano de estudo sugerido (8 semanas)

Semana	Tema	Meta
1	Fundamentos, arrays, hash	10 problemas fáceis ou médios; dominar Big-O
2	Two pointers, sliding window, prefix sum	3, 11, 15, 76, 560
3	Pilhas, filas, busca binária	20, 739, 84, 875, 33, 4
4	Listas ligadas, backtracking	206, 143, 146, 39, 51, 79
5	Árvores e heaps	102, 98, 236, 124, 295, 23
6	Grafos	200, 207, 684, 743, 787, 127
7	Programação dinâmica	198, 322, 300, 1143, 72, 312
8	Gulosos, intervalos, avançadas e revisão	56, 435, 253, 212, 315 + refazer os que errou

A regra de ouro do estudo

Se você ficou 30 a 40 minutos sem progresso, leia **só a ideia** da solução (não o código), feche e implemente sozinho. Anote a **observação-chave** num caderno de padrões. Refaça o problema do zero 3 dias depois. Seu objetivo não é ter resolvido muitos problemas, é **reconhecer o padrão** no próximo.

Referência rápida de Python para algoritmos

```
from collections import deque, defaultdict, Counter
from heapq import heappush, heappop, heapify
from bisect import bisect_left, bisect_right
from functools import cache
from itertools import accumulate, combinations, permutations, product

INF = float("inf")
grid = [[0] * 3 for _ in range(2)]                  # matriz 2x3 correta
assert list(accumulate([1, 2, 3])) == [1, 3, 6]     # prefix sum
assert list(combinations([1, 2, 3], 2)) == [(1, 2), (1, 3), (2, 3)]
assert len(list(permutations([1, 2, 3]))) == 6
assert list(product("ab", repeat=2)) == [("a", "a"), ("a", "b"), ("b", "a"), ("b", "b")]
assert divmod(17, 5) == (3, 2)                      # quociente e resto
assert ord("c") - ord("a") == 2 and chr(ord("a") + 2) == "c"
assert sorted("bca") == ["a", "b", "c"] and "".join(sorted("bca")) == "abc"
assert max([], default=0) == 0                      # max de lista vazia sem erro
assert [x for x in range(10) if x % 3 == 0] == [0, 3, 6, 9]
assert {v: i for i, v in enumerate("abc")} == {"a": 0, "b": 1, "c": 2}
```